

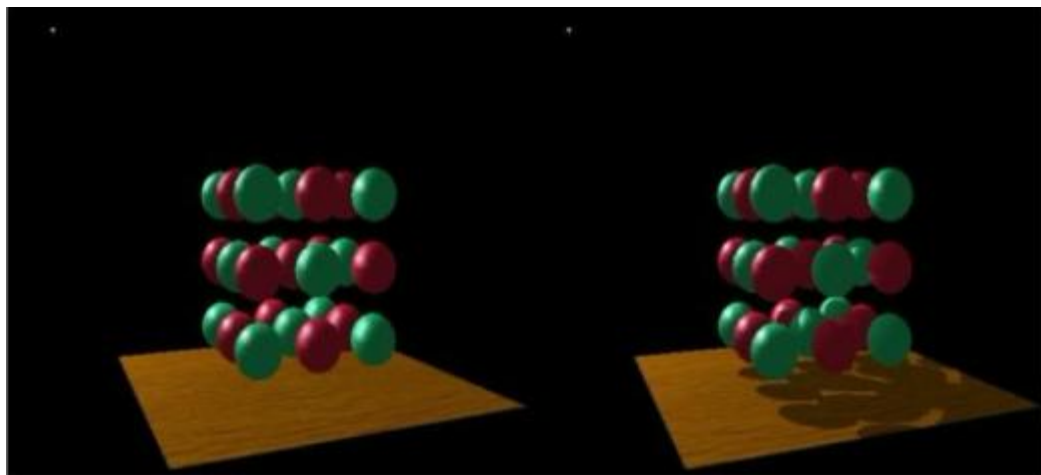
阴影和纹理

高林

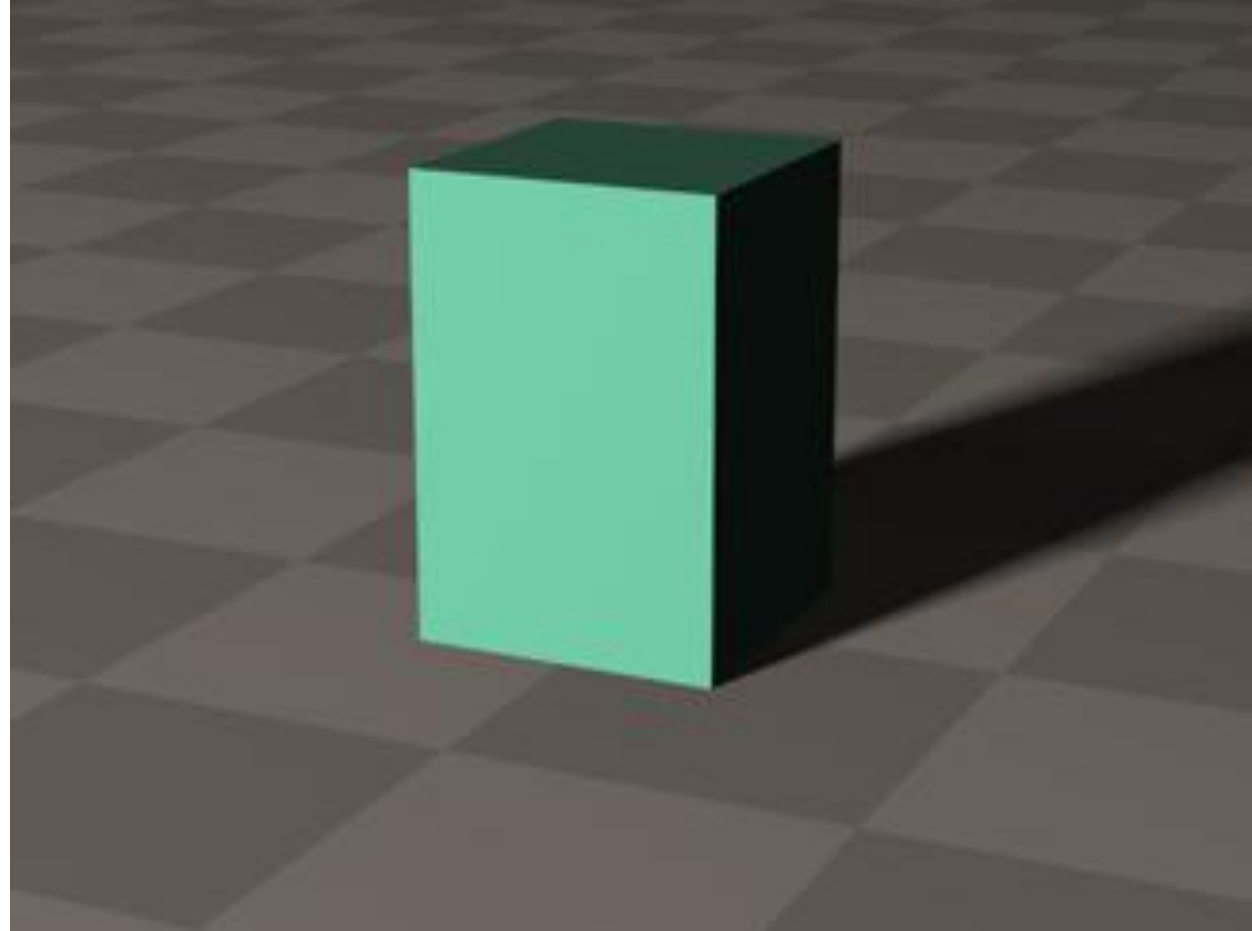
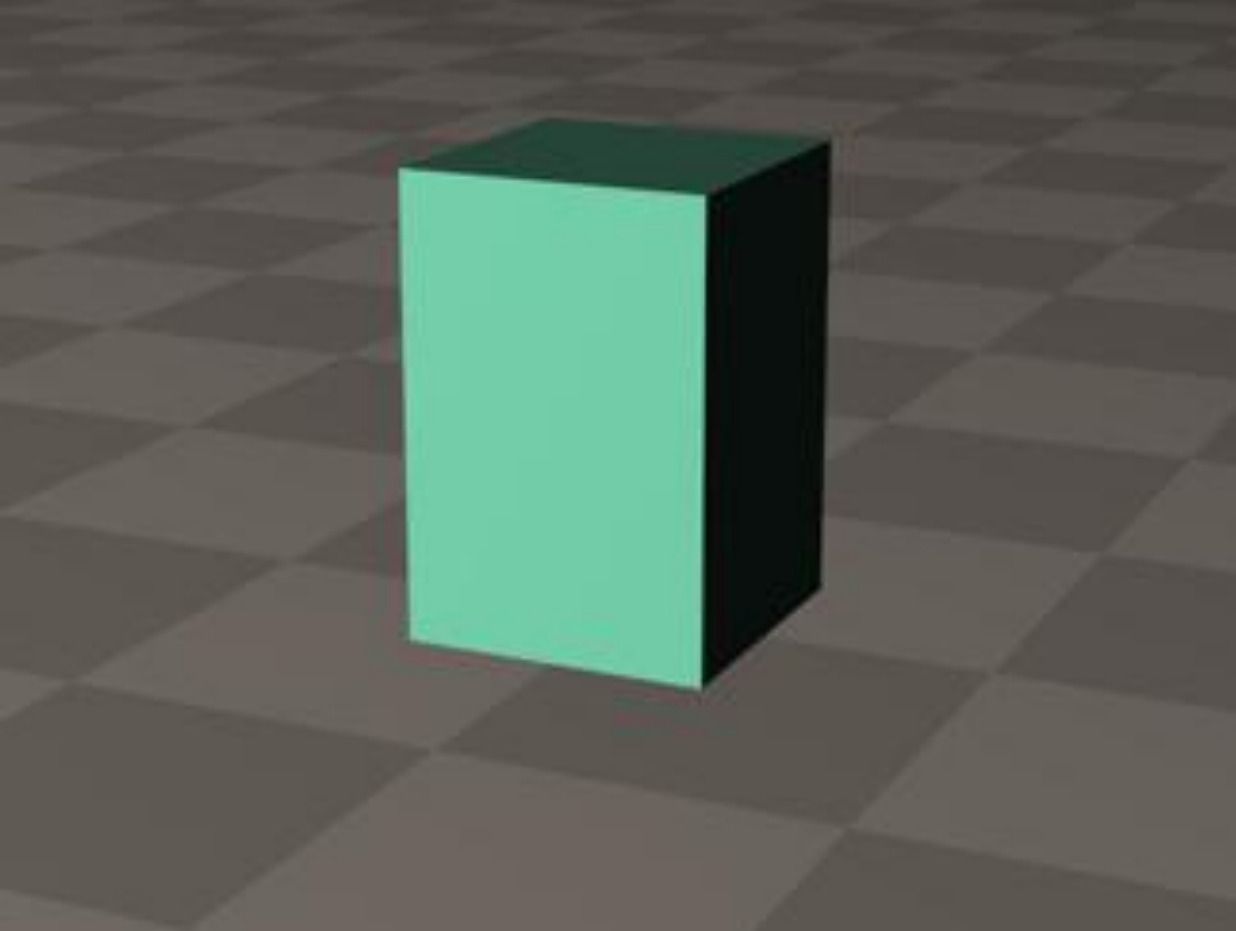
中国科学院计算技术研究所

阴影 (Shadow)

- 阴影是在构建真实感图像时的一个十分重要的元素，它可以为物体的位置和摆放提供视觉上的提示。
- 为什么阴影如此重要？让我们来看两个例子。

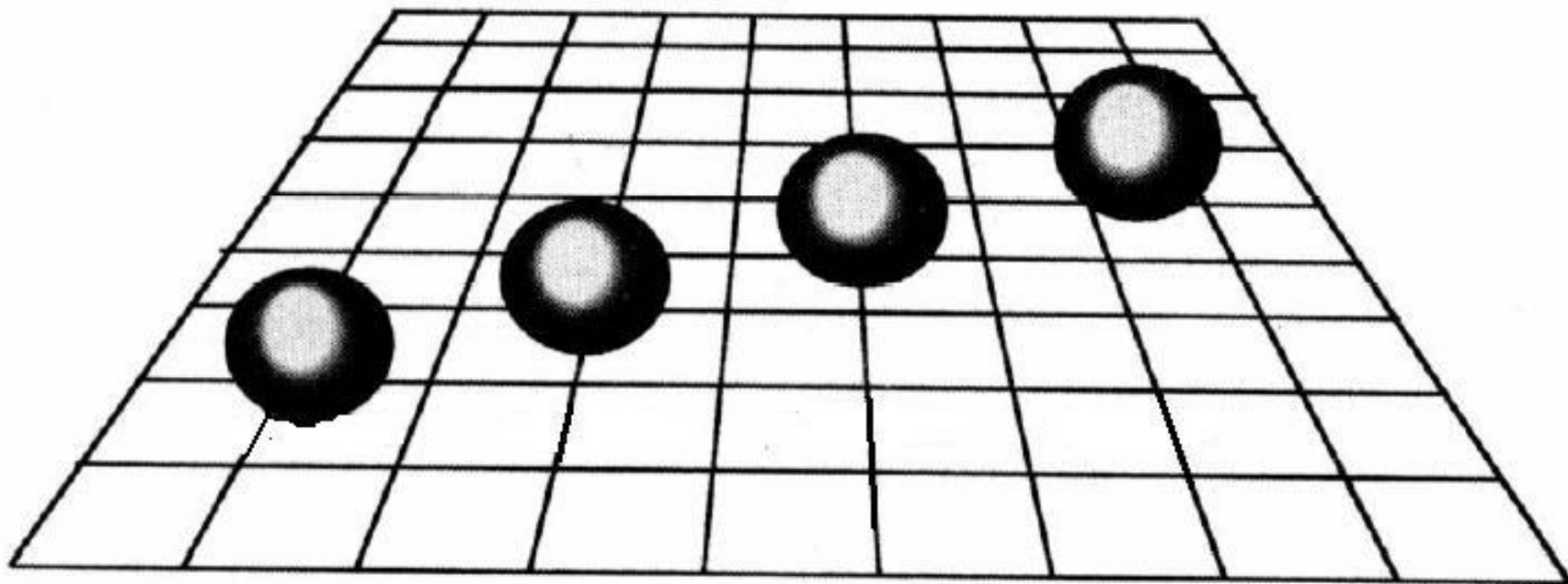


阴影 (Shadow)



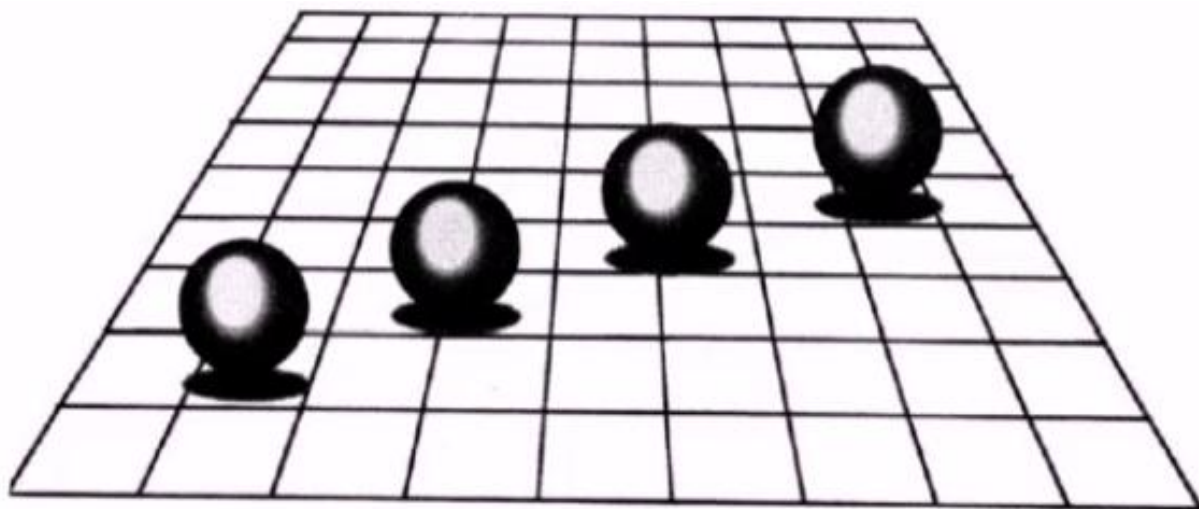
为什么阴影如此重要？

- 在没有阴影的提示下，你能知道下图中的小球的空间位置吗？很难确定小球的实际位置。有了阴影的提示，我们可以清楚地知道小球的位置。



阴影 (Shadow)

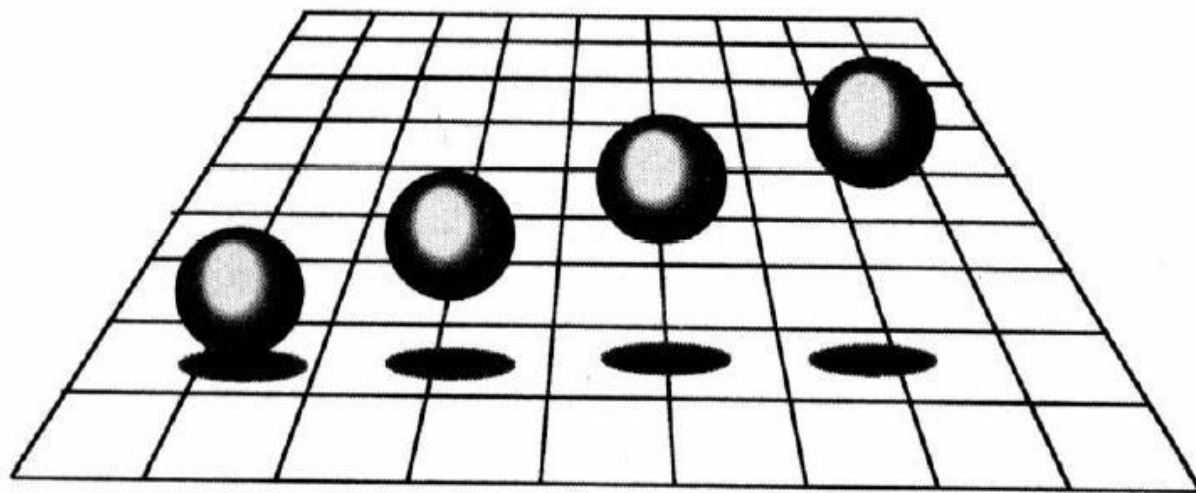
- 不同的阴影能够对应不同的小球位置



- 通过这个例子，我们得出结论：阴影给物体的位置提供了非常重要的视觉提示，同样的图片，被赋予不同的阴影，可以代表不同的物体位置。所以，阴影是非常重要的。

阴影 (Shadow)

- 不同的阴影能够对应不同的小球位置



- 通过这个例子，我们得出结论：阴影给物体的位置提供了非常重要的视觉提示，同样的图片，被赋予不同的阴影，可以代表不同的物体位置。所以，阴影是非常重要的。

阴影 (Shadow)



VS



阴影 (Shadow)

■ 阴影是什么？

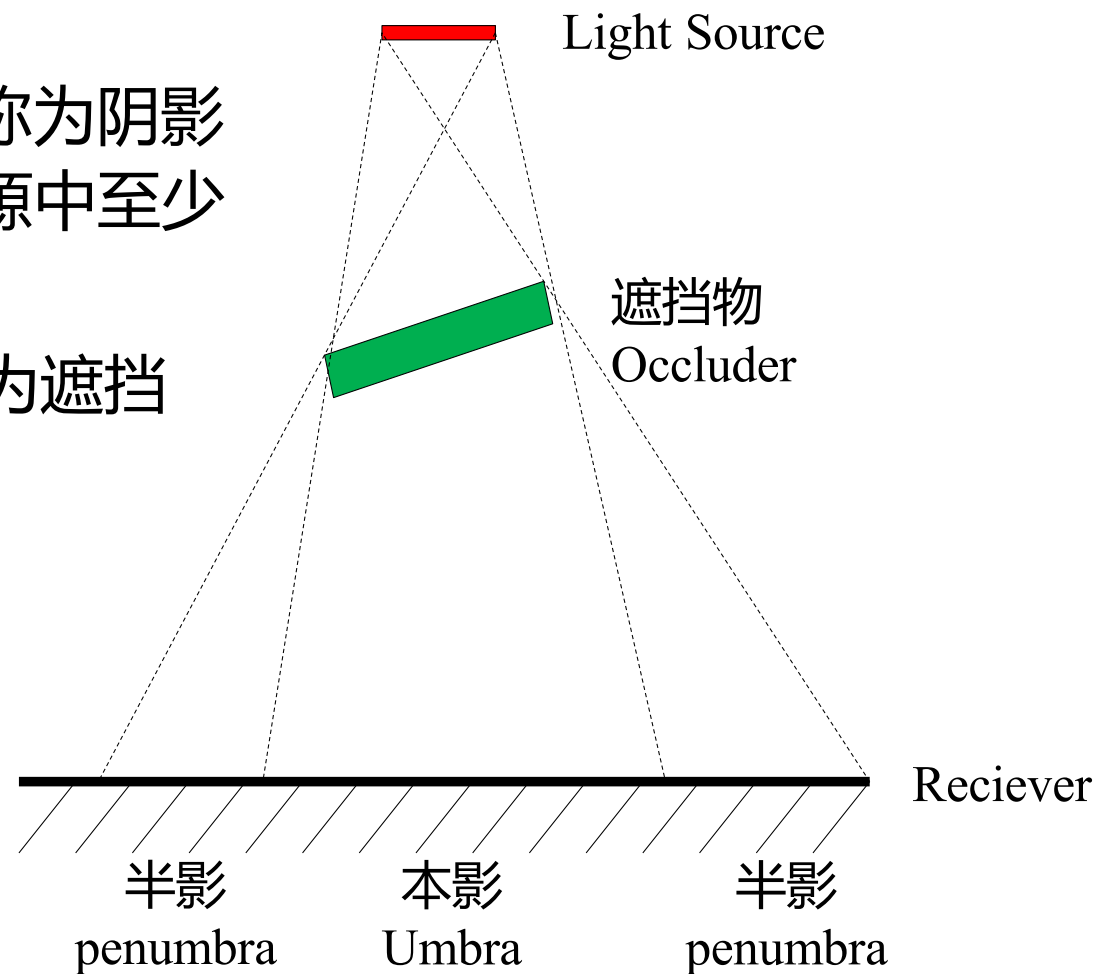
■ 阴影的定义

- ◆ 考虑一个由光源 (light source) L 照明的场景：
- ◆ 场景中的每个物体作为接收者 (receivers)，都有可能被光源 L 照明到。
- ◆ 场景中的一点 P 被称为本影 (umbra)，如果从 P 点不能看到光源 L 的任何一部分。
- ◆ 如果从 P 点可以看到光源 L 的某一部分，却不能看到全部，那么将 P 称为半影 (penumbra)。

阴影 (Shadow)

■ 阴影的定义：

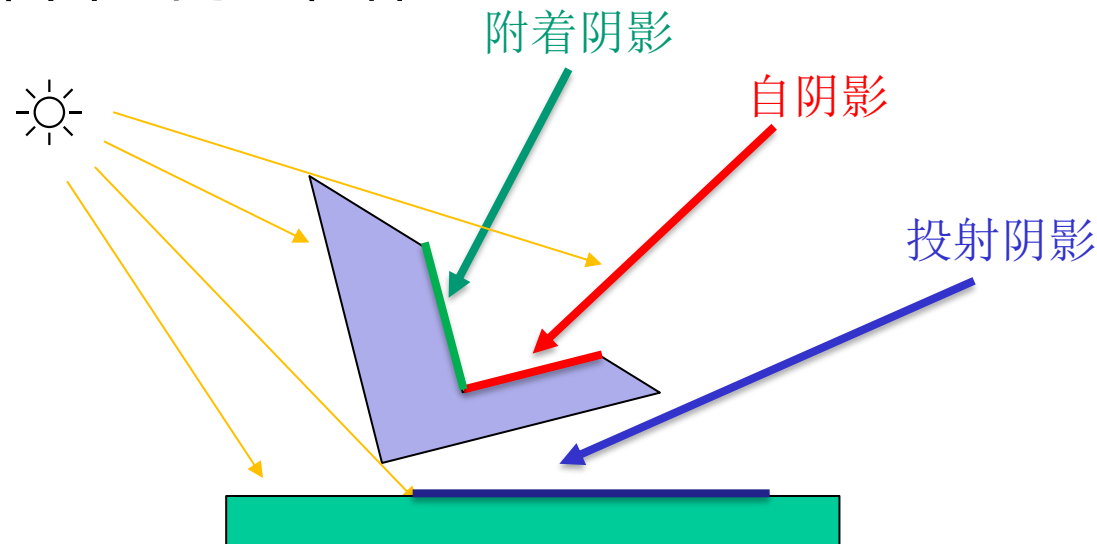
- ◆ 本影 (umbra) 和半影 (penumbra) 统称为阴影 (Shadow); 对阴影中的任一点, 光源中至少有一点会被遮挡。
- ◆ 挡住由光源到阴影区域的物体被称为遮挡物 (occluders)



阴影 (Shadow)

■ 阴影的类型 (Types of Shadows):

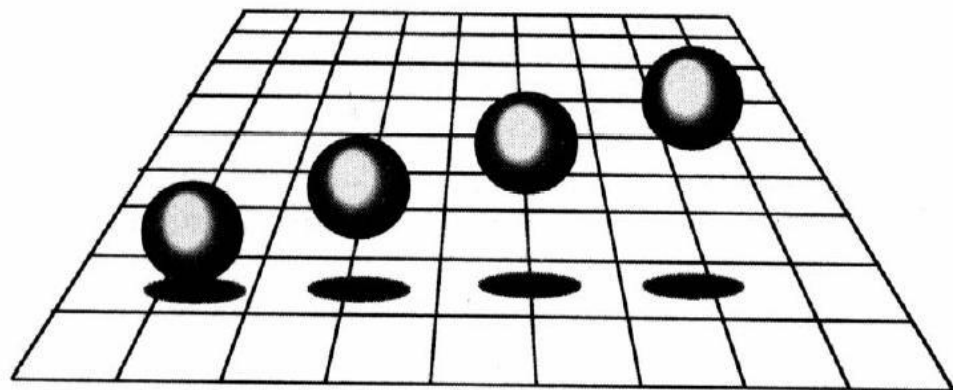
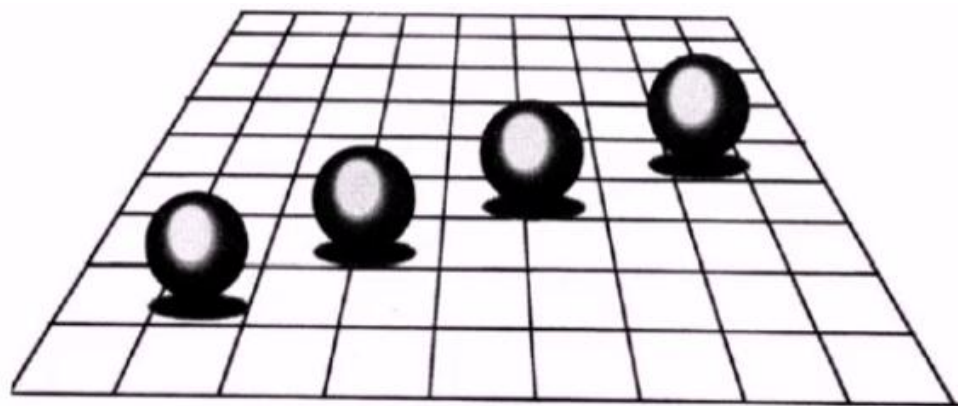
- ◆ 附着阴影 (attached shadows): 发生于接收者(receiver)的法向背离光源方向时。
- ◆ 投射阴影 (cast shadows): 发生于接收者 (receiver)的法向朝着光源方向, 但光源被遮挡物 (occluder) 所遮挡时。
- ◆ 自阴影 (self shadows): 是一类特殊的投射阴影; 对于自阴影, 接收者 (receiver) 和遮挡物 (occluder)来自于同一物体。



阴影 (Shadow)

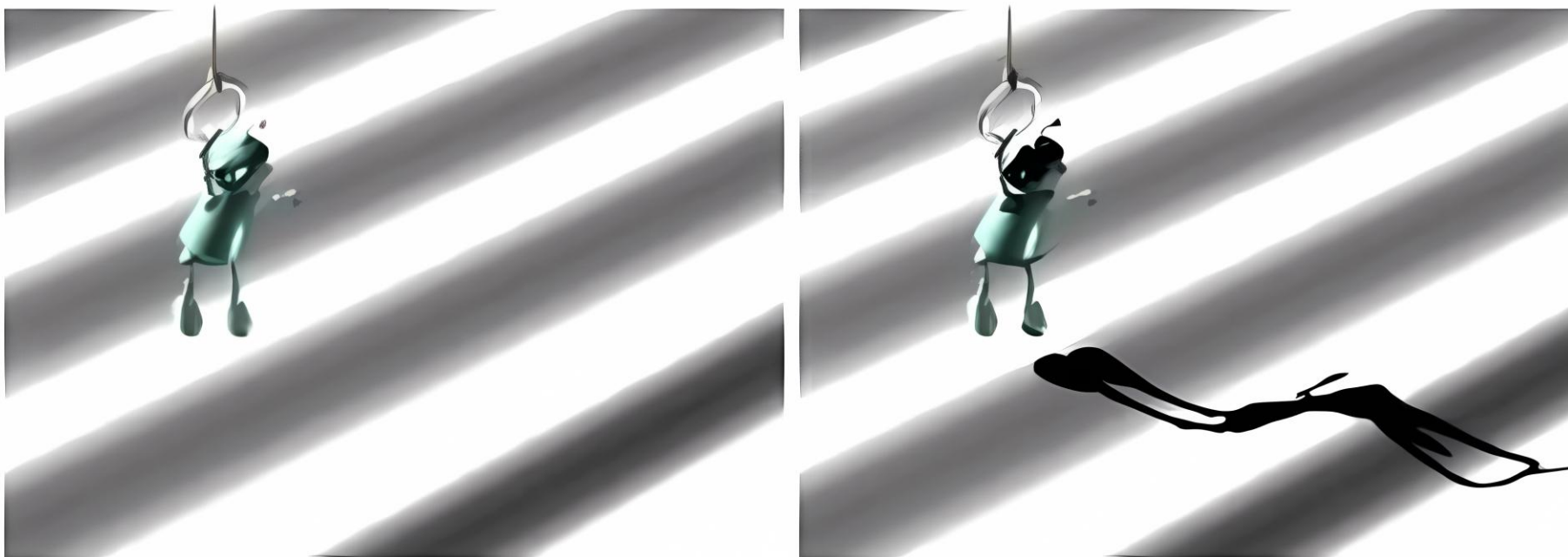
■ 阴影的重要性

- ◆ 理解场景中物体的相对位置及大小关系
- ◆ 理解复杂接收者 (receiver) 的几何形状
- ◆ 理解复杂遮挡物 (occluder) 的几何形状



阴影 (Shadow)

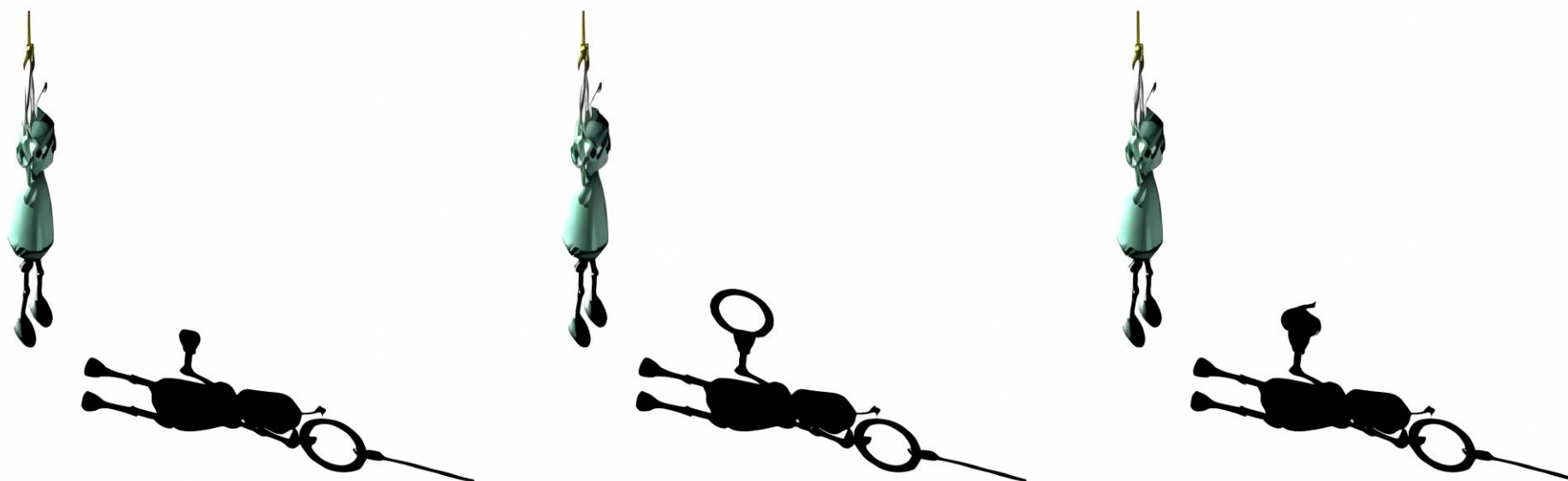
■ 阴影的重要性



◆ 阴影能够帮助我们认识接收者 (receiver) 的几何形状

阴影 (Shadow)

■ 阴影的重要性

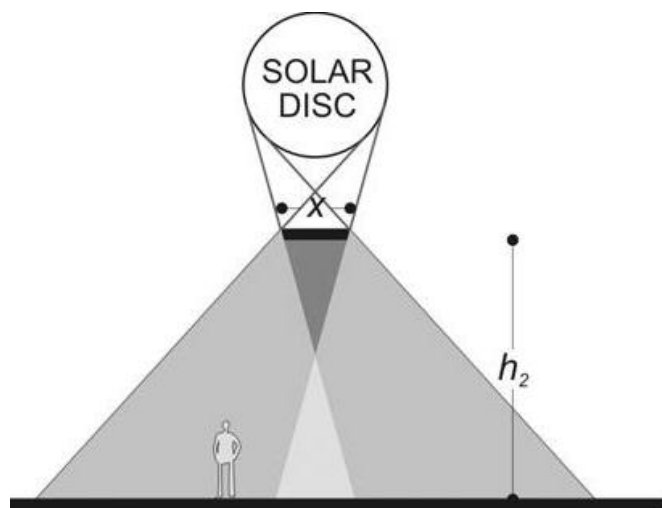


◆ 阴影能够帮助我们认识遮挡物 (occluder)的几何形状

硬阴影和软阴影

■ 硬阴影 (Hard Shadow) 和 软阴影 (Soft Shadow)

- ◆ 通常会将“阴影”理解为一个二值的状态：每个点要么在阴影中，要么不在。这样的理解对应的其实是硬阴影。硬阴影由点光源产生。
- ◆ 然而，点光源在实际中并不存在，并且硬阴影会给图像带来一种相当不真实的感觉。注意到，即使是我们日常生活中最常见的光源：太阳，也具有很显著的角展(angular extent)产生的也并不是硬阴影。



硬阴影和软阴影

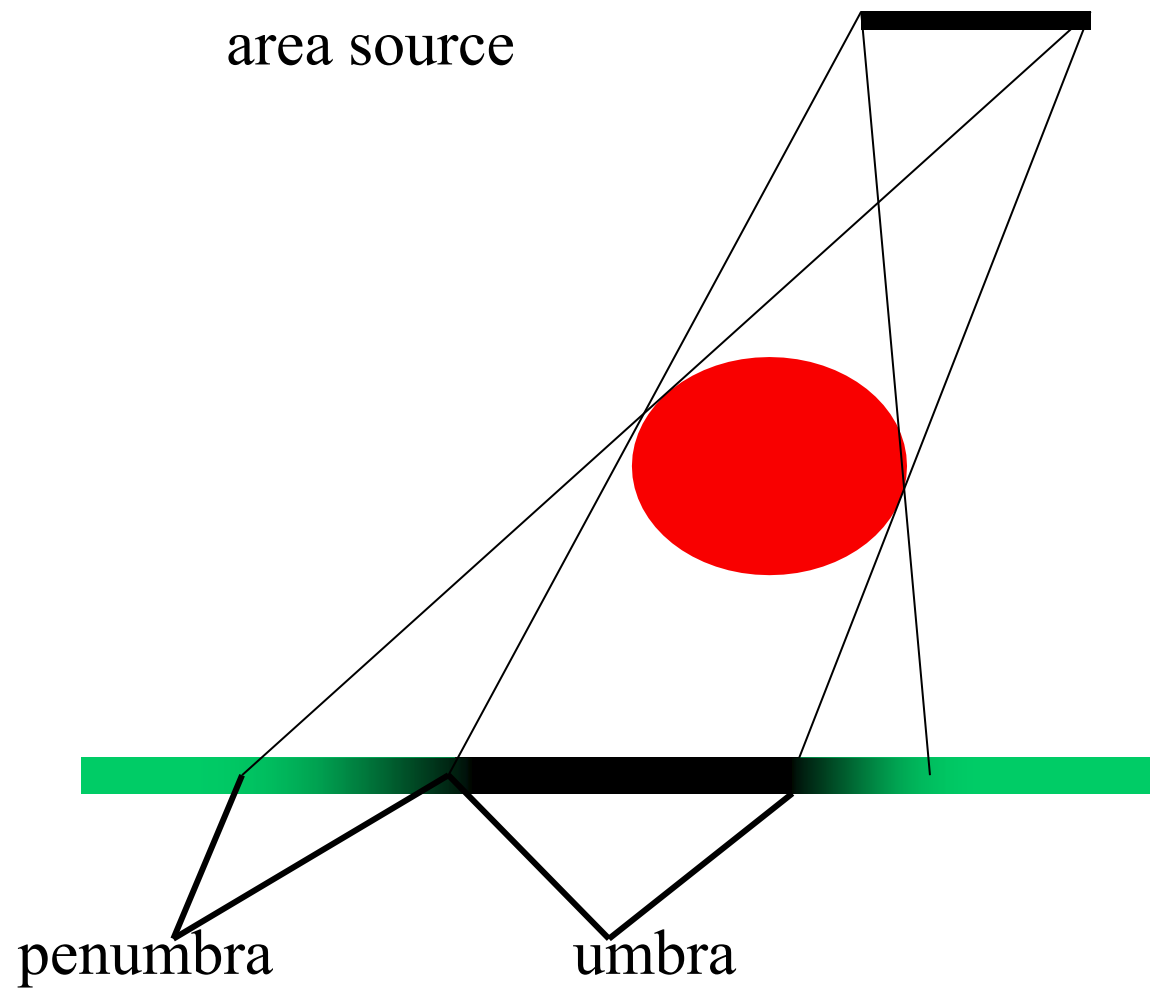
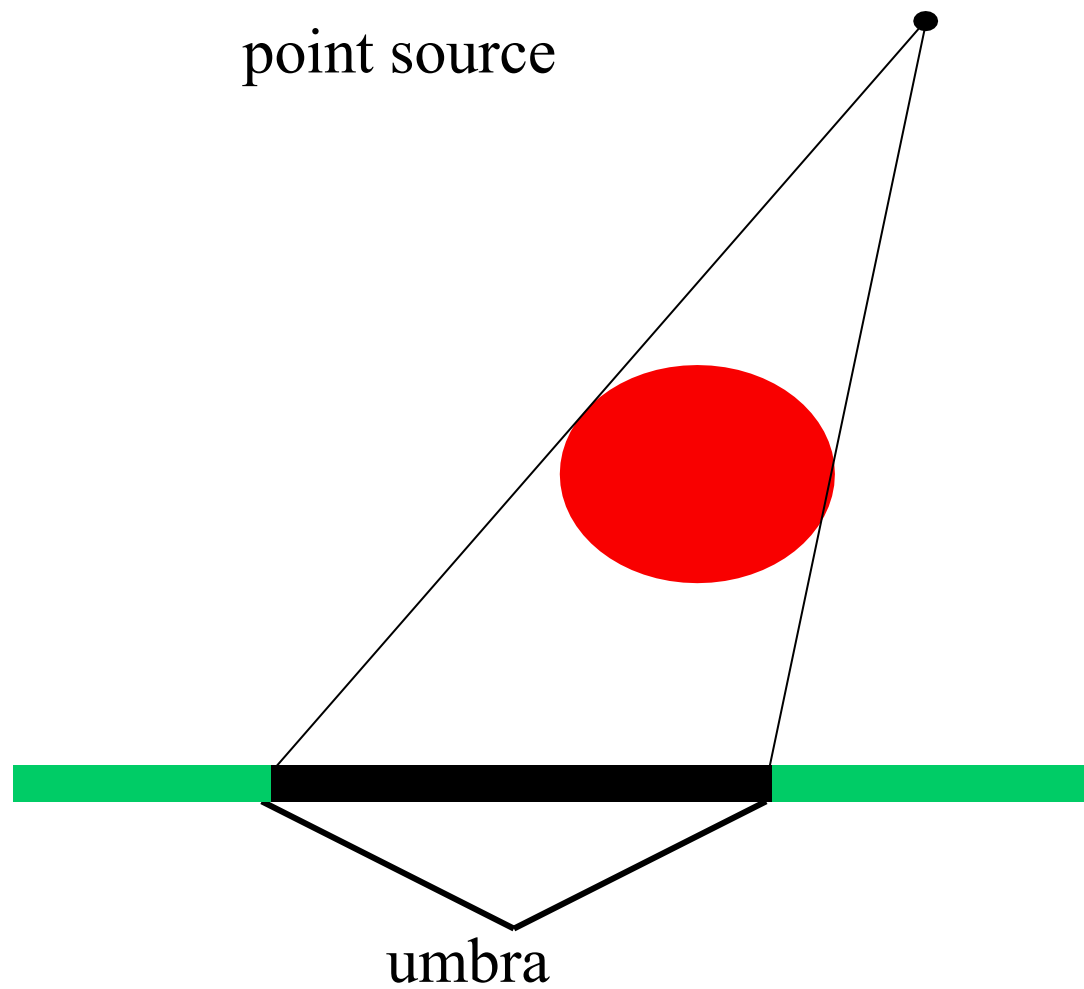
■ 硬阴影 (Hard Shadow) 和 软阴影 (Soft Shadow)

- ◆ 由于点光源在图形学中容易被模拟，不少实时算法(real-time algorithms) 围绕对硬阴影进行计算。

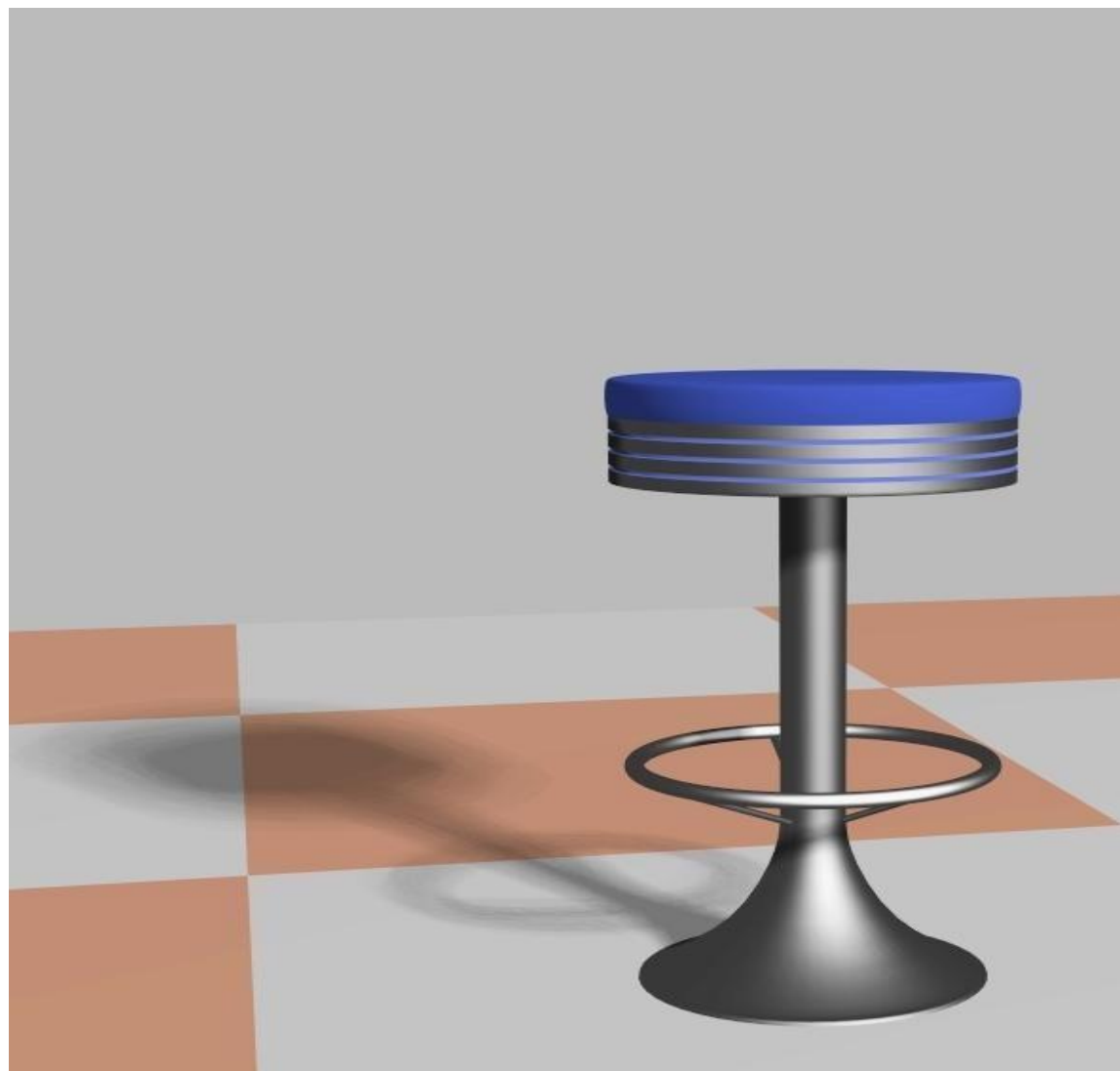
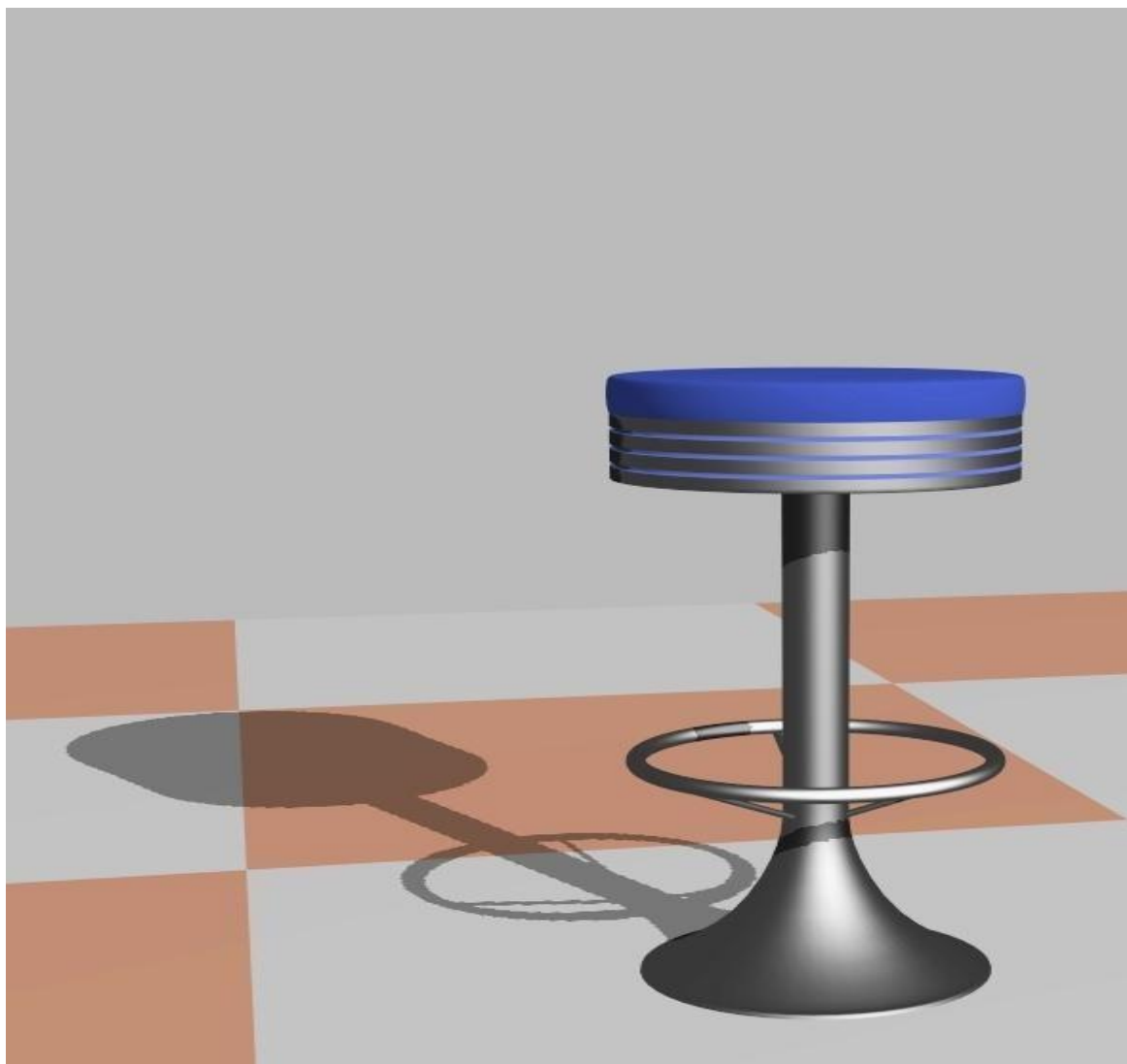
- ◆ 【难点】

在有限展度 (finite extent) 的光源 (通常是面光源) 下，确定本影 (umbra) 和半影 (penumbra) 区域通常比较困难，因为这意味着需要求解 3D空间中的可见关系 (visibility relationships)，而这是个非常困难的问题。

硬阴影和软阴影

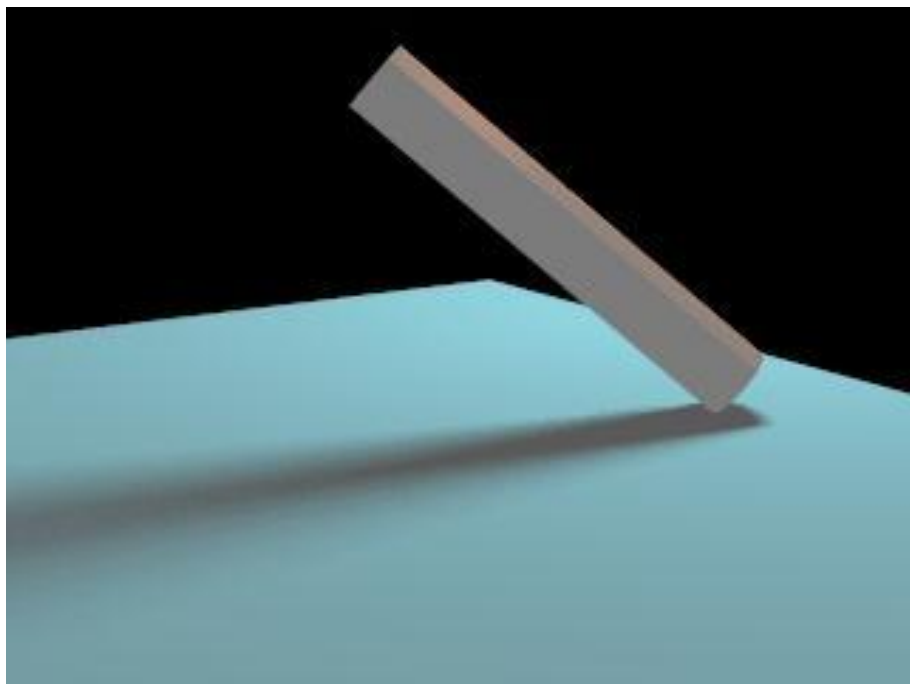


硬阴影和软阴影

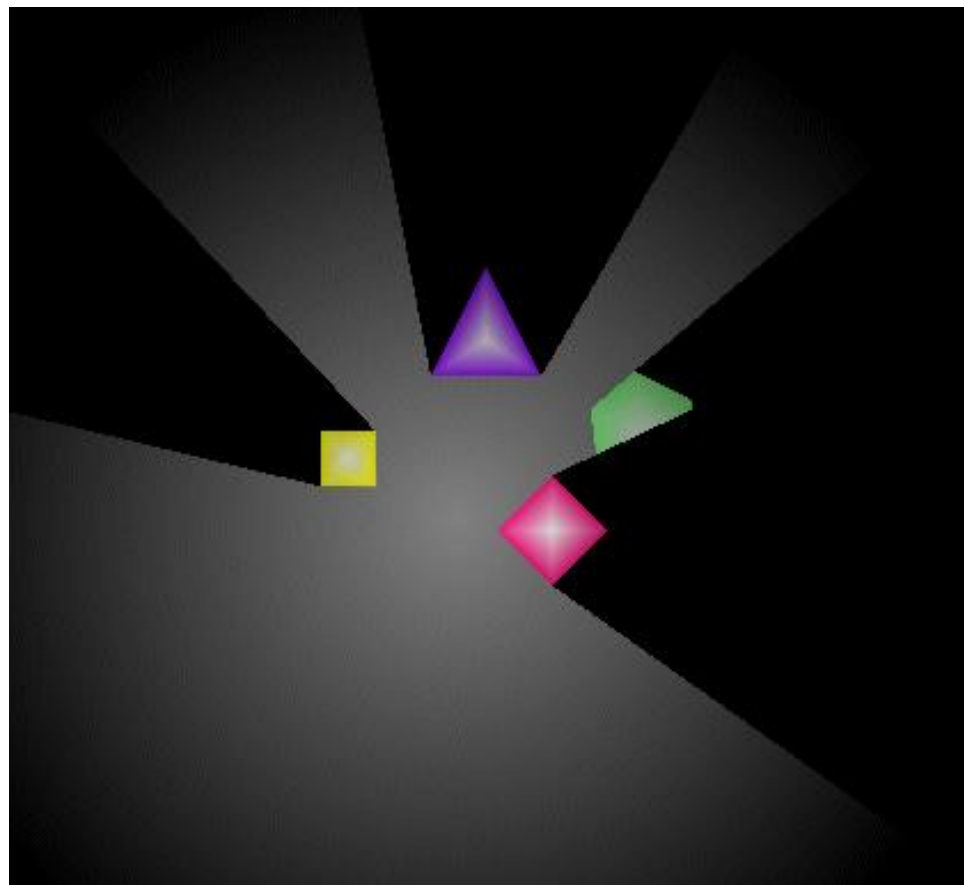


硬阴影和软阴影

软阴影



硬阴影



平面阴影

■ 什么是平面阴影？

- ◆ 当物体的阴影被投射到平的表面时，会产生平面阴影。平面阴影是一种最简单的阴影。
- ◆ 下面我们将介绍计算平面阴影的算法。

平面阴影

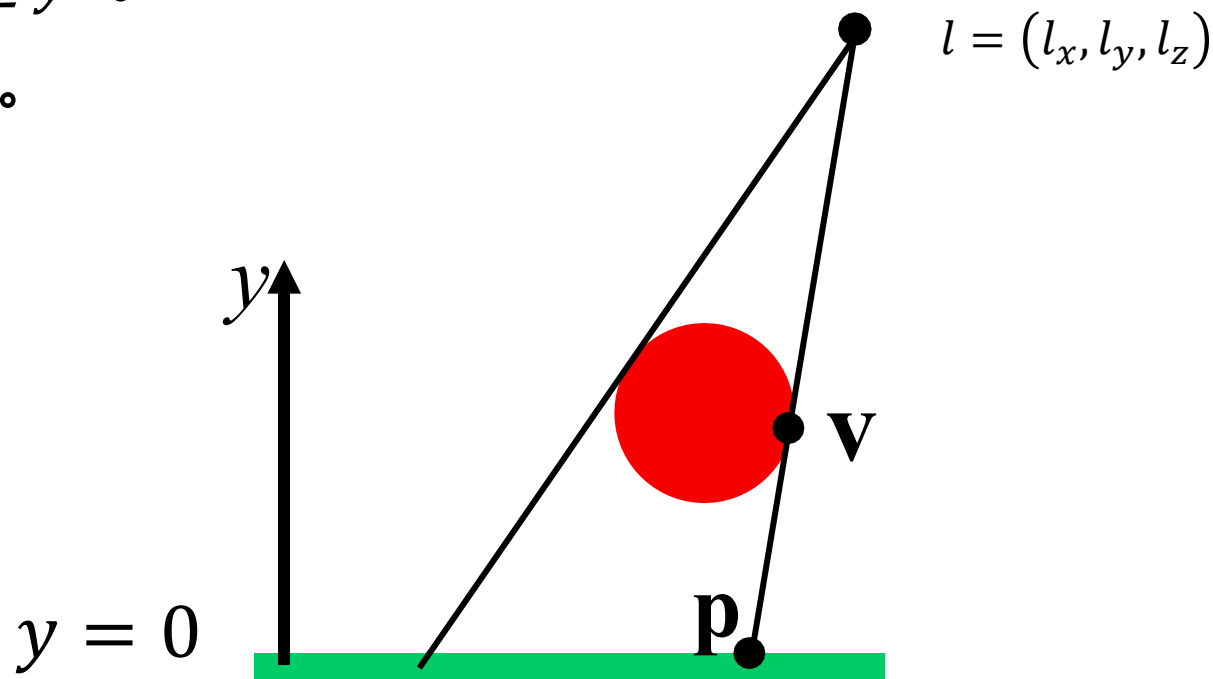
■ 阴影投影 (Projection Shadows)

- ◆ 三维空间中的物体需要被绘制两次以实现阴影效果。
- ◆ 具体来说，是使用一个矩阵将遮挡物表面的点投影到需要计算阴影的平面上。

平面阴影

■ 阴影投影 (Projection Shadows)

- ◆ 考虑下图的例子，光源位于 l ，需要阴影的点位于 v ， v 被投影到 p
- ◆ 进一步假设需要计算阴影的平面是 $y=0$
- ◆ (推广到任意其他平面也并不困难)。



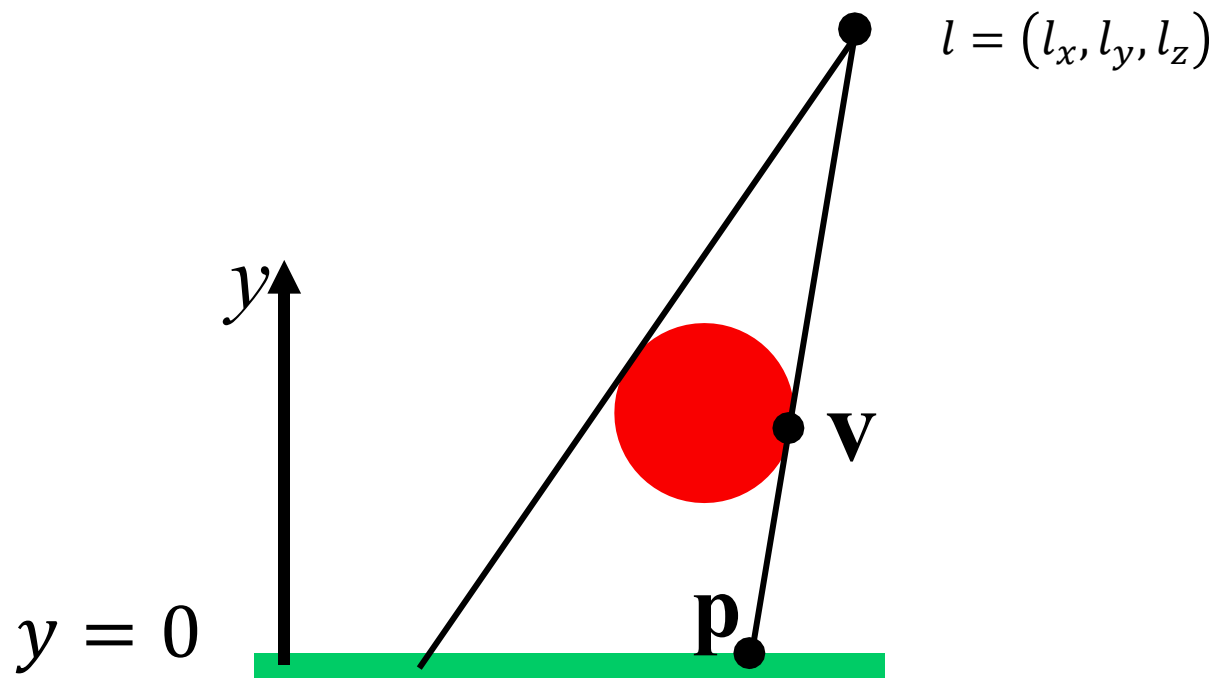
平面阴影

■ 阴影投影 (Projection Shadows)

◆ 由相似关系，可以按照以下方法计算 p 点的 x 坐标：

$$\frac{p_x - l_x}{v_x - l_x} = \frac{l_y}{l_y - v_y}$$

$$p_x = \frac{l_y v_x - l_x v_y}{l_y - v_y}$$



平面阴影

■ 阴影投影 (Projection Shadows)

- ◆ z 坐标可以类似计算得到

$$p_z = \frac{l_y v_z - l_z v_y}{l_y - v_y}$$

- ◆ 这两个方程可以统一地使用一个投影矩阵 M 来表示。

平面阴影

■ 阴影投影 (Projection Shadows)

◆ 投影矩阵:

$$M = \begin{pmatrix} l_y & -l_x & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -l_z & l_y & 0 \\ 0 & -1 & 0 & l_y \end{pmatrix}$$

容易验证: $M v = p$

平面阴影

■ 阴影投影 (Projection Shadows)

- ◆ 通常情况下需要计算阴影的平面并不会是 $y=0$ ，而是一般的平面 $n \cdot x + d = 0$
- ◆ 类似于 $y=0$ 的情况，可以算出 p 需要满足：

$$p = l - \frac{d + n \cdot l}{n \cdot (v - l)} (v - l)$$

平面阴影

■ 阴影投影 (Projection Shadows)

◆ 这个方程同样可以表示成为投影矩阵的形式, 使得: $Mv = p$

$$M = \begin{pmatrix} \mathbf{n} \cdot \mathbf{l} + d - l_x n_x & -l_x n_y & -l_x n_z & -l_x d \\ -l_y n_x & \mathbf{n} \cdot \mathbf{l} + d - l_y n_y & -l_y n_z & -l_y d \\ -l_z n_x & -l_z n_y & \mathbf{n} \cdot \mathbf{l} + d - l_z n_z & -l_z d \\ -n_x & -n_y & -n_z & \mathbf{n} \cdot \mathbf{l} \end{pmatrix}$$

◆ 当平面恰好为 $y=0$ (即 $n=(0,1,0)$, $d=0$) 时, 矩阵 M 与之前的推导相吻合。

平面阴影

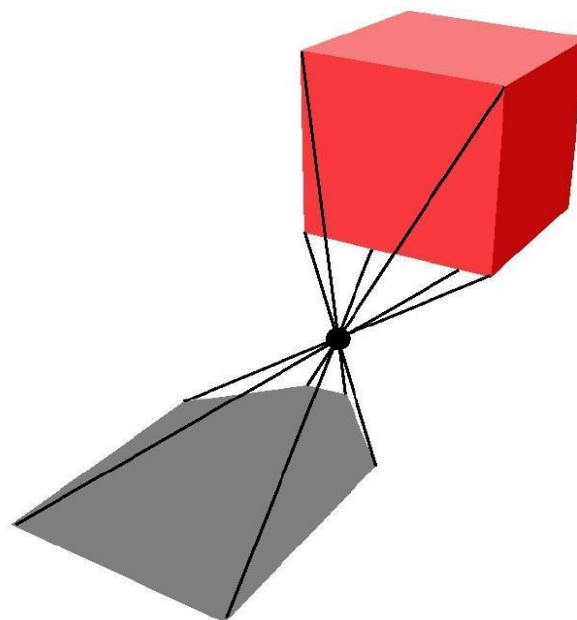
■ 阴影投影 (Projection Shadows)

- ◆ 为了绘制阴影，可以简单地利用以上矩阵将三维场景中的物体投影到目标平面上，然后对投影物体使用暗色 (dark color) 并去除光照 (without illumination) 进行绘制。
- ◆ 局限性：

接收者 (receiver) 必须是平面，每一帧的阴影需要重新绘制，即使这些阴影没有变化。

平面阴影

- 两种会产生错误阴影的情况 (Failure cases):
 - ◆ antishadow 当光源位于最上方物体之下时，算法会生成错误的
 - ◆ false shadow 当物体位于接收平面背对光源的一方 (即下方) 时，也会导致产生错误的阴影。



曲面上的阴影

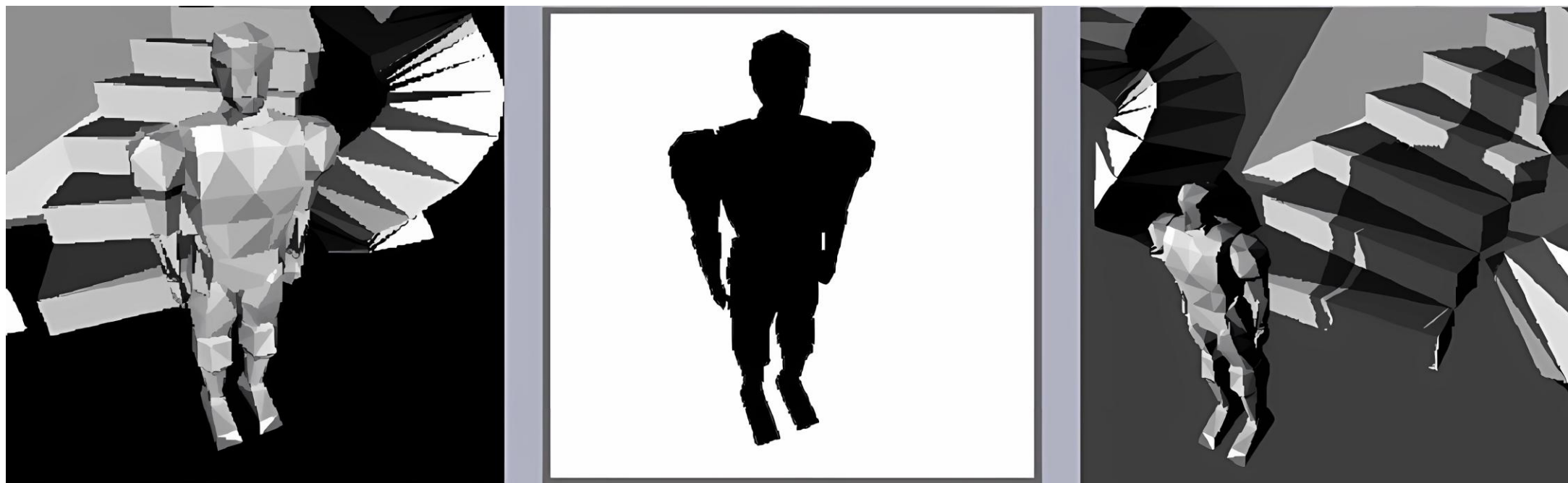
- 要将前面平面阴影的计算方法扩展到曲面 (curved surfaces) 上, 一个自然的想法是: 使用阴影图像 (shadow image) 作为投影的纹理 (projective texture)。
- 试想, 将光源作为视点: 从光源看得到的区域, 就被绘制; 从光源看不到的区域, 就是阴影

曲面上的阴影

- 从光源出发进行绘制，将遮挡物绘制成黑白场景，在白色的背景上，所有的遮挡物都被绘制成黑色。这样可以获得一张阴影的纹理图。
- 这张纹理被投影到需要绘制阴影的曲面之上。于是，在曲面上的每一点都可以计算一个其纹理图上的UV坐标，而利用此坐标，则可以直接判定该点是否属于阴影区域。
- 这个方法被称为阴影纹理 (“shadow texture”) 技术。

曲面上的阴影

- 不足 (disadvantages):
 - ◆ 物体不能投射阴影到其自身。



阴影的绘制算法

- 下面我们介绍两个重要的阴影绘制算法：
 - ◆ 阴影域 (shadow volume) 算法,
 - ◆ 阴影图 (shadow map) 算法。
- 它们都可以用于绘制任意场景、任意几何的阴影效果 (包括自阴影)。

阴影域 (Shadow Volume)

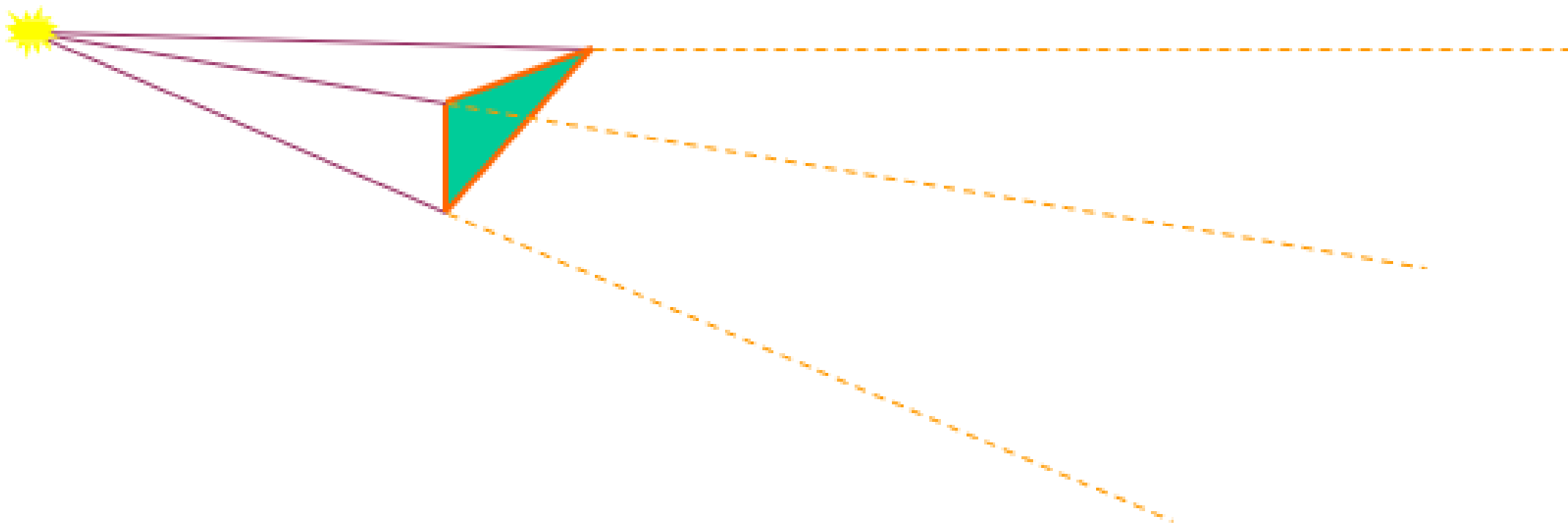
- 阴影域 (shadow volume) 算法由 Crow 提出, 可以将阴影投射到任何物体表面。阴影域技术有时也被称为体阴影 (volumetric shadows)。



SIGGRAPH 1977“Shadow algorithms for computer graphics”779 citations

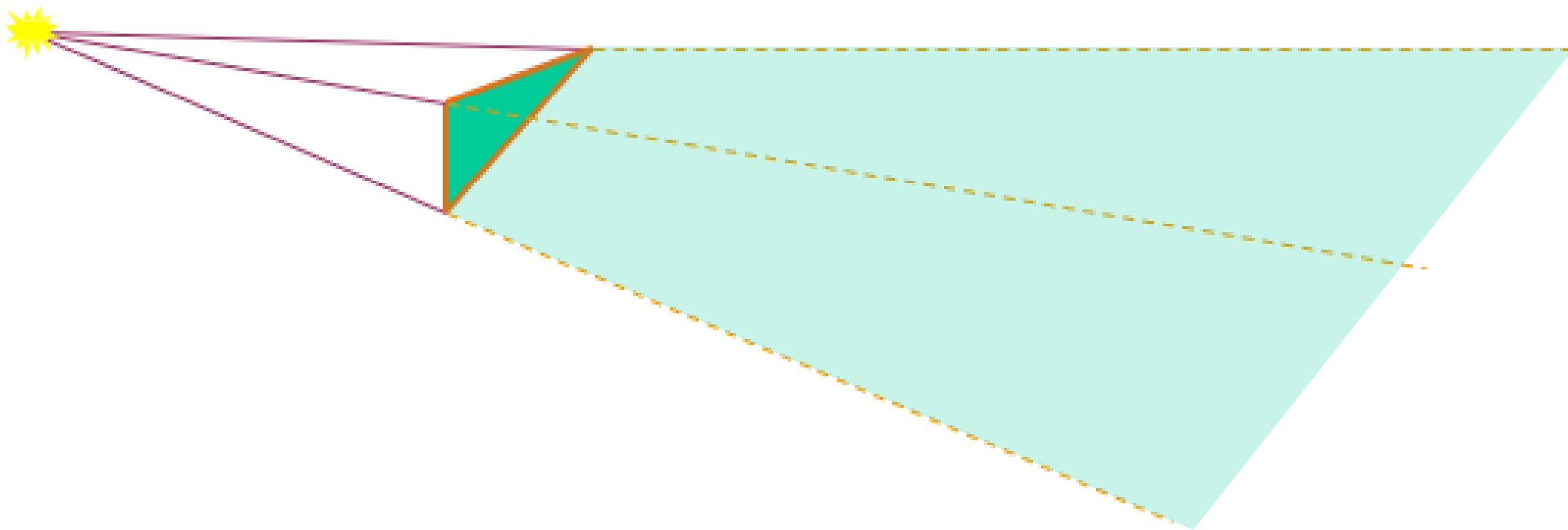
阴影域 (Shadow Volume)

- 首先，想象三维空间中的一个点和一个三角形，连接点到三角形的三个顶点并延长，可以得到一个衍生到无穷远处的三棱锥。



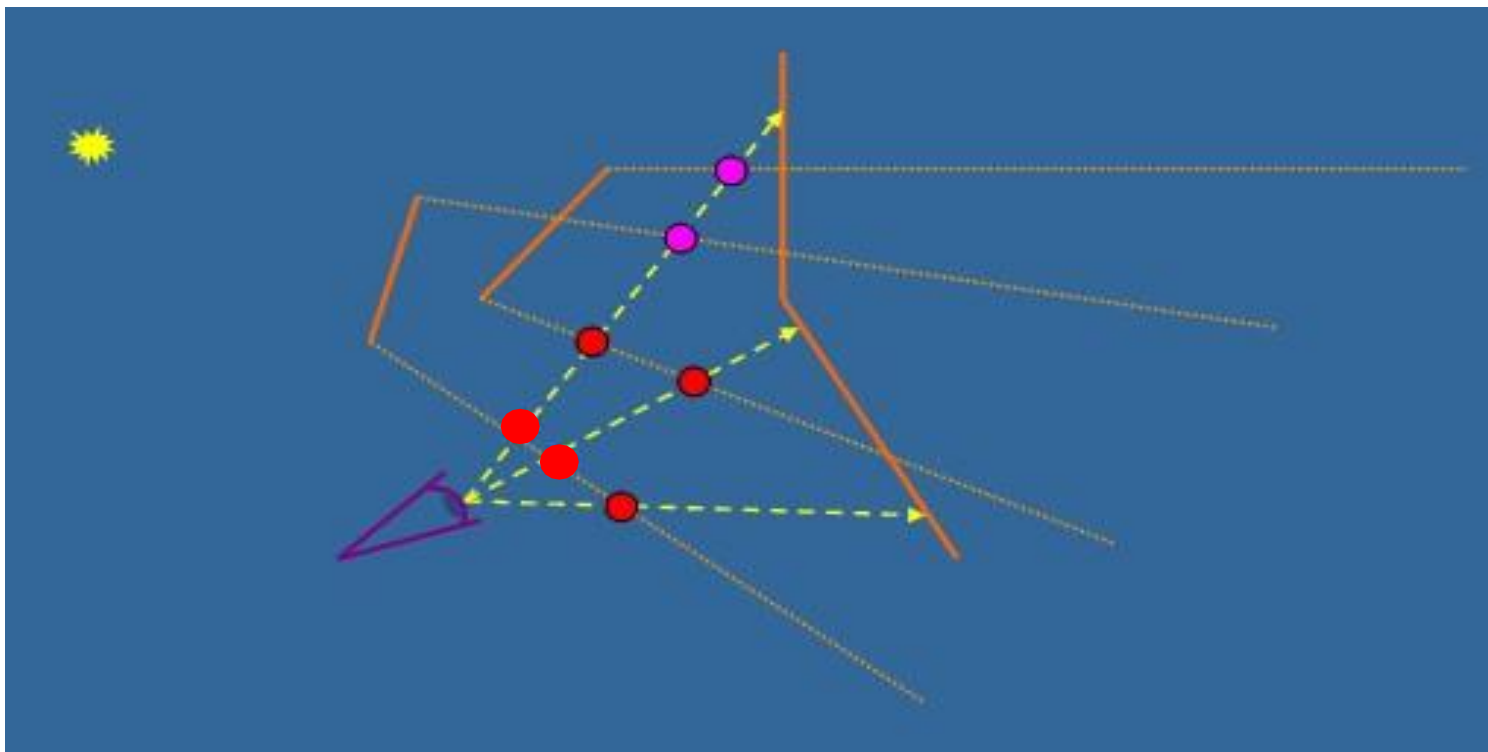
阴影域 (Shadow Volume)

- 现在，将点想象成点光源。那么，任何物体的任何一部分，只要在下图中切去顶端的三棱锥中，就属于阴影区域。这个被切去顶端的三棱锥我们称之为阴影域 (shadow volume)。



阴影域 (Shadow Volume)

- 假设在绘制中，我们从视点向屏幕的某个像素投射一条射线。该射线与场景中某一物体交于一点，我们需要确定该交点是否位于阴影当中
- 我们需要做的仅仅是判断该交点是否位于某个阴影域当中。



阴影域 (Shadow Volume)

- 假设视点在所有的shadow volume之外，我们维护一个计数器，其初始值是零。每次当从视点射向目标像素的射线进入到一个shadow volume中时，将计数器 +1；而当射线从一个shadow volume中射出时，将计数器 -1。
- 这样，我们只需检验当射线到达交点时，计数器是否大于零：如果大于零，则交点位于阴影中；否则不属于阴影区域。

阴影域 (Shadow Volume)

■ 使用模板缓存 (Stencil Buffer)

- ◆ 当然，如果从几何上实现上述算法是麻烦而费时的。然而，一个巧妙的办法是使用硬件中的模板缓存 (stencil buffer) 来实现计数。
- ◆ 模板缓存 (stencil buffer) 可以对每个像素存储一个整数值；它与深度缓存 z-buffer 的区别仅仅在于z-buffer 存储的是实数的深度值 (depth value)

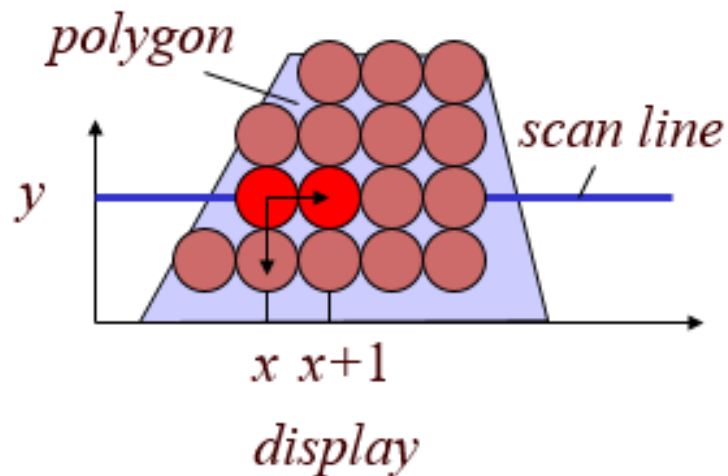
阴影域 (Shadow Volume)

■ 深度缓存 (Z - Buffer)

```
var zbuf: array[N,N] of real;           {zbuffer: 0=near, 1=far}
    fbuf: array[N,N] of color;         {frame-buffer}
For all 1 ≤ i,j ≤ N do
    zbuf[i,j] := 1.0; col[i,j] := BackgroundColour;
For all polygons do                     {scan conversion}
    For all covered pixels (i,j) do
        Calculate depth z;
        if z < zbuf[i,j] then          { closer! }
            zbuf[i,j] := z;
            fbuf[i,j] := surfacecolor(i,j);
```

阴影域 (Shadow Volume)

- 快速计算深度值: use coherence.



$$\text{Plane: } Ax + By + Cz + D = 0$$

$$\text{Hence: } z(x, y) = \frac{-Ax - By - D}{C}$$

$$\text{Also: } z(x+1, y) = \frac{-A(x+1) - By - D}{C}$$

$$\text{Thus: } z(x+1, y) = z(x, y) - \frac{A}{C}$$

$$\text{Also: } z(x, y) = z(x, y-1) + \frac{B}{C}$$

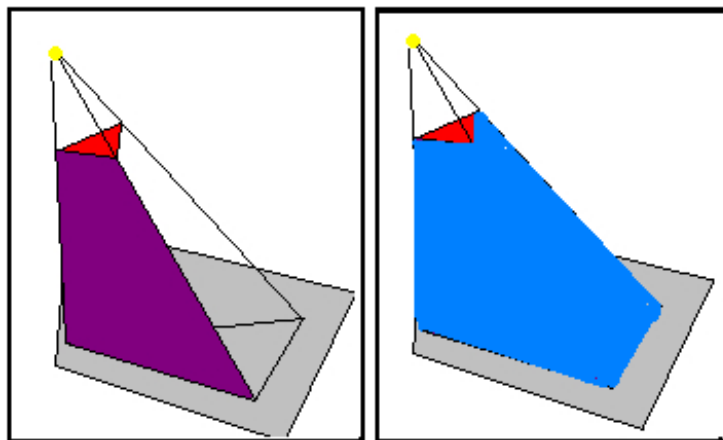
阴影域 (Shadow Volume)

■ 使用模板缓存 (Stencil Buffer) 阴影绘制

- ◆ 首先，清空模板缓存。
- ◆ 然后，将整个场景绘制到帧缓存中，这次绘制只使用环境光分量和发光分量，并获取响应的颜色信息（在color buffer中）及深度信息（在z-buffer中）。
- ◆ 第三部，关闭颜色缓存的写入和深度检测，绘制所有shadow volume的正面（即射线射入shadow volume时相交的面）。在这个过程中，如果一个像素其深度值小于之前算好的z-buffer中的深度值，那么将这个像素的模板缓存（stencil buffer）上的计数器+1。

阴影域 (Shadow Volume)

- 第四步，类似于前一步骤，将所有 shadow volume 的反面绘制一遍，只是这次是每发现一个像素其深度值小于之前算好的 z-buffer 中的深度值，将该像素的模板缓存上的 $a-1$ 。
- 最后，再将整个场景根据模板缓存的信息绘制上漫反射分量和高光分量：只有模板缓存是 0 的像素才绘制，以实现阴影效果。



阴影域 (Shadow Volume)

■ Shadow Volume 算法的优点:

- ◆ 首先, 它可以使用通用的图形学硬件实现, 而仅仅需要一个模板缓存
- ◆ 其次, 由于它不是基于图像的方法 (不像下面将要介绍的 shadow map 算法), shadow volume 算法并不会产生由采样和分辨率带来的各种问题, 从而可以在任何地方生成正确和清晰的阴影 (sharp shadows)。

阴影域 (Shadow Volume)

- Shadow Volume 算法的不足:

- ◆ 绘制效率较低

由于shadow volume的多边形面片数通常都较多, 且会覆盖住大量的像素, 这在很大程度上影响了算法运行和光栅化过程的速度, 使得绘制的效率较低。

阴影域 (Shadow Volume)

■ 一些结果：



阴影域 (Shadow Volume)

■ 一些结果：

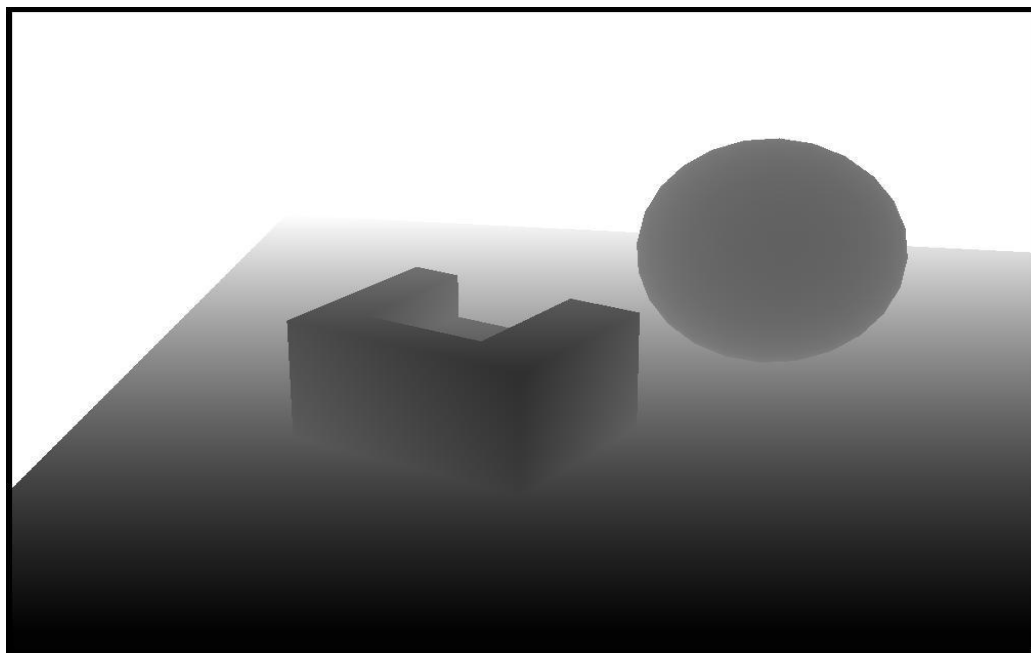


阴影图 (Shadow Map)

- 1978年, L Williams 提出了一个基于 z-buffer 的通用的算法, 可以对于任意的场景物体快速地计算阴影。
- 这个算法的思想是: 以光源作为视点的位置, 使用z-buffer算法绘制场景, 获得阴影图, 并将其结果用于任意场景的阴影绘制。
- Casting curved shadows on curved surfaces, ACM SIGGRAPH 1978, 1054citations
- 2001 因在mip mapping, shadow buffers, facial animation by image warping等方面的贡献, 获ACM SIGGRAPH Coons奖

阴影图 (Shadow Map)

- 使用深度缓存 z-buffer, 可以获得从光源出发到任意一个方向最近点的距离, 并以图像的形式存储下来, 如图:
- 我们将这张深度图像称为阴影图(shadow map)。

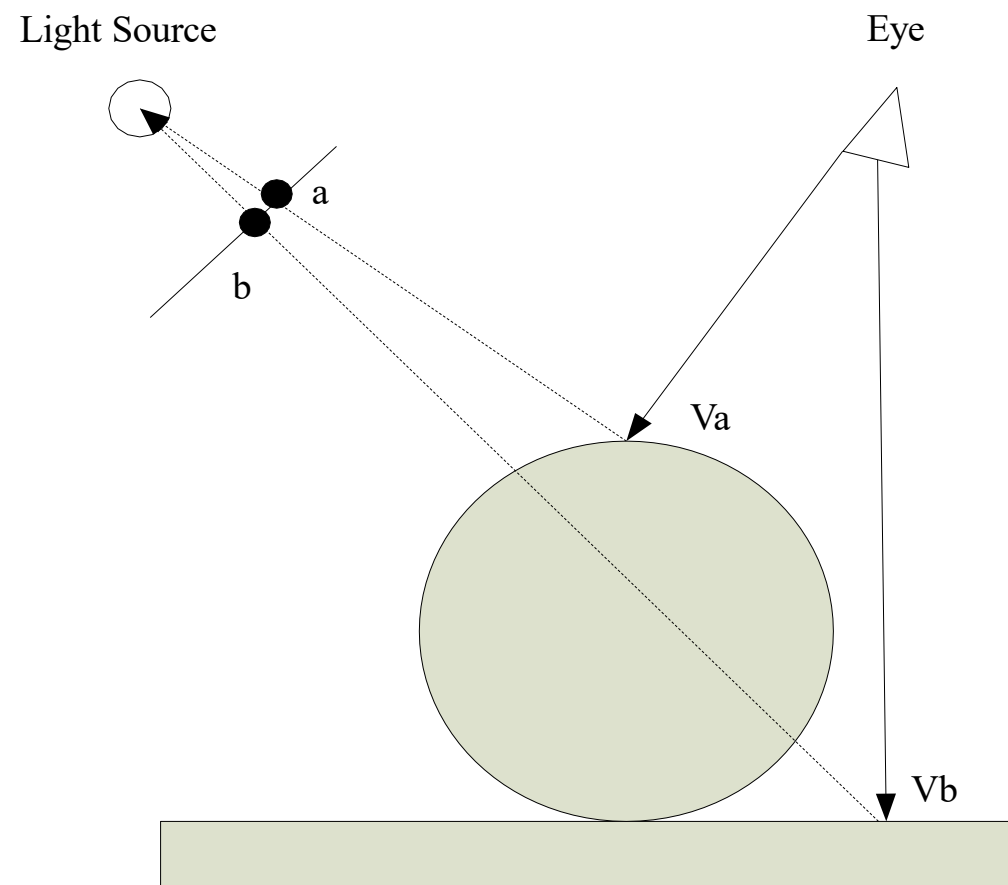
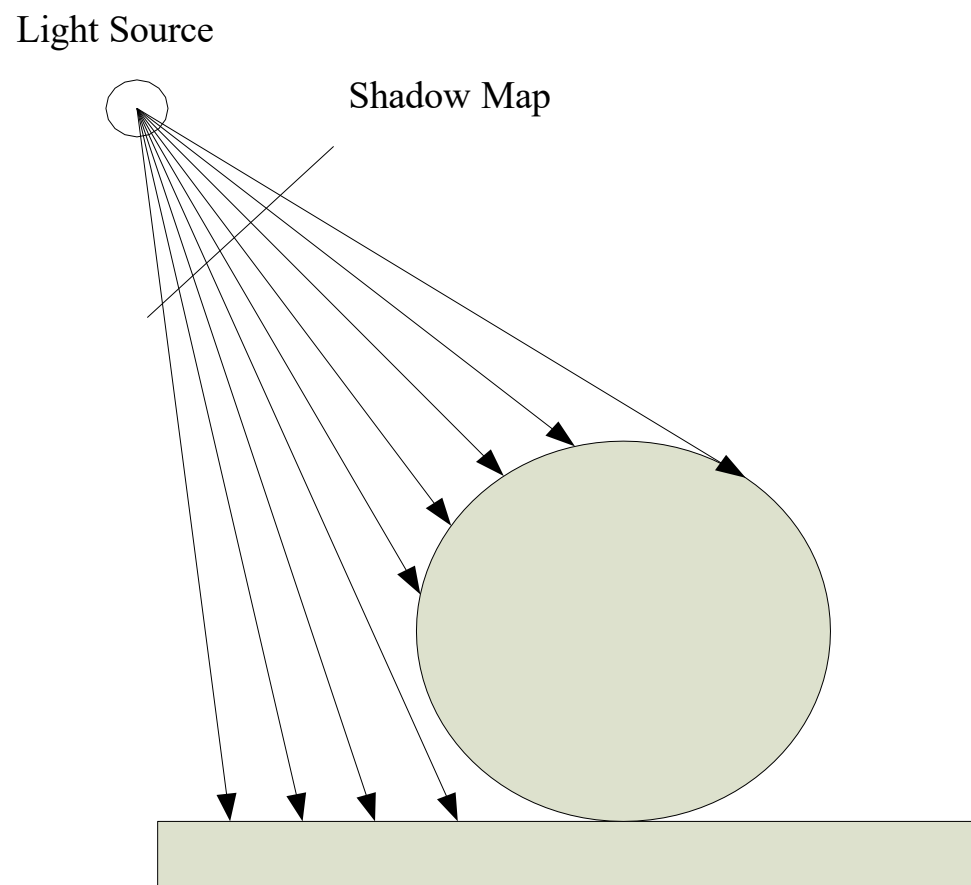


阴影图 (Shadow Map)

- 使用阴影图 (shadow map), 可以在任意视点位置对场景进行带阴影效果的第二次绘制。
- 在进行绘制时, 对于沿视点向屏幕每个像素发出的光线与场景中物体的交点, 使用阴影图判断该点是否位于阴影区域:
- 如果该交点到光源的距离大于阴影图上对应点所存储的深度值, 则该点位于阴影中。否则, 该点不在阴影中。

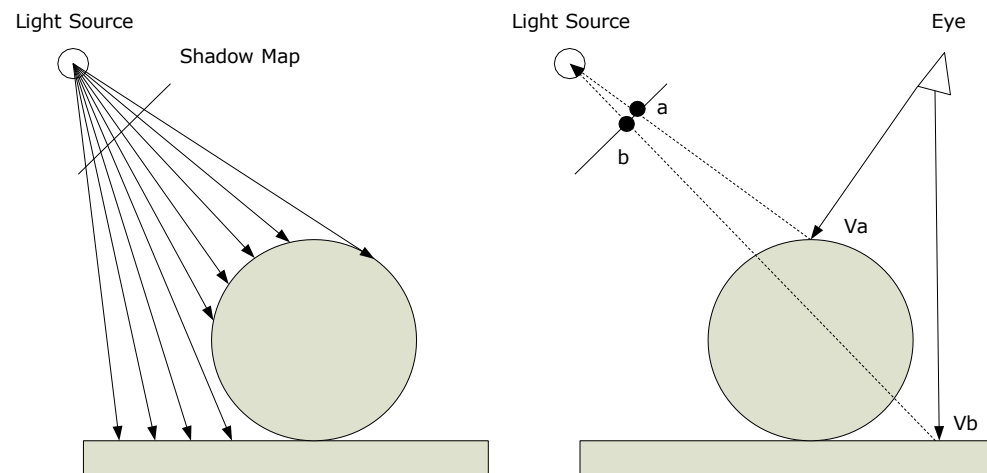
阴影图 (Shadow Map)

■示意图:



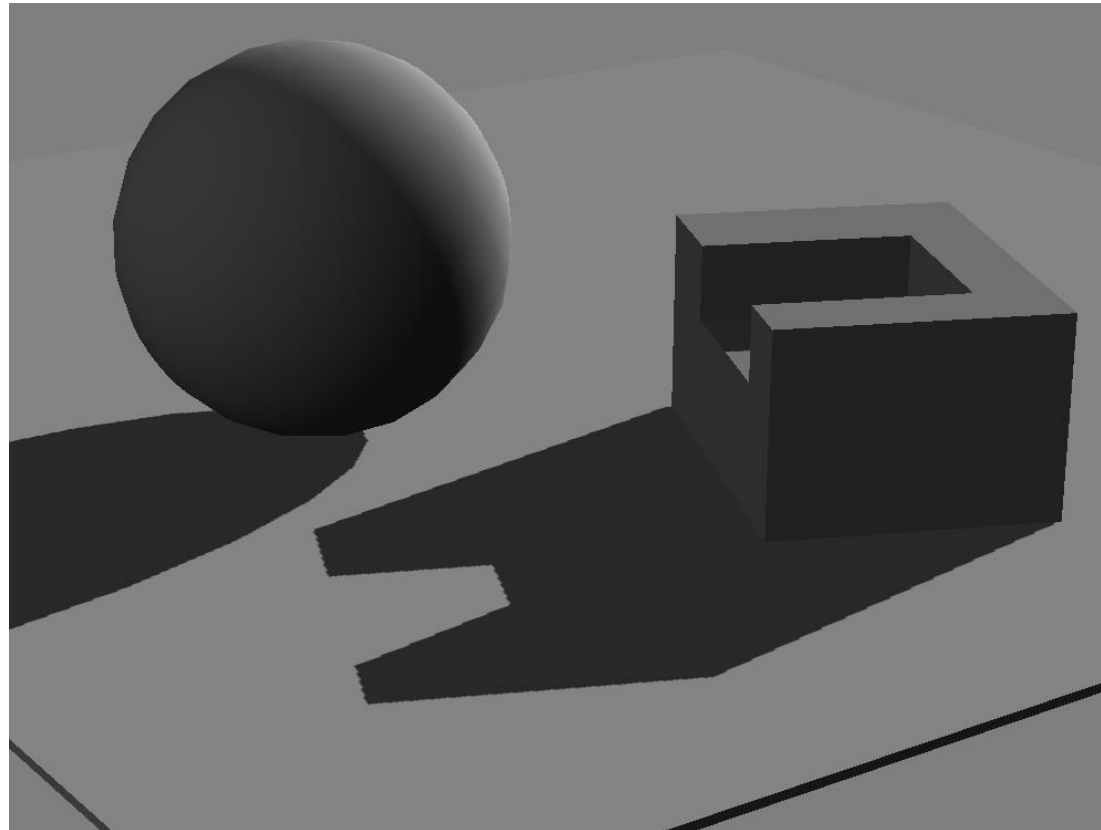
阴影图 (Shadow Map)

- 在示意图中，阴影图存储了由光源到场景中物体表面的深度信息；由视点对 V_a 和 V_b 这两个位置进行观察：
- 对于 V_a ： V_a 在阴影图上的纹理坐标为 a ，而存储在 a 的阴影图的深度信息不小于 V_a 到光源的距离，因此 V_a 不属于阴影区域。
- 对于 V_b ： 由 V_b 到光源的距离要比存储在 b 的阴影图的深度信息更大，因此 V_b 位于阴影区域。



阴影图 (Shadow Map)

■ 结果示例：



阴影图 (Shadow Map)

■ 阴影图算法的优点：

- ◆ 绝大部分的硬件都可以直接支持阴影图 (shadow map) 算法，以用于绘制任意场景的阴影效果。
- ◆ 阴影图算法很快，阴影图的构建开销与要绘制的基元 (rendered primitives) 数目成正比；
- ◆ 使用阴影图进行任意视点的阴影绘制时，每个像素只需要额外对阴影图进行一次查询，以及一次距离的比较。

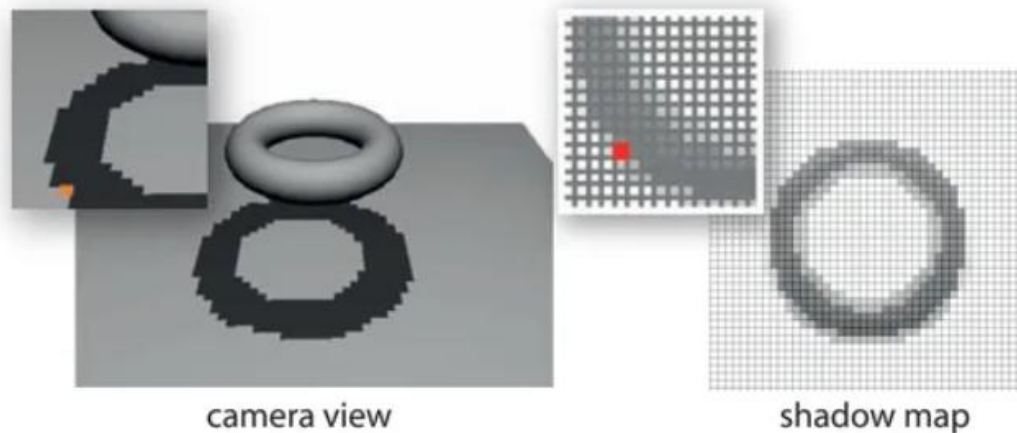
阴影图 (Shadow Map)

■ 阴影图算法的不足：

- ◆ 由于阴影图算法是基于图像的算法，因此其绘制质量会受到阴影图的分辨率、以及 z-buffer 的数值精度的影响。

■ 阴影图算法的问题：

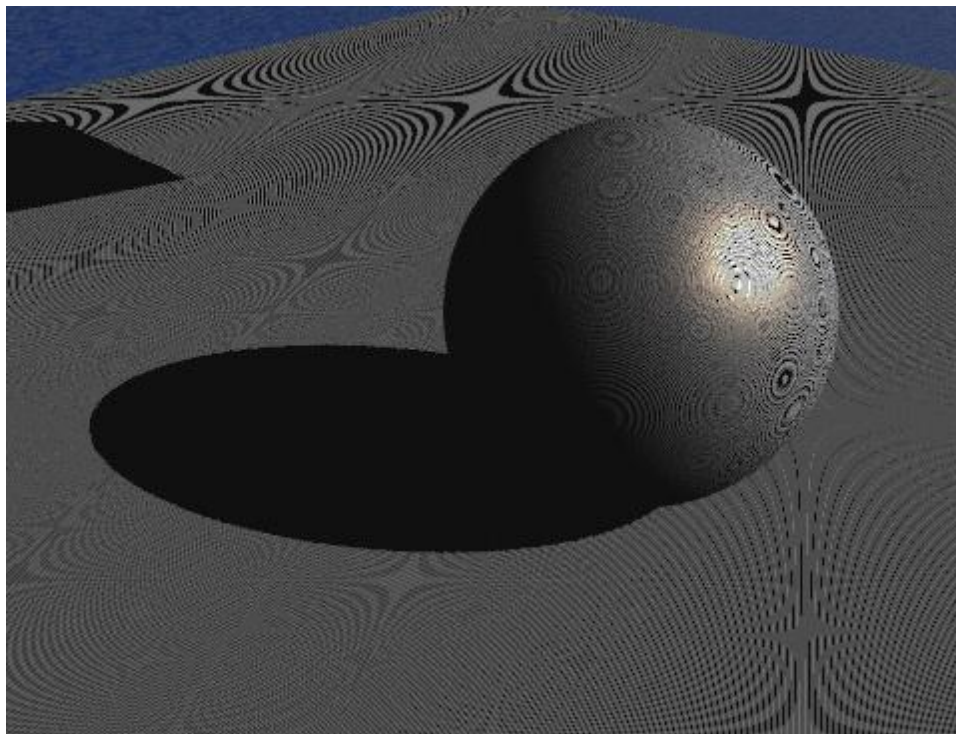
- ◆ 由于阴影贴图分辨率是有限的，每个像素占据一定大小，并且离光源越远，每个像素覆盖的片元就越多，并且涉及到采样和重采样，那么就可能导致阴影产生锯齿 (Aliasing)。



阴影图 (Shadow Map)

■ 阴影图算法的问题：

- ◆ 由于深度的数值精度和阴影贴图分辨率都有限，所以在进行深度比较的时候，有可能会出现 Z-fighting 的现象，所以需要在比较时添加偏差，称之为 Depth Bias.



z-fighting

阴影总结

■阴影图 (Shadow map) 和阴影域 (shadow volume) 算法是最为广泛使用的阴影生成与绘制算法 (其中阴影图算法使用得更为广泛)。我们介绍的只是这两个算法的最基本的原型，之后的研究者提出了它们的许多变种。

■更多的信息可以参见：

- ◆ Nvidia 的官方网站
- ◆ <http://www.realtimerendering.com>
- ◆ SIGGRAPH 和 Eurographics 的网站

阴影总结

■优点:

- ◆ Shadow Map不需要模板缓冲区。
- ◆ 当阴影多的时候哪个方法快？为什么？

■缺点:

- ◆ Shadow volume 和 Shadow Map 哪一个方法快，精度高？为什么？



阴影总结

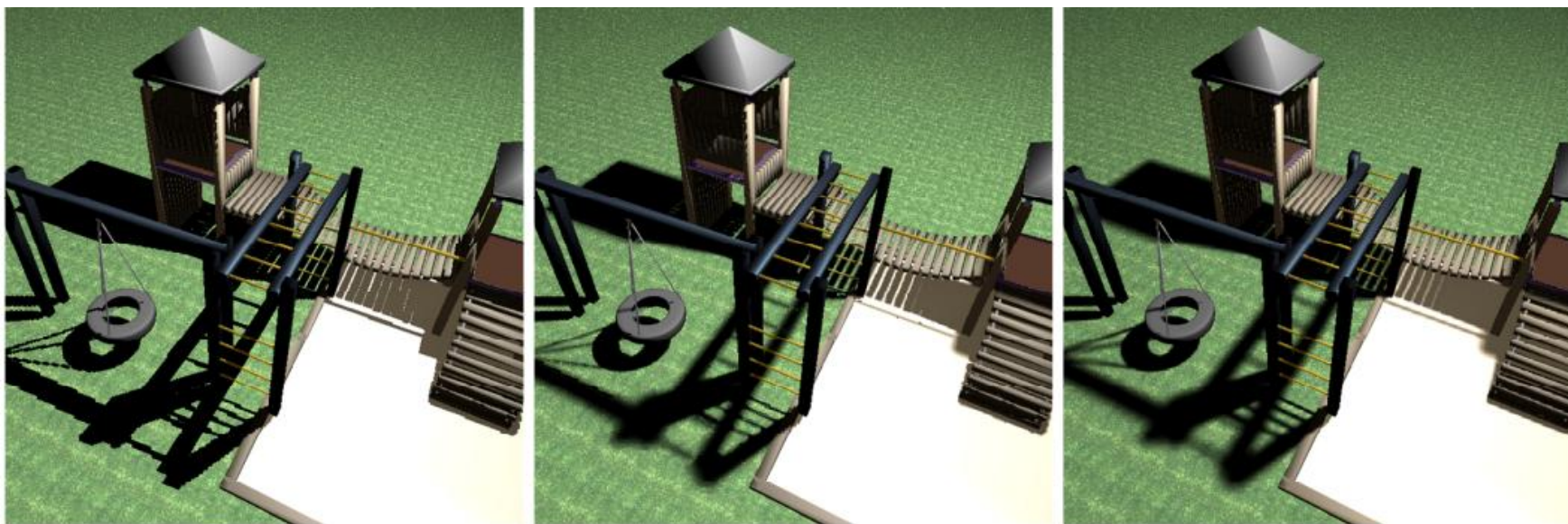
- 使用 ATI 显卡绘制的一张效果：



研究进展

■ Rendering Soft Shadows using Multilayered Shadow Fins

- ◆ Xiao-Hua Cai, Yun-Tao Jia, Xi Wang, Shi-Min Hu and Ralph R. Martin



纹理映射

提纲

- 前言
- 基本操作
- 高级主题

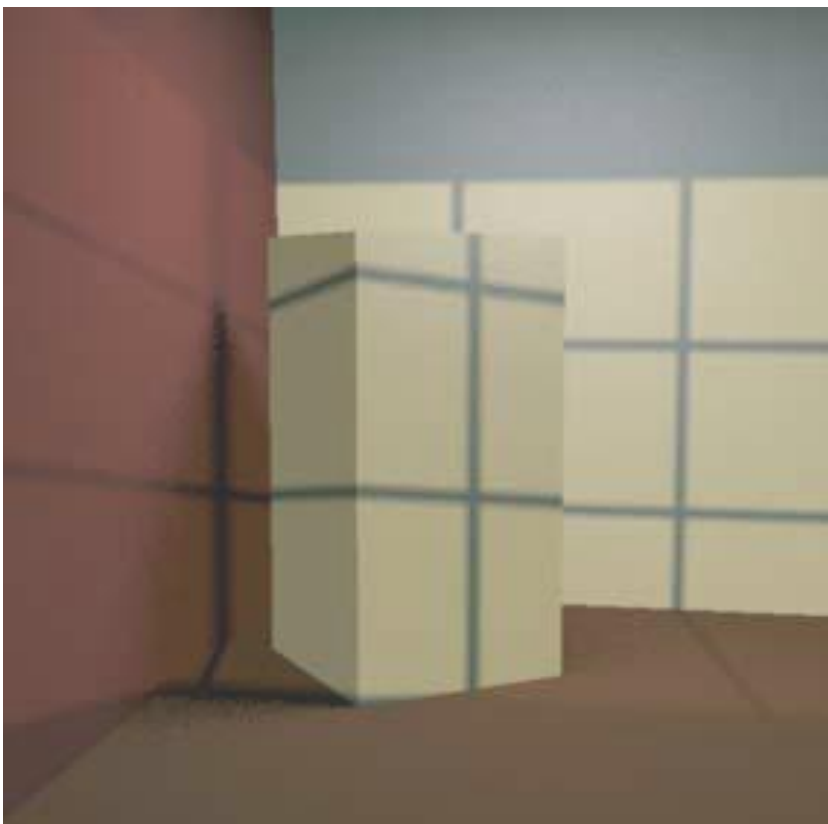
Part 1: 前言

纹理

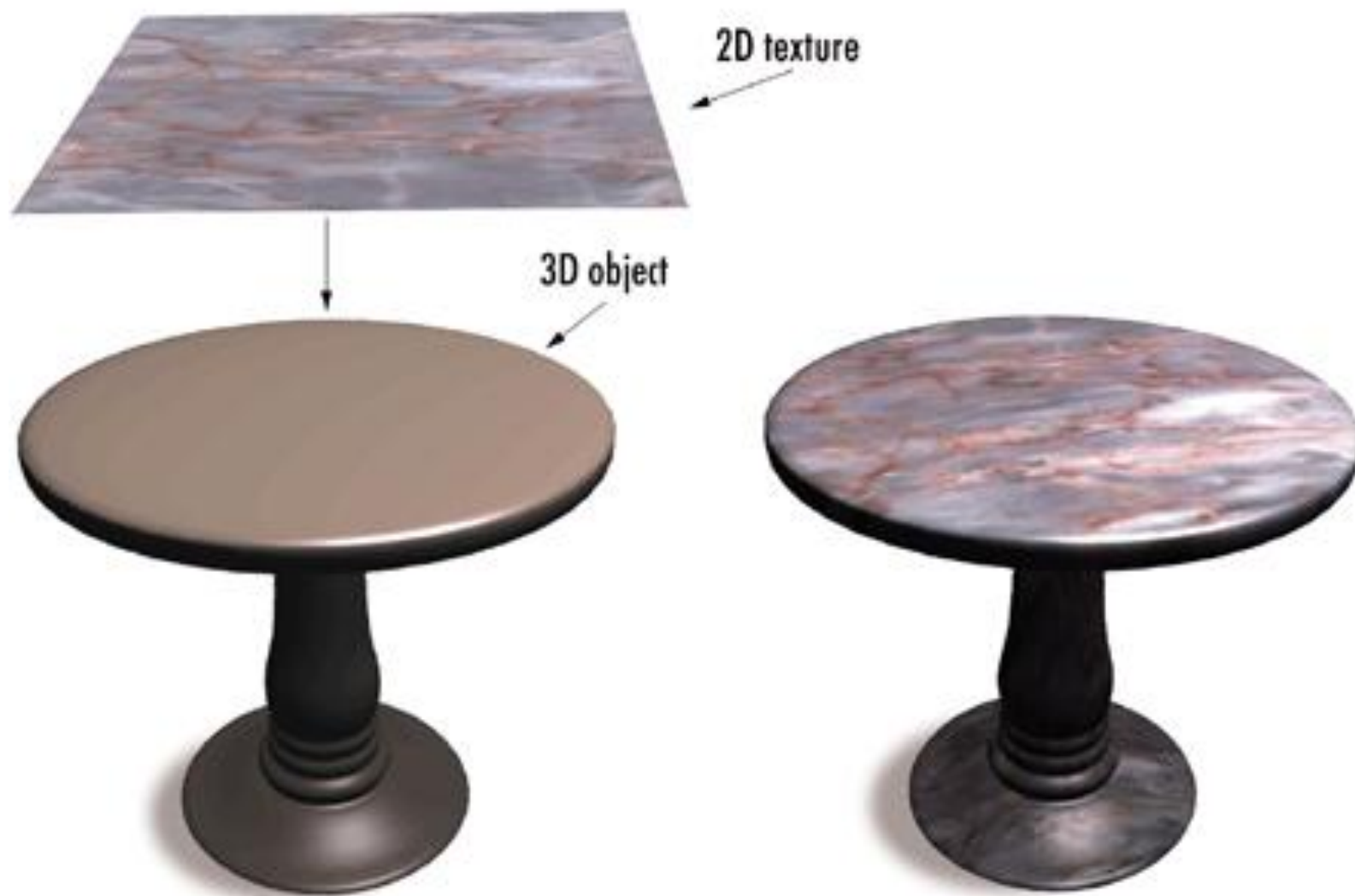
- 纹理是计算机图形学的一个基础技术。
- 它使得用户能够不必对模型的几何细节进行建模就可以表示丰富的表面细节。
- 这次课我们将讲述纹理的基本思想和应用。

纹理

- 请注意左图(无纹理)和右图(有纹理)的区别，右图更加逼真。



利用纹理图像增强真实感



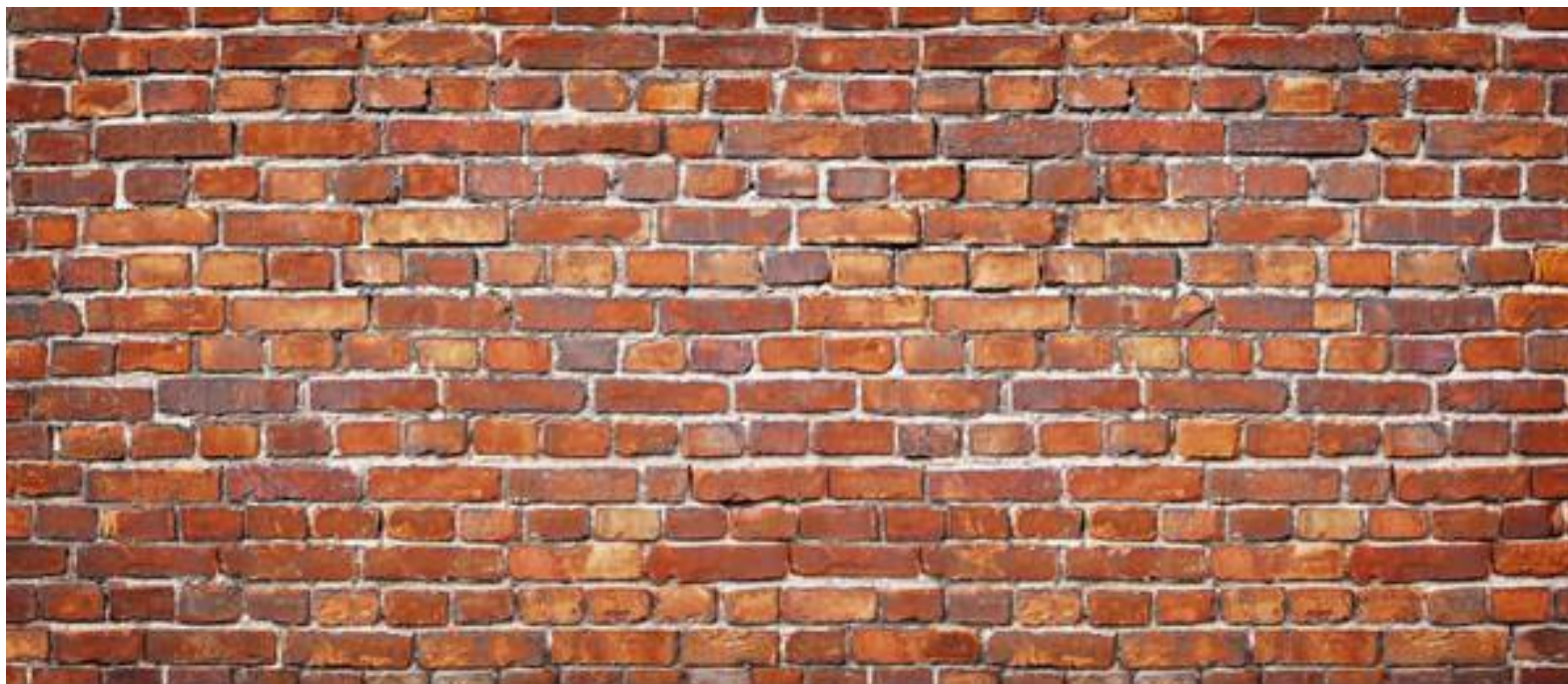
纹理

- 绘制真实感图像的最重要的任务之一就是対表面细节进行建模。
- 例如，如果我们想对一块砖墙进行建模。
 - ◆ 用大量的具有恰当表面颜色和反射属性的多边形对表面细节建模。
 - ◆ 用大的多边形平面对墙进行建模，并把一幅墙的图片贴在多边形上以使得这个多边形看起来像一面砖墙。



什么是纹理

- 这个例子中用于近似真实砖墙的图像，在图形学中即被称为纹理。
- 将纹理应用于物体表面的过程被称为纹理映射(texture mapping)或贴纹理(texturing)。



纹理解决了两个问题

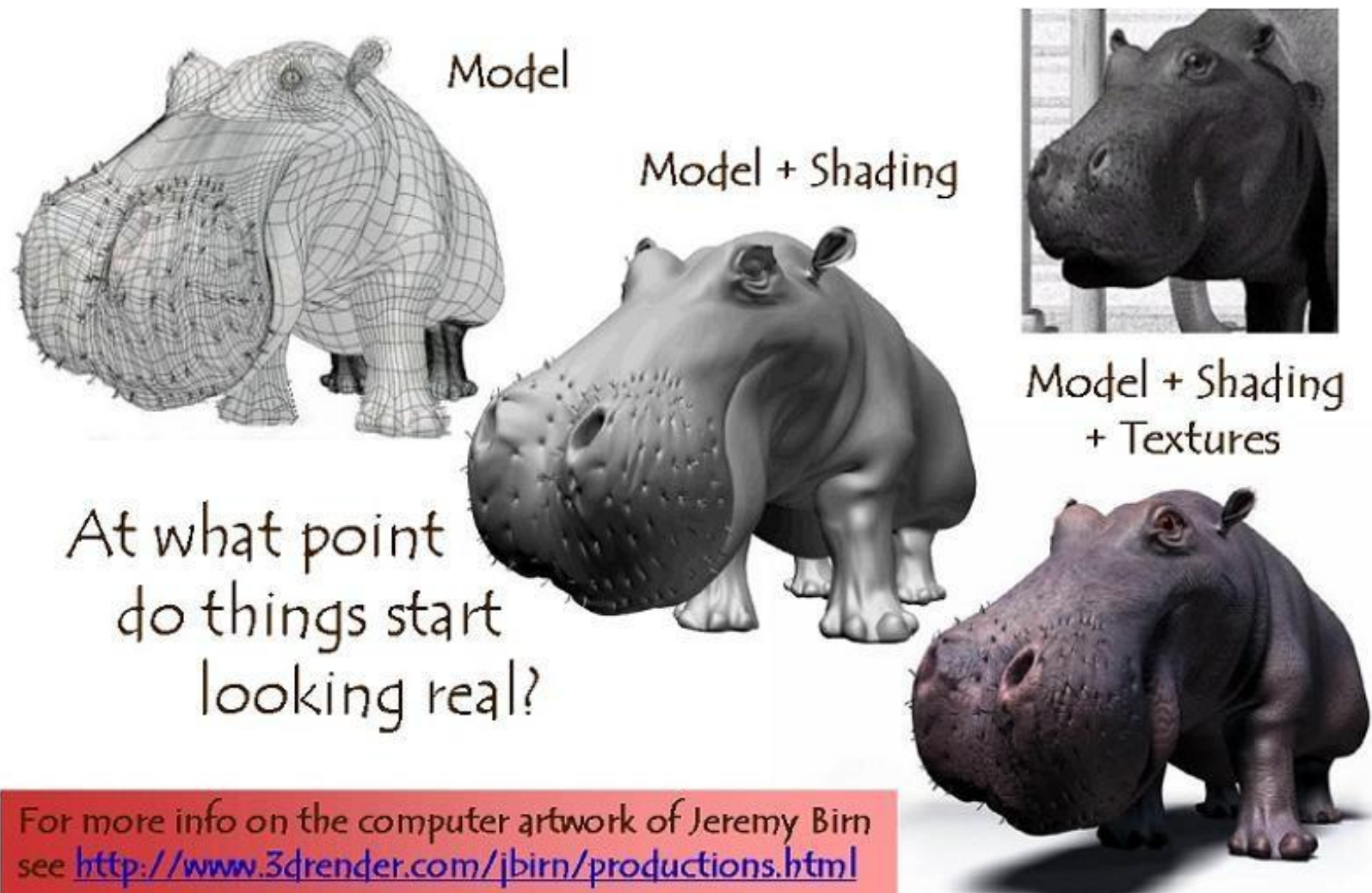
■ 提高建模效率

- ◆ 通过纹理图像，用户不必建模所有的几何及材质细节，使得用户可以将精力集中于其它更重要的事情上。

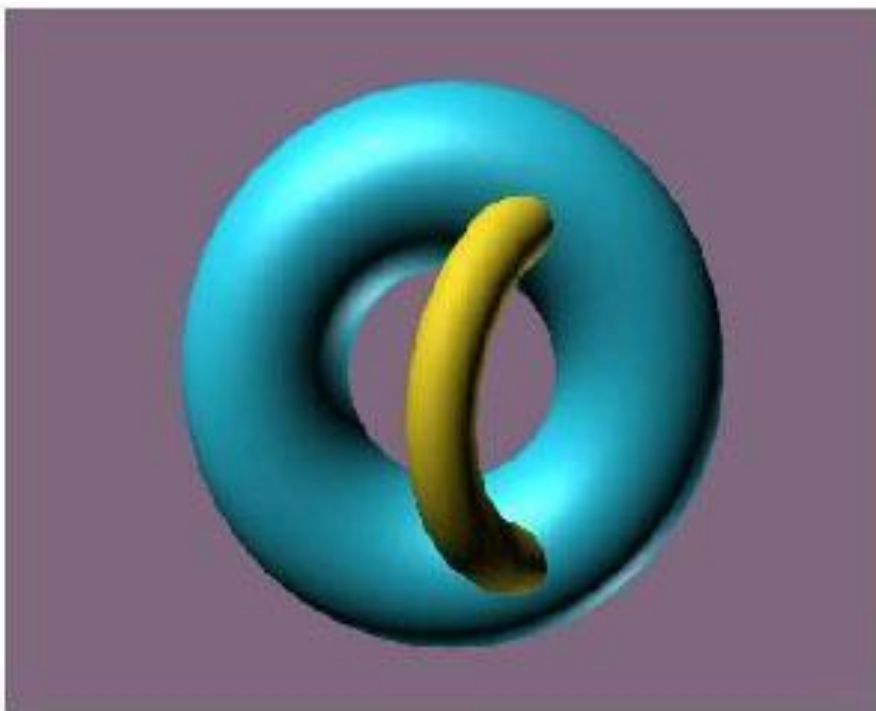
■ 提高渲染效率

- ◆ 通过渲染简单的多边形模型（例如：用一个单一的多边形表示砖墙）和纹理，而非具有不同 BRDF 的模型几何细节，可节约计算时间和计算资源。

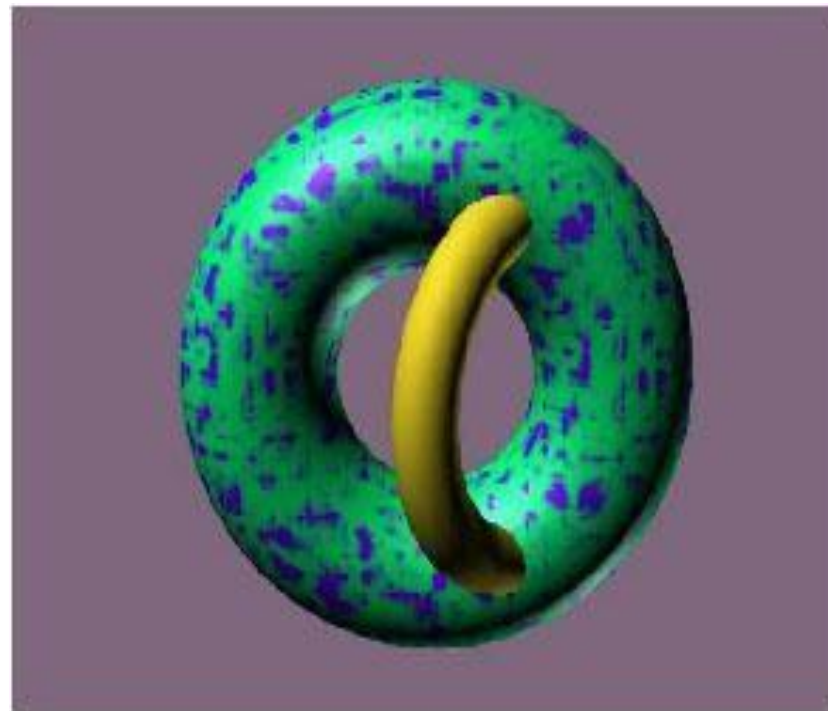
纹理示例



纹理示例

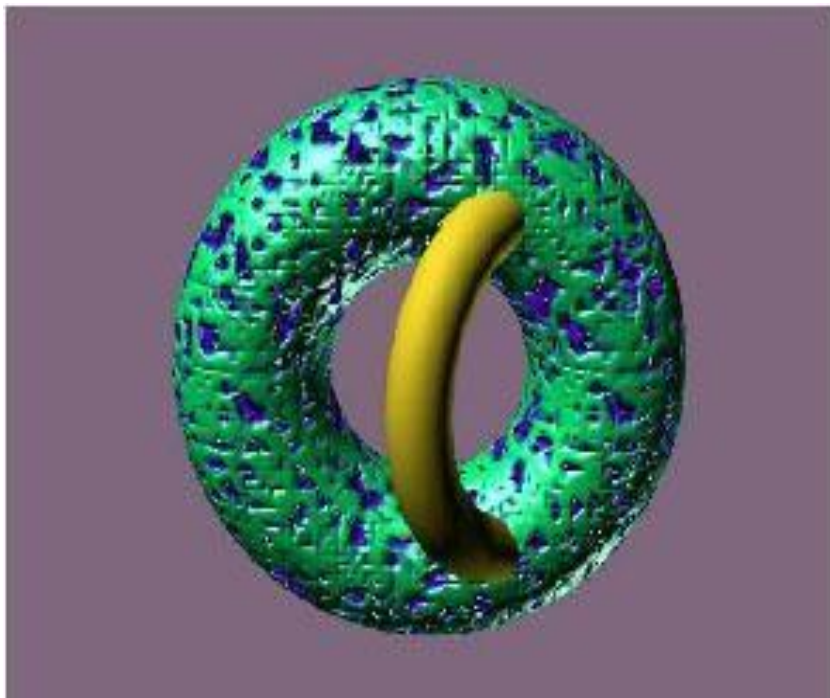


原始模型

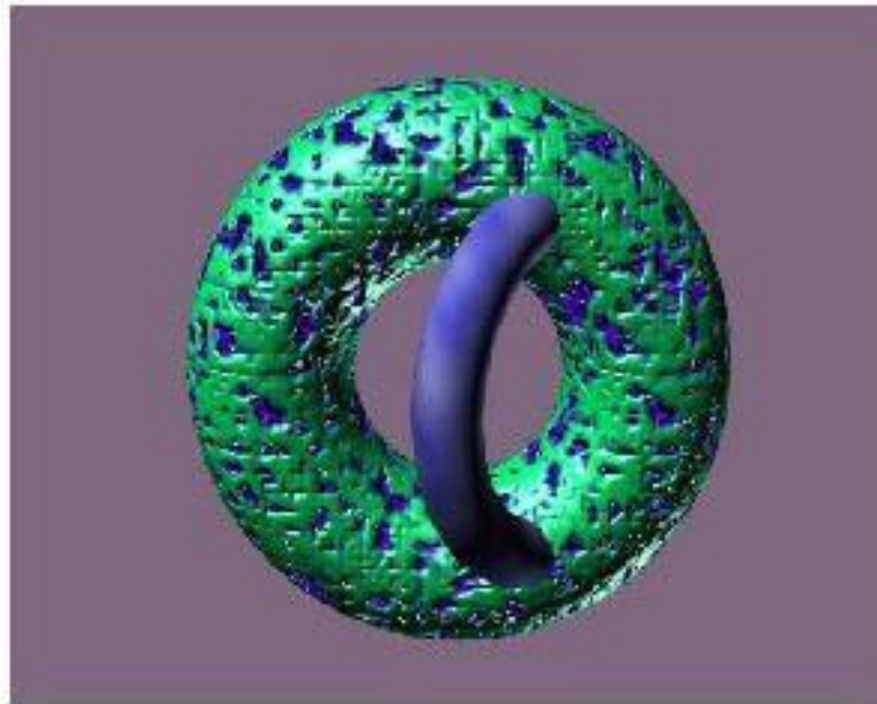


二维纹理

纹理示例



几何纹理



组合

纹理示例



模型



纹理



贴纹理的模型

纹理示例



Part 2: 基本操作

纹理使用三步骤

- 首先是**纹理采集**，这个步骤可以通过以下方式完成：手工绘制、拍照、过程纹理、纹理合成；
- 其次，**纹理贴图**，给定纹理图像和一个三维模型，还需要弄清楚如何将纹理映射到模型上。这可以通过给每个三维模型的顶点指定一个纹理坐标，并利用这个坐标把纹理图像上的色彩赋值给模型来实现。
- 最后，**纹理滤波** (texture filtering)：在决定如何将纹理映射到物体上去之后，绘制时还需仔细地对纹理进行采样。否则，容易产生一些不好的人为痕迹(artifacts)，这种问题的产生源于信号处理理论中的一些基本问题。

纹理采集

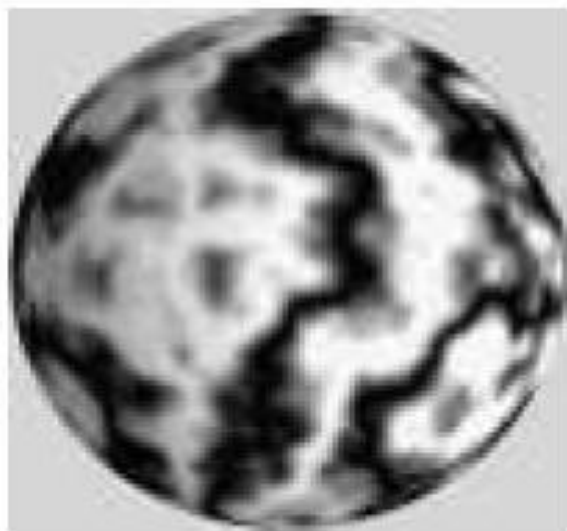
- 纹理映射的第一个任务就是纹理采集，采集纹理图像的方式多种多样。
 - ◆ 手工绘制纹理---艺术家。
 - ◆ 采集待建模物体的照片---摄影师。
 - ◆ 技术性的方法：
 - 过程纹理
 - 纹理合成

过程纹理

- 通过编写特殊的程序模拟物理形成过程或者材质外观，来合成纹理。
- 例如，像大理石或者木头等一些特定的模式，可以通过非常简单的函数来模拟。
- 优点：
 - ◆ 表示紧致，因为只需存储过程本身和参数。
 - ◆ 材质外观参数化，提供优良的可控性。
- 缺点：
 - ◆ 只适用于一些特定的纹理。
 - ◆ 无法合成未知材质。

过程纹理

- 最著名的过程式合成方法: 柏林噪声[Perlin2002]



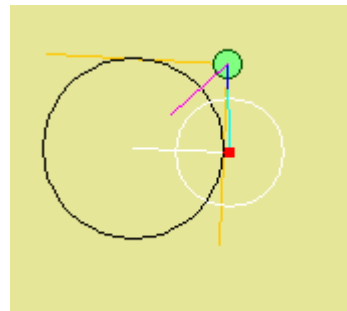
marble



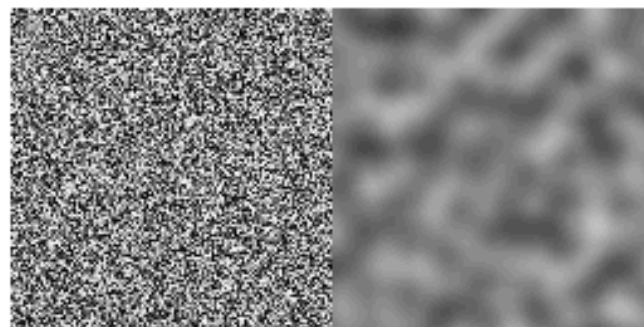
wood

过程纹理

- 通过噪音的随机性的来模拟真实的自然现象
- 太随机了也不自然
- Perlin 噪音通过在不同尺度上进行叠加以模拟自然的纹理

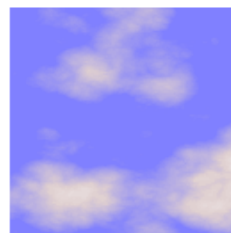
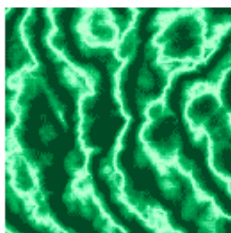
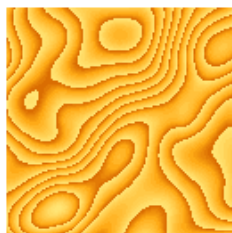
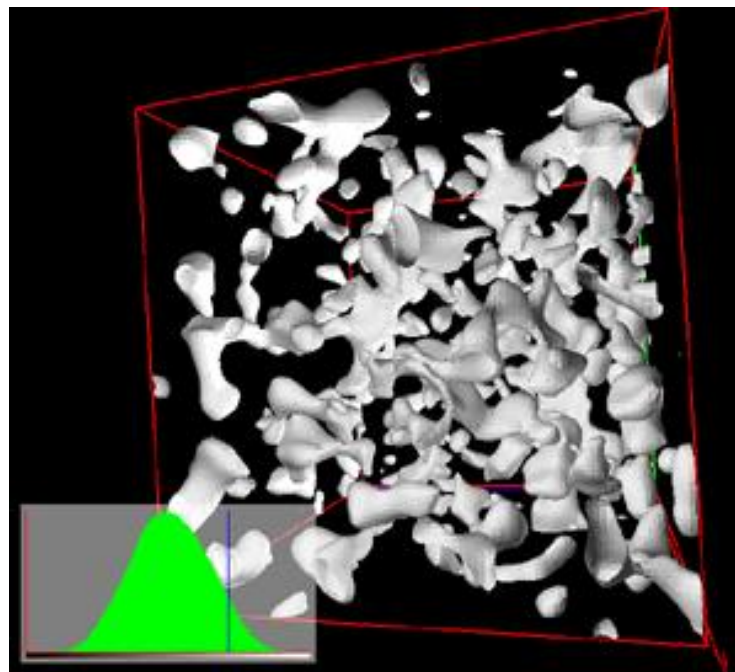


散步可以视为带有噪音

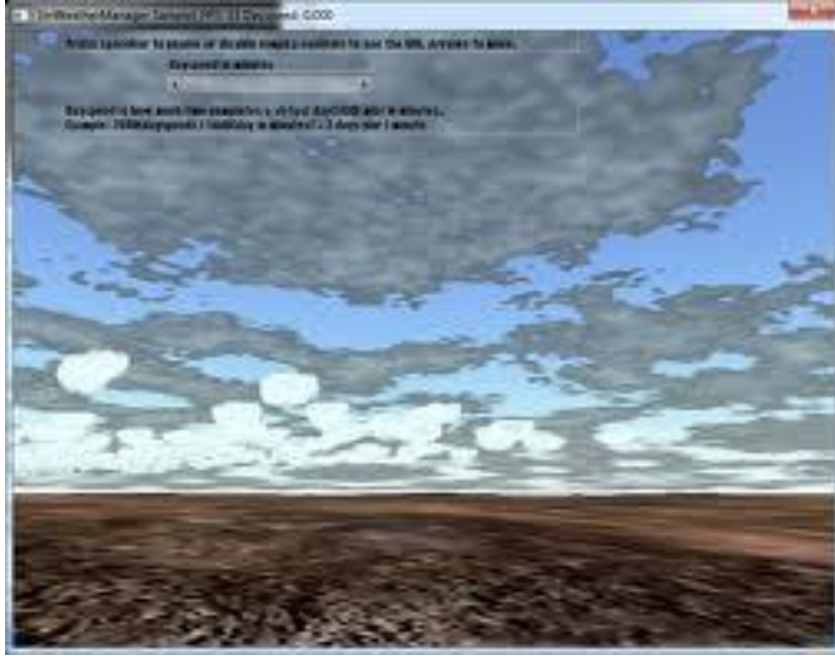


Non-coherent noise (left) and Perlin noise (right)

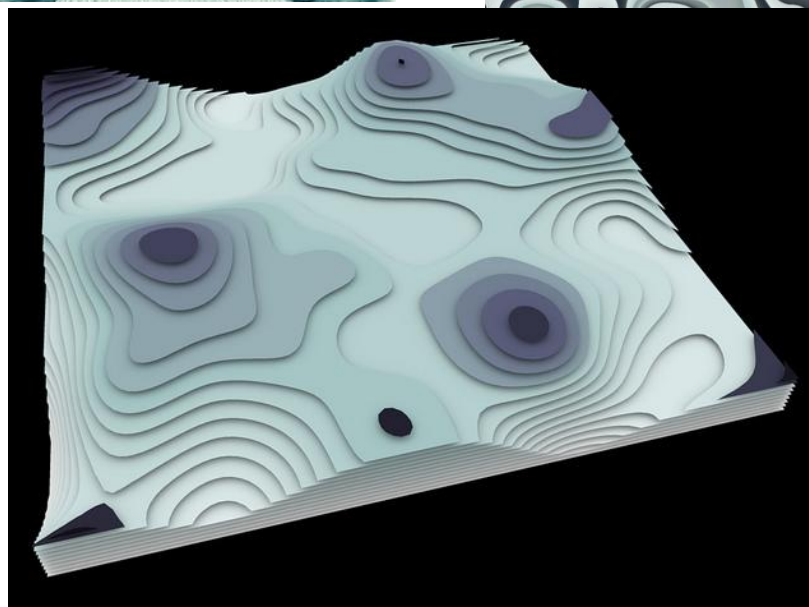
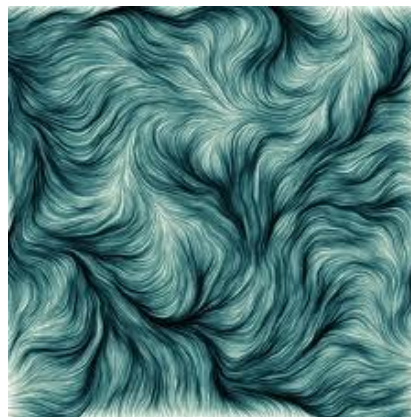
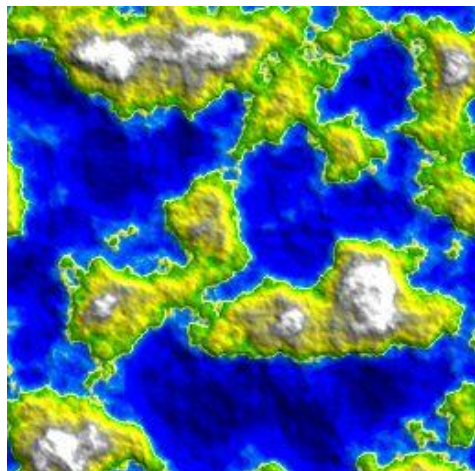
Perlin 噪音应用



Perlin 噪音应用



Perlin 噪音应用

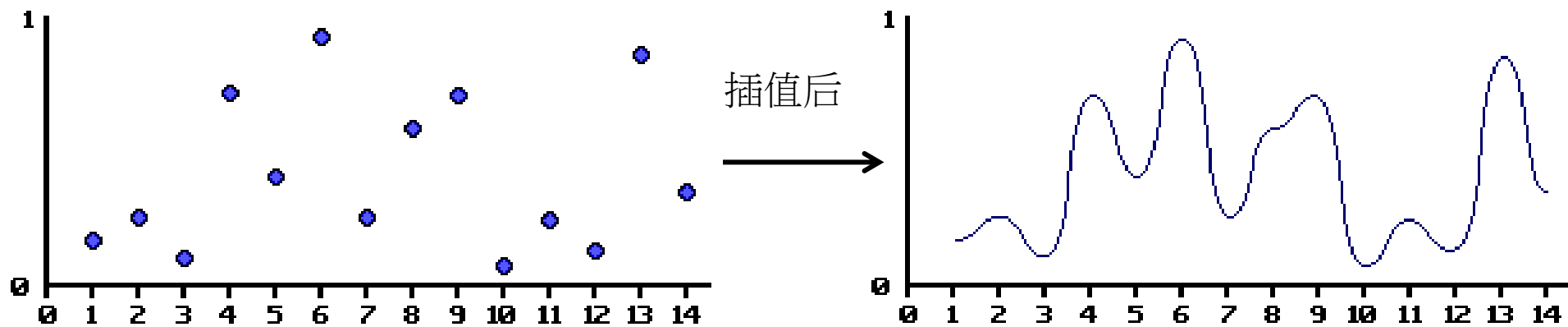


噪声函数

- 噪声函数本质上是随机数生成器
- 使用一个整型作为生成器的输入，随机生成一个浮点数

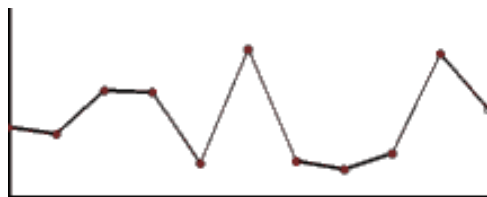
$$f:Z \rightarrow (-1,1)$$

- 例子:



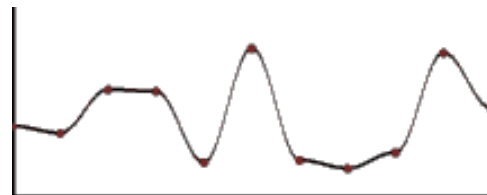
插值

Linear Interpolation



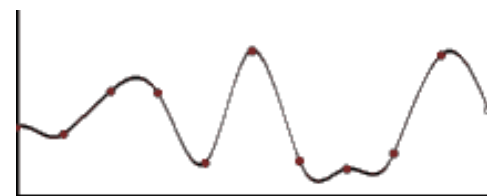
```
function Linear_Interpolate(a, b, x)
return a*(1-x) + b*x
```

Cosine Interpolation



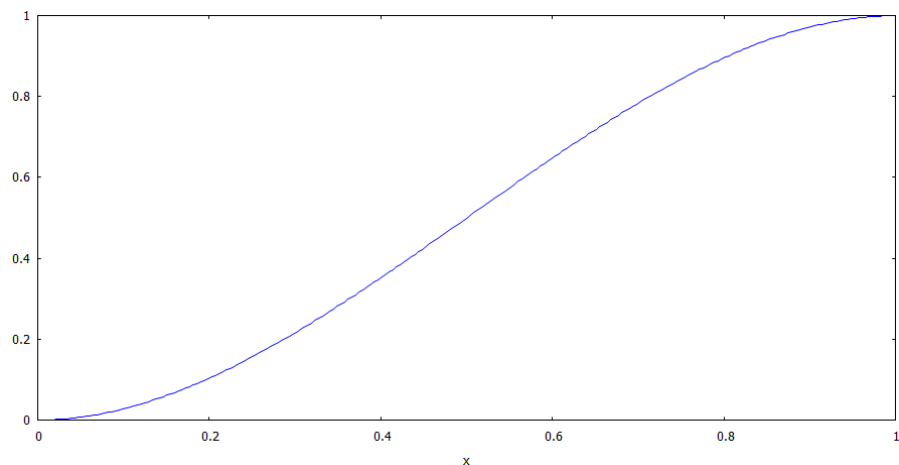
```
function Cosine_Interpolate(a, b, x)
  ft = x * 3.1415927      x = 0, ft = 0, f = 0
  f = (1 - cos(ft)) * .5  x = 1, ft = p, f = 1
  return a*(1-f) + b*f
```

Cubic Interpolation

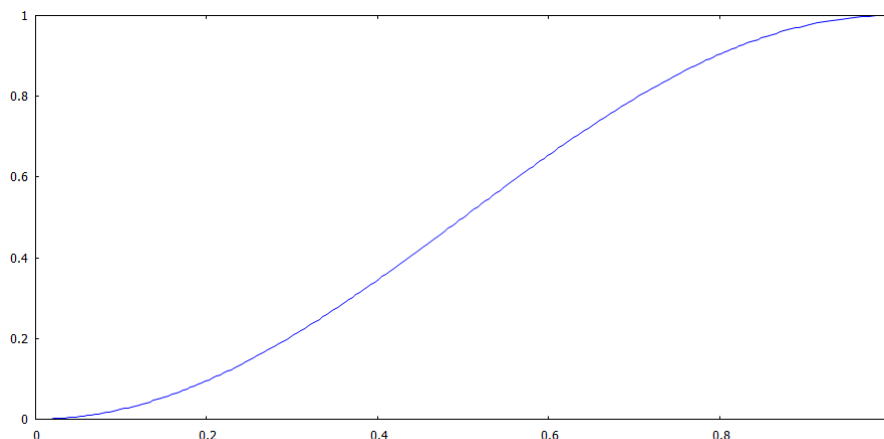


```
function Cubic_Interpolate(a, b, x)
  f = 2*x*x - 2*x*x*x
  return a*(1-x) + b*f
```

插值



$$f(x) = \frac{1}{2} (1 - \cos(\pi x))$$

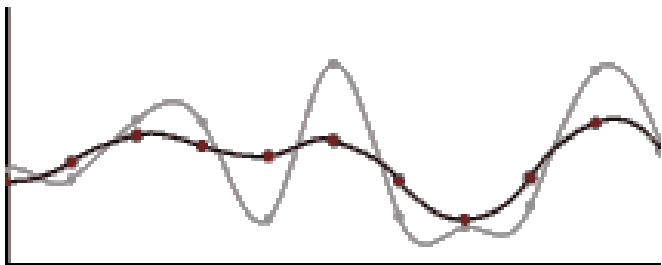


$$f(x) = 3x^2 - 2x^3$$

使用更少的计算达到更好的插值效果

对噪音进行光滑

■ 对噪音使用光滑滤波算子



```
function SmoothNoise_1D(x)

    return Noise(x)/2 + Noise(x-1)/4 + Noise(x+1)/4

end function
```



1	2	1
2	4	2
1	2	1

```
function SmoothNoise_2D(x, y)

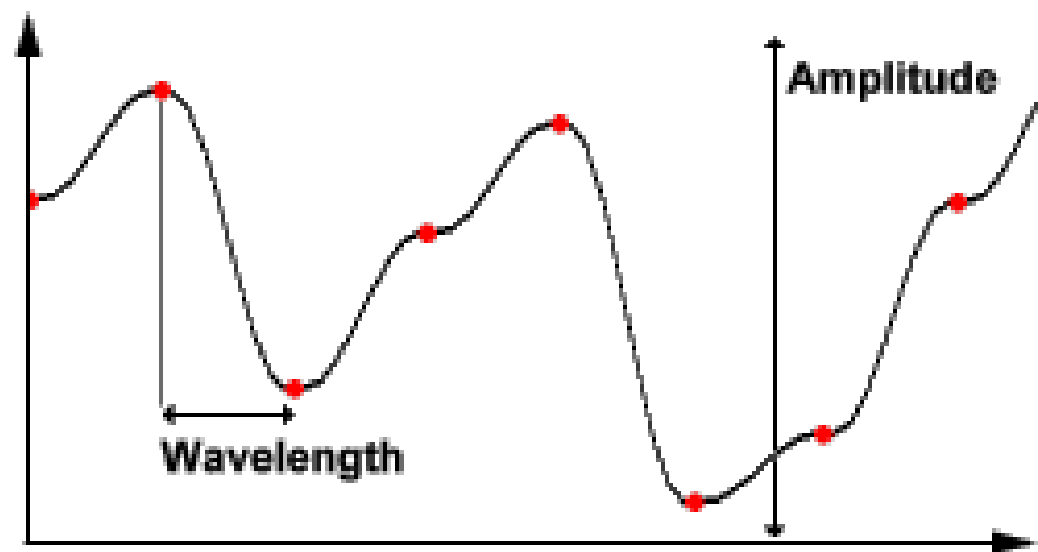
    corners = ( Noise(x-1, y-1)+Noise(x+1, y-1)+Noise(x-1, y+1)+Noise(x+1, y+1) ) / 16
    sides   = ( Noise(x-1, y) +Noise(x+1, y) +Noise(x, y-1) +Noise(x, y+1) ) / 8
    center  = Noise(x, y) / 4

    return corners + sides + center

end function
```

振幅 & 频率

- 红点表示沿着自变量维度上的采样点

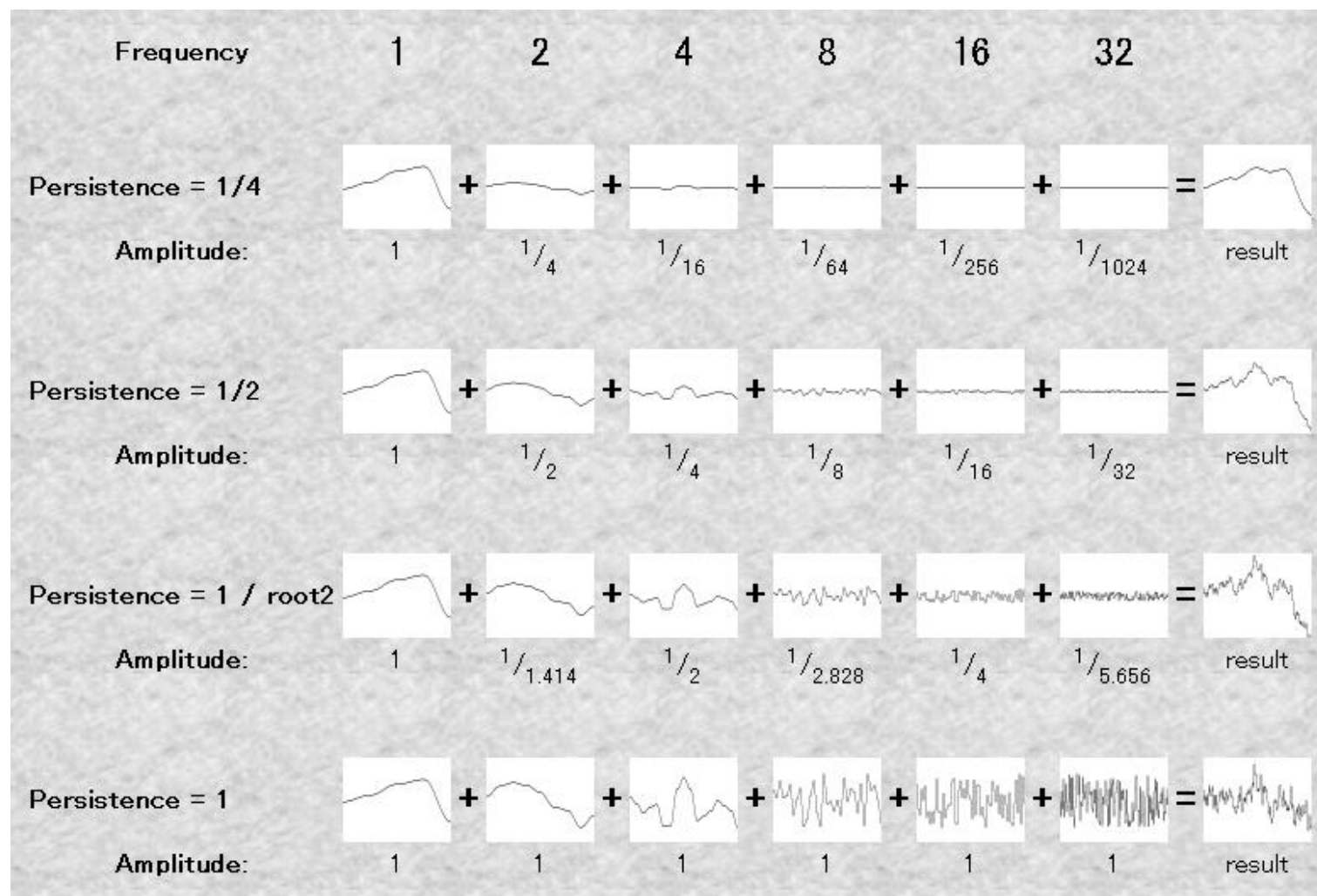


- 频率=1/波长

持久性 (Persistence)

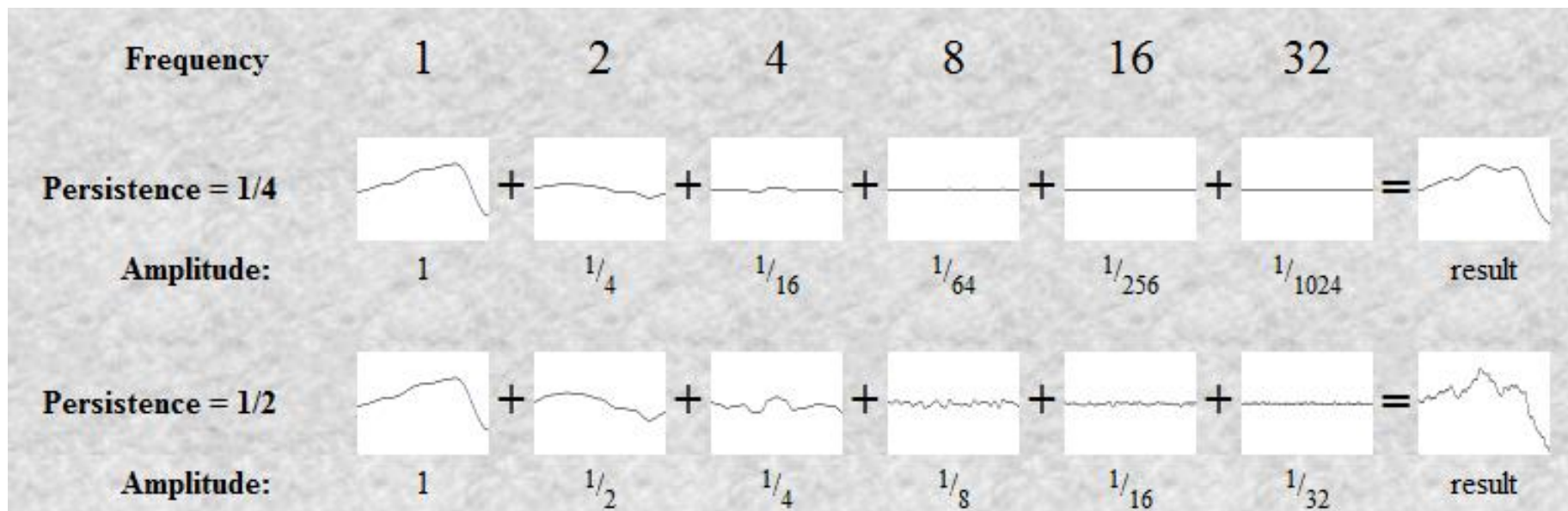
- 使用不同频率和振幅的噪音函数进行叠加
- 频率 (Frequencies) = 2^i
- 振幅 (Amplitude) = $Persistence^i$

持久性 (Persistence)



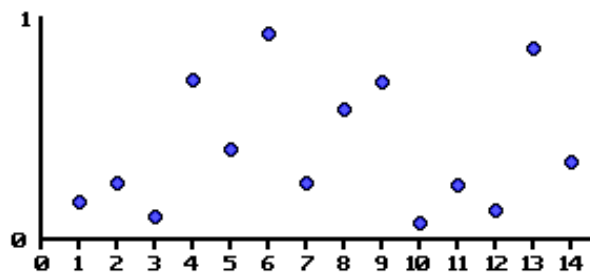
创建 Perlin 噪音函数

- 使用具有不同频率和振幅的光滑噪音函数
- 想法类似于分形与傅立叶级数
- 对这些噪音函数进行叠加

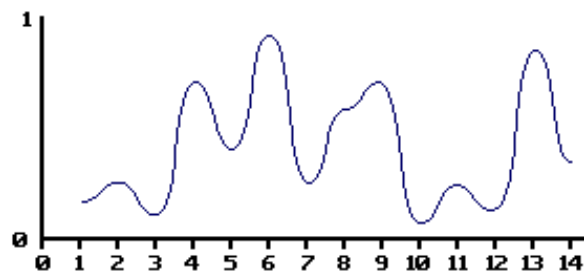


一维 Perlin 噪音总结

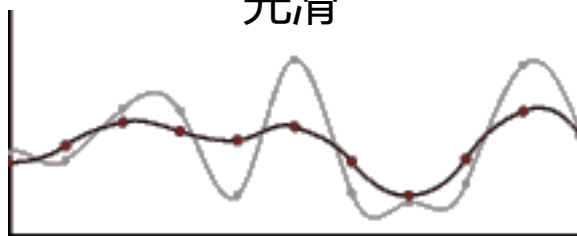
随机数生成



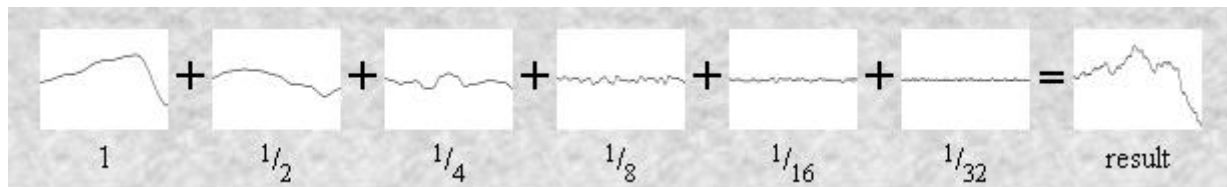
样条插值



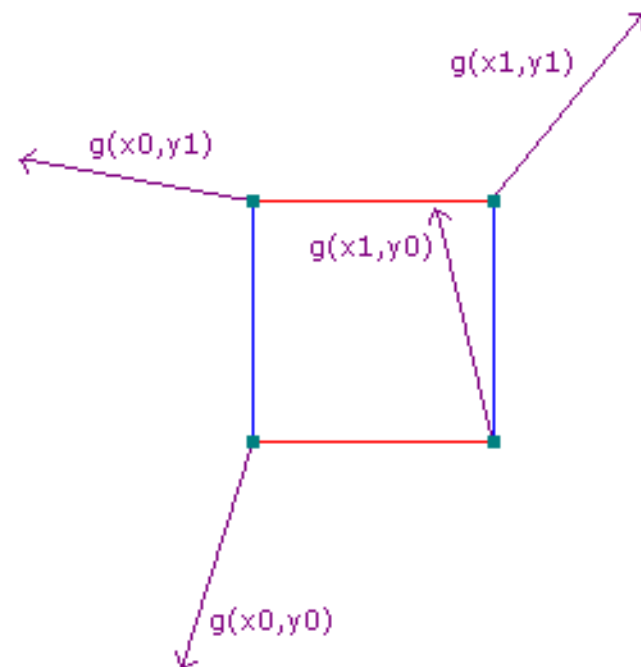
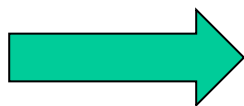
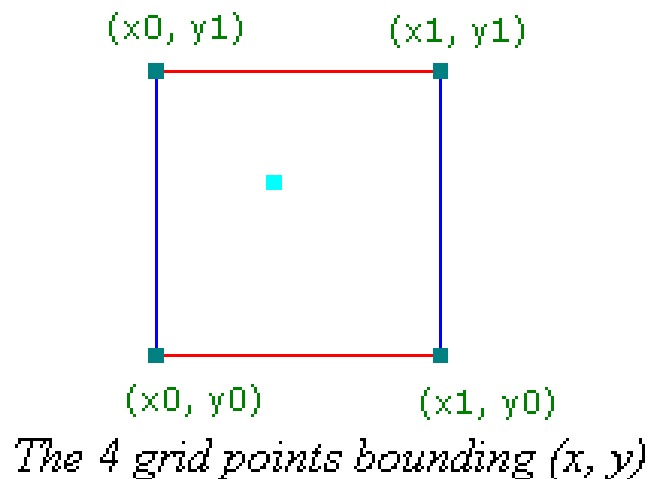
光滑



对不同频率的噪音函数进行叠加



二维 Perlin 噪音



The 4 pseudorandom gradients associated with the grid points

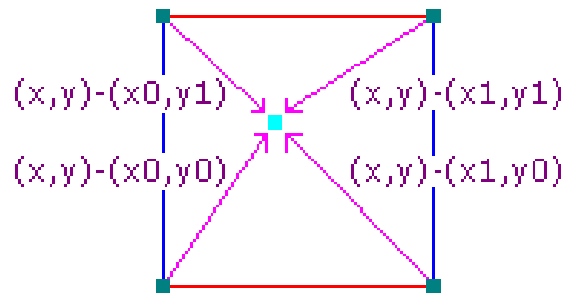
二维 Perlin 噪音

$$s = g(x_0, y_0) \cdot ((x, y) - (x_0, y_0)),$$

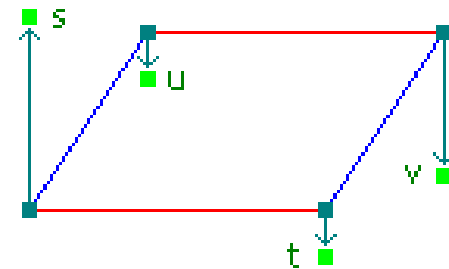
$$t = g(x_1, y_0) \cdot ((x, y) - (x_1, y_0)),$$

$$u = g(x_0, y_1) \cdot ((x, y) - (x_0, y_1)),$$

$$v = g(x_1, y_1) \cdot ((x, y) - (x_1, y_1)).$$

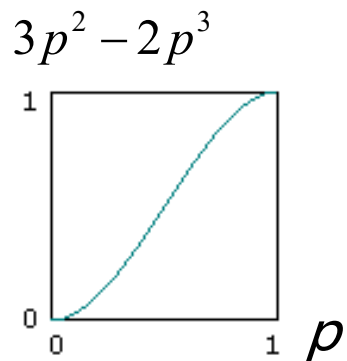


Vectors from the grid points to (x, y)

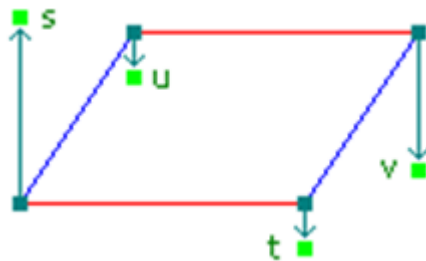


Influences from the grid points

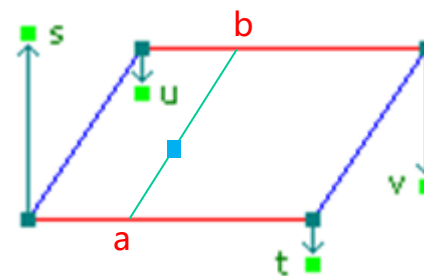
二维 Perlin 噪音



样条插值



双线性插值



$$S_x = 3(x - x_0)^2 - 2(x - x_0)^3$$

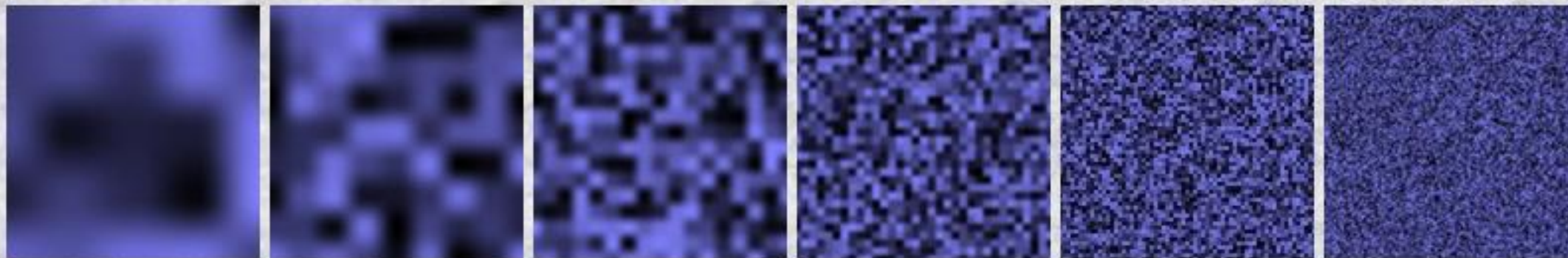
$$a = s + S_x(t - s)$$

$$b = u + S_x(v - u)$$

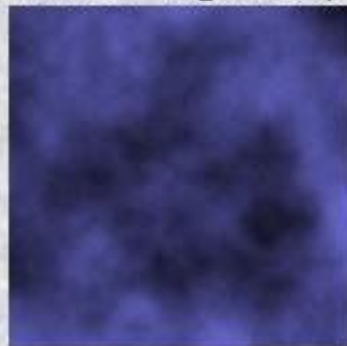
$$S_y = 3(y - y_0)^2 - 2(y - y_0)^3$$

二维的例子

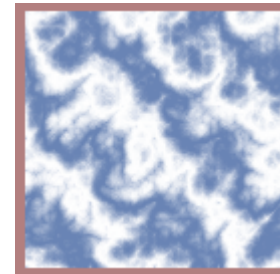
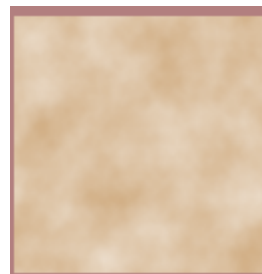
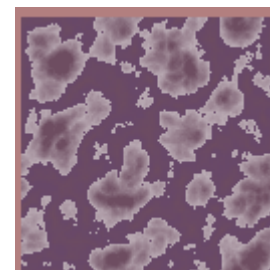
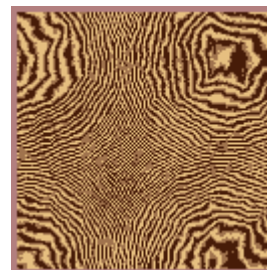
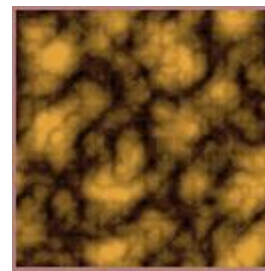
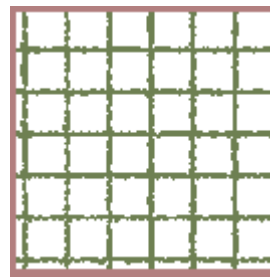
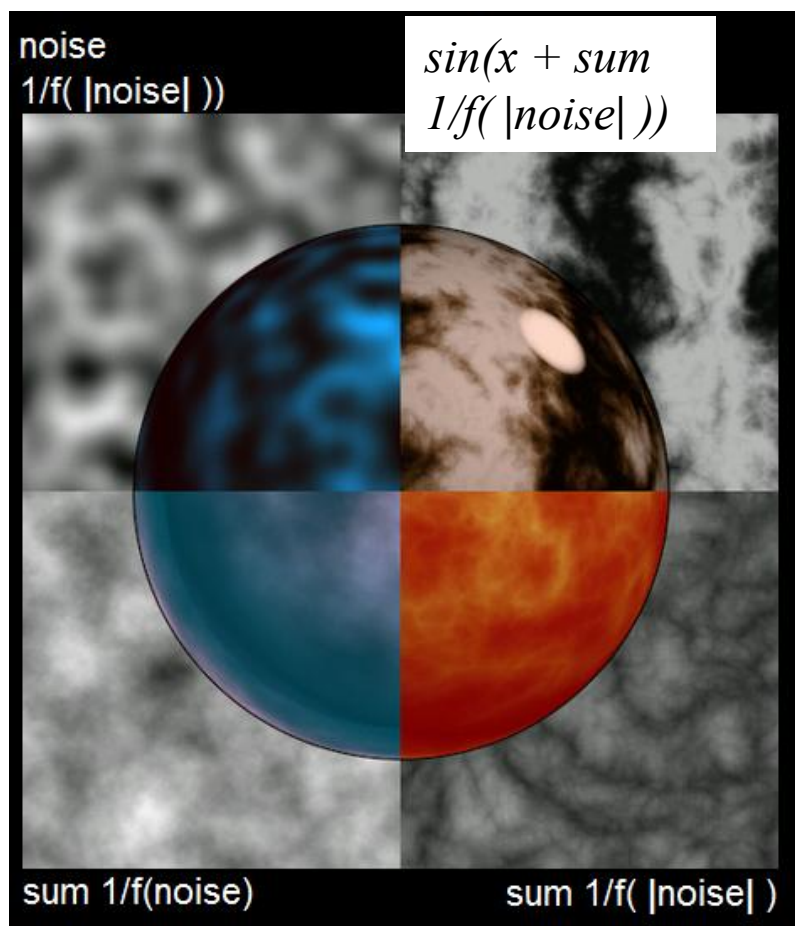
Some noise functions are created in 2D



Adding all these functions together produces a noisy pattern.



例子



Perlin 噪音的应用

■ 一维:

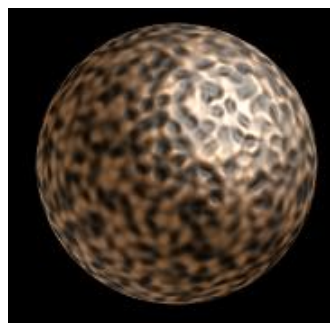
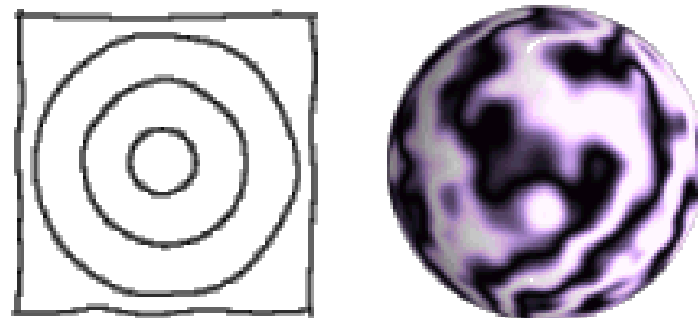
- ◆ 人体运动路径, 笔划

■ 二维:

- ◆ 地形等

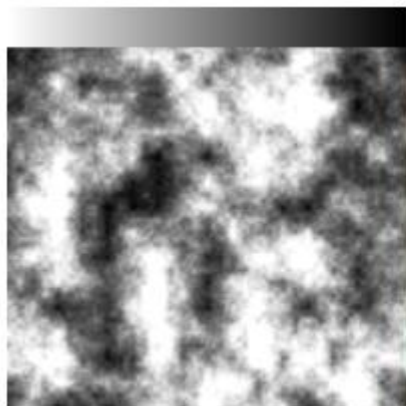
■ 三维:

- ◆ 云彩, 实体纹理

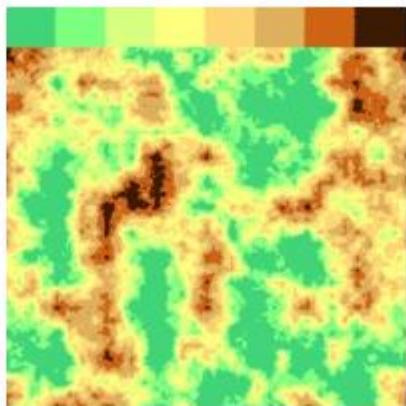


游戏中的 Perlin 噪音

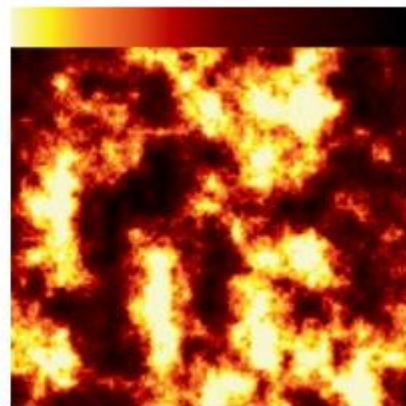
■ 纹理合成



Grayscale gradient



Gradient with discrete colours



Fire gradient

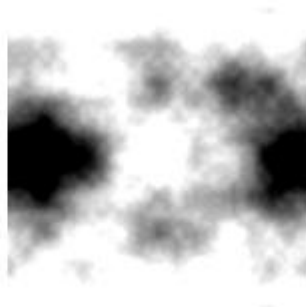
■ 纹理混合



Image 1



Image 2



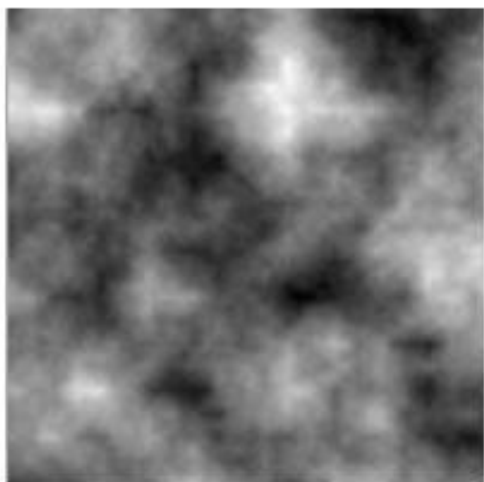
Perlin noise



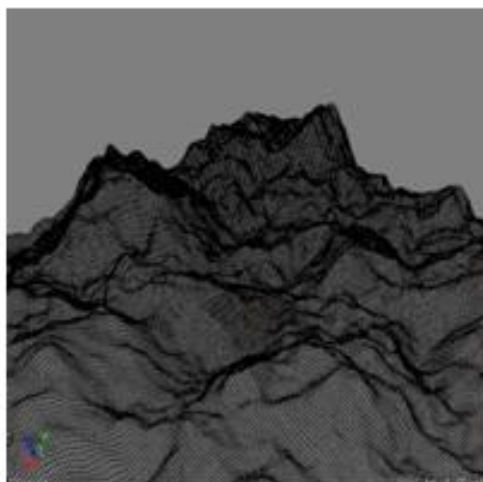
Blend using noise

游戏中的 Perlin 噪音

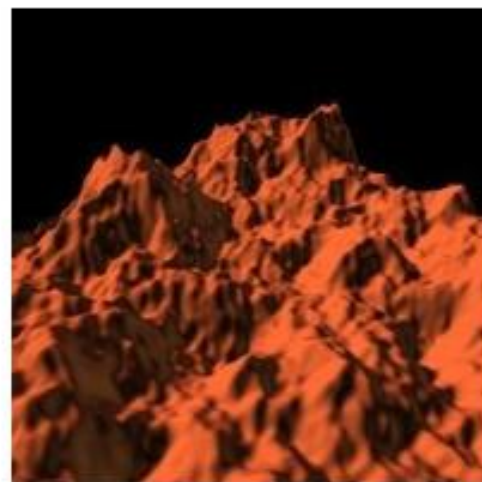
■ 地形



*Height map generated from
Perlin noise*



*Mesh generated from height
map*



Rendered mesh

纹理合成

- 什么是纹理合成?
 - ◆ 从一个特定的样本合成一个新的纹理。

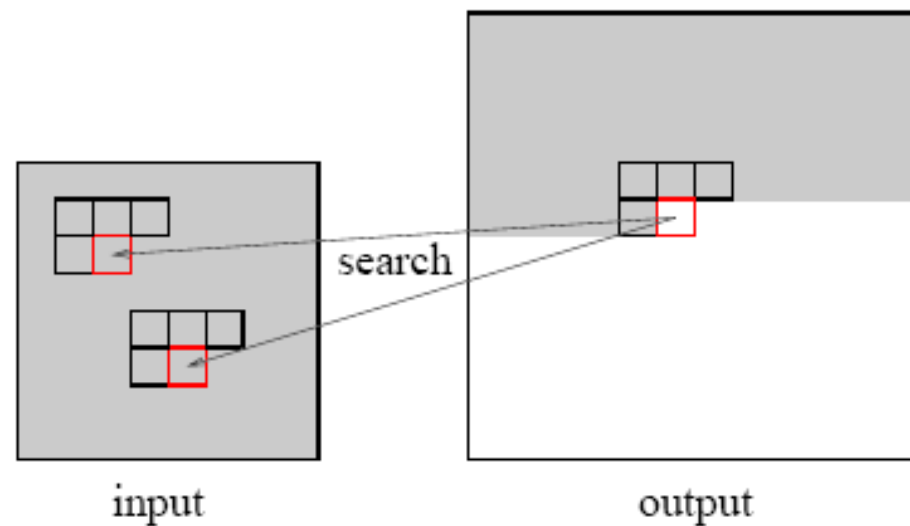
纹理合成

- 在计算机图形学中，纹理合成技术可以分为2类：
 - ◆ 基于像素(Pixel-based)的纹理合成
 - ◆ 基于块 (Patch-based)的纹理合成

纹理合成

- 基于像素的 (pixel based) 纹理合成
- 主要思想

逐像素地合成新的纹理，
其中每个像素的值由局部
邻域来确定，(例如：3*3，
5*5)，选择输入像素中具
有最相似邻域的那个。

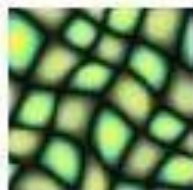


纹理合成

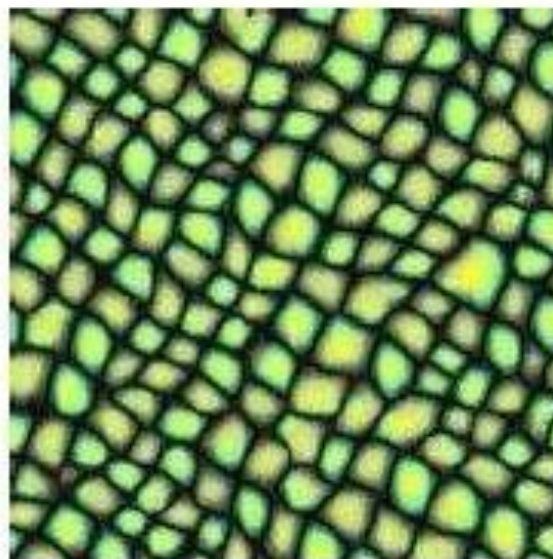
- 基于像素的纹理合成

- 这类工作包括：

- ◆ “Texture Synthesis by Non-parametric Sampling” [Efros99] (2954cites)
- ◆ “Fast texture synthesis using tree-structured vector quantization” [Wei and Levoy00] (1031 cites)



sample

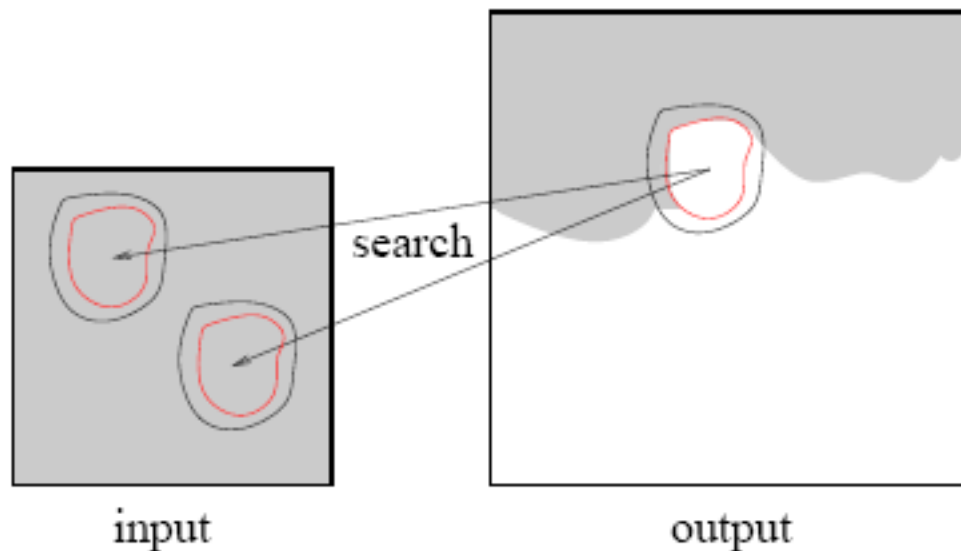


result

纹理合成

■ 基于块的 (Region based) 纹理合成

- ◆ 基于像素的方法可以通过合成区块来改进。
- ◆ 最重要的方法之一是 Graph-cut[Kwatra03]，该方法是目前为止效果最好的。
- ◆ 区块的选择同样是根据邻域匹配来进行的。

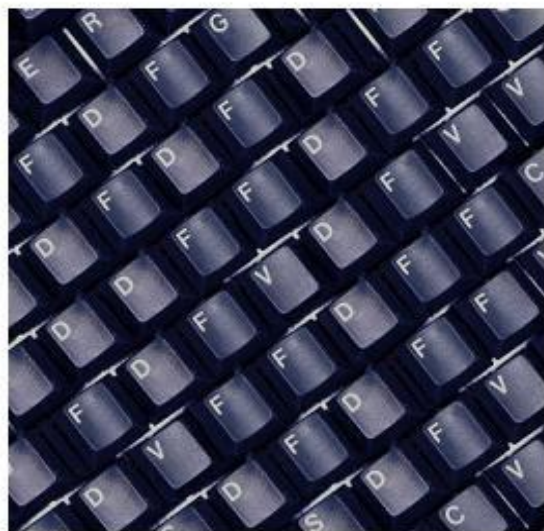


纹理合成

■ 一些Graph-Cut算法的结果



Input



Graph cut



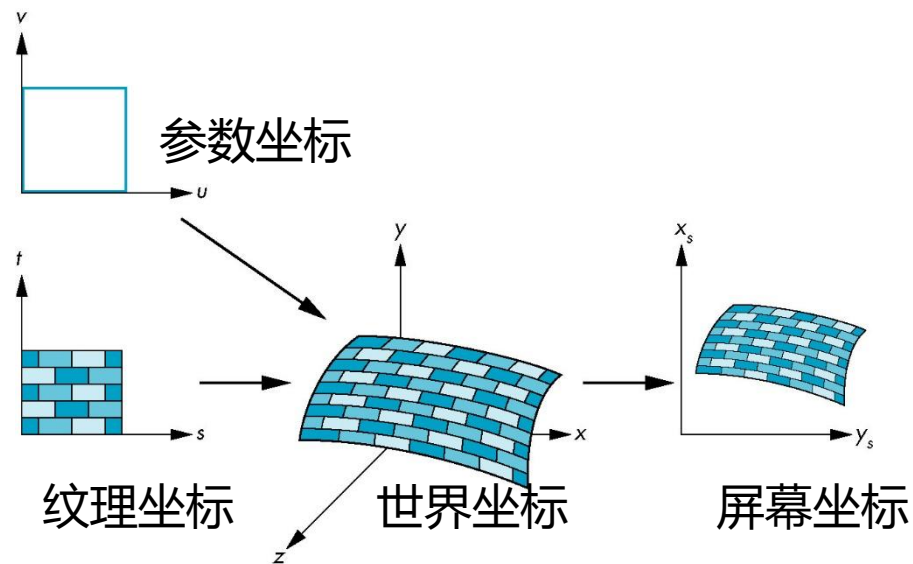
Sample Texture



Synthesized Texture

纹理映射

- 什么是纹理映射?
- 给定一个模型和2维纹理图像。将图像映射到模型上：通过一个将模型点映射到 (u, v) 图像坐标的方程来完成。这个方程被称为**表面映射函数**。给模型上的点着色时，我们从2维纹理中寻找适当的像素，并用该像素的值去影响最终的颜色。



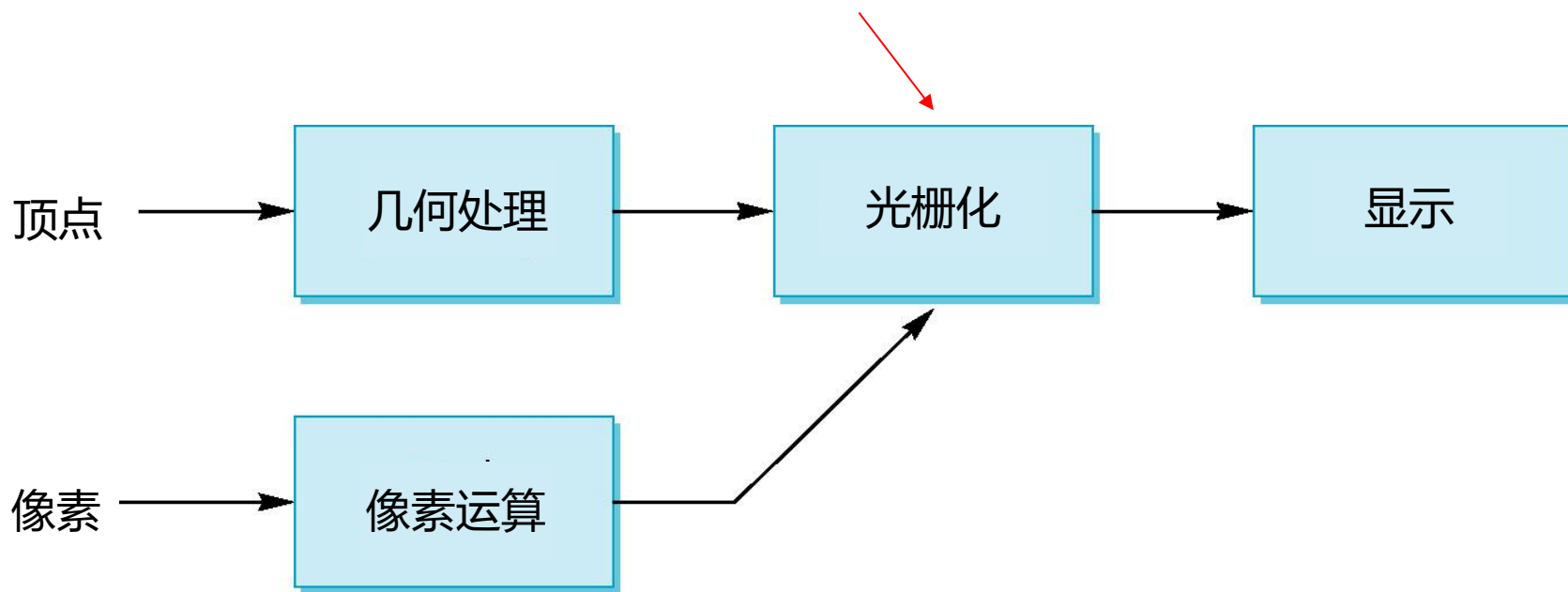
纹理映射

■ 如何制定表面映射函数?

- ◆ 通过**自然参数化**(Natural parameterization), 对于规则物体, 如球体、立方体、圆柱体...
- ◆ 通过**手工制定**纹理坐标, 对于复杂物体, 为每一个顶点指定一个纹理坐标。

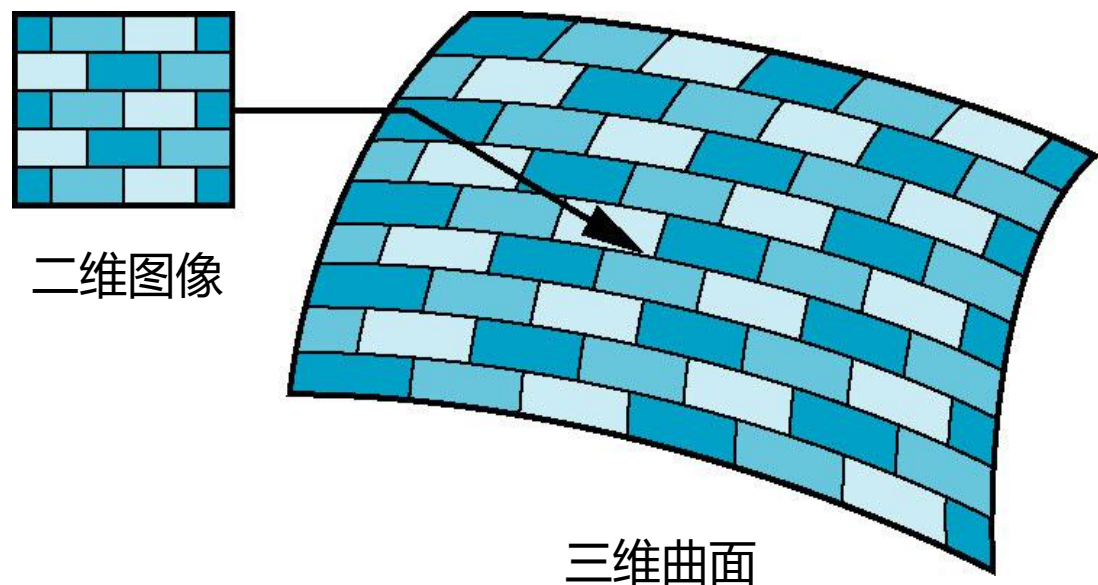
映射在什么地方进行？

- 映射技术是在输出流水线的最后阶段实现的
 - ◆ 非常有效，因为在经过所有的操作后，减少了许多不必要的映射

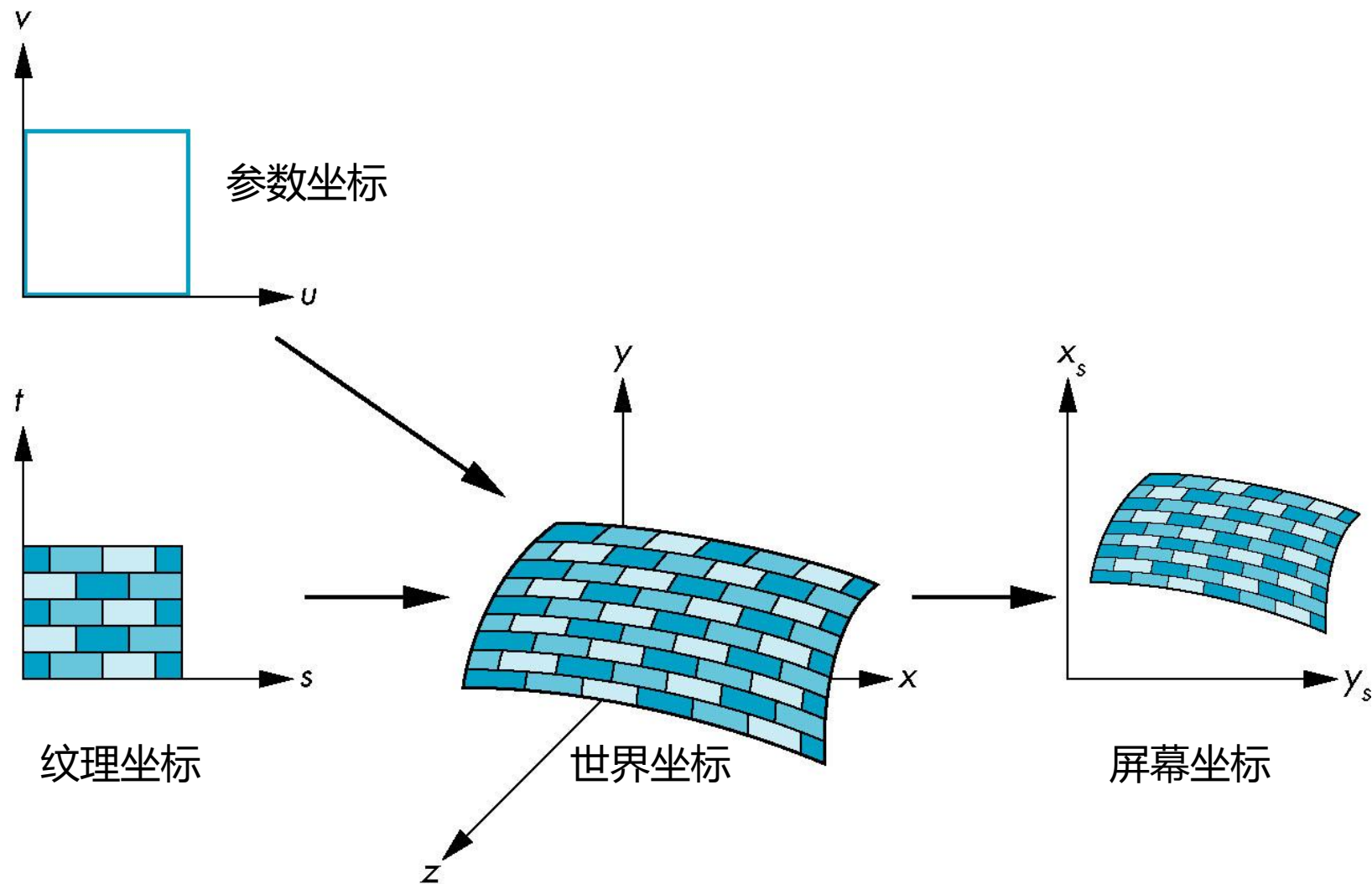


坐标系

- 参数坐标系
 - ◆ 可以用来建立曲面
- 纹理坐标系
 - ◆ 用来区别要被映射的图像上的点
- 世界坐标系
 - ◆ 从概念上说，就是映射发生的地方
- 屏幕坐标系
 - ◆ 最终图像生成的地方



纹理映射框架

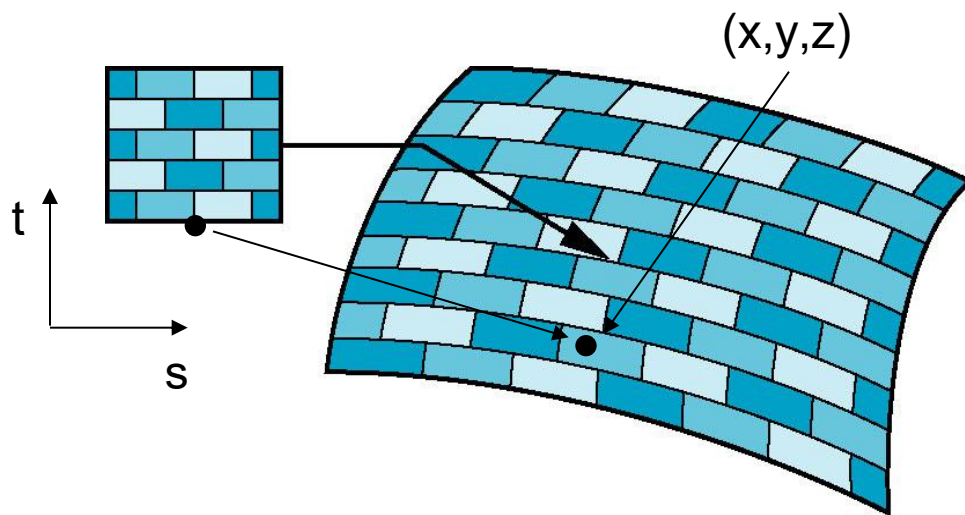


映射函数

- 基本问题就是如何定义映射
- 考虑从纹理坐标到曲面上一点的映射
- 直观地看，应当需要三个函数

$$x = x(s,t), y = y(s,t), z = z(s,t)$$

- 虽然从概念上讲，最终必定用到上述的函数，但实际采用的是却是间接的方法



逆映射

■ 我们需要的是逆向操作

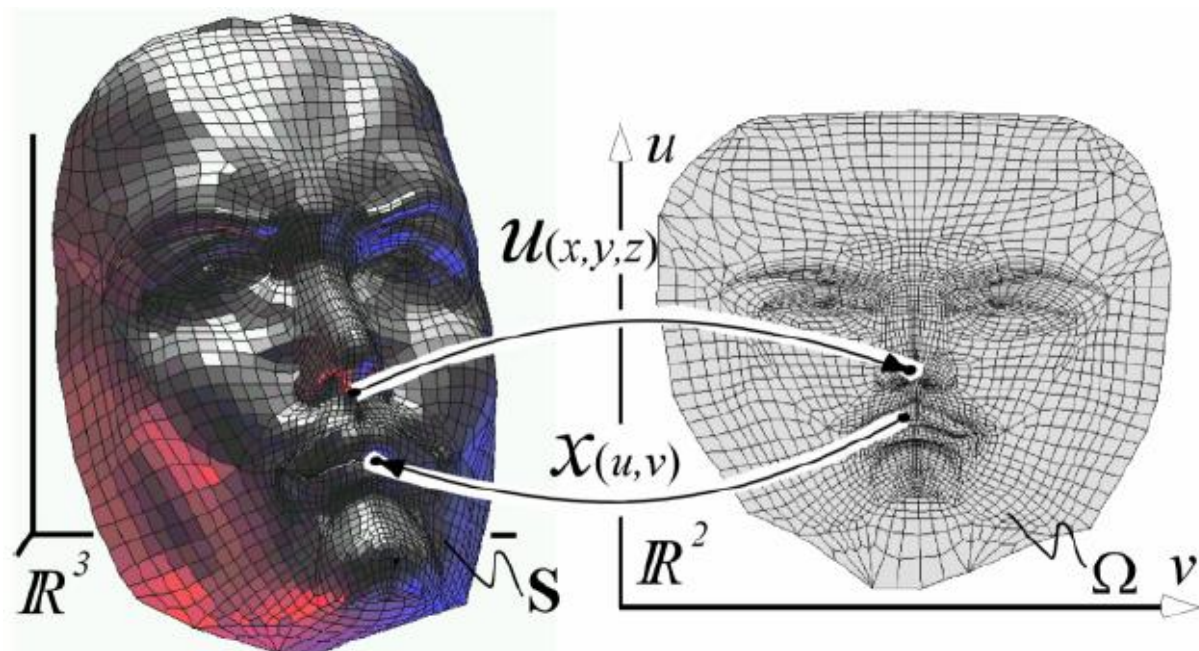
- ◆ 给定一个像素，我们想知道它对应于对象上的哪个点
- ◆ 给定对象上的一个点，我们想知道它对应于纹理中的哪个点

■ 此时需要如下形式的映射

- ◆ $s = s(x, y, z), t = t(x, y, z)$

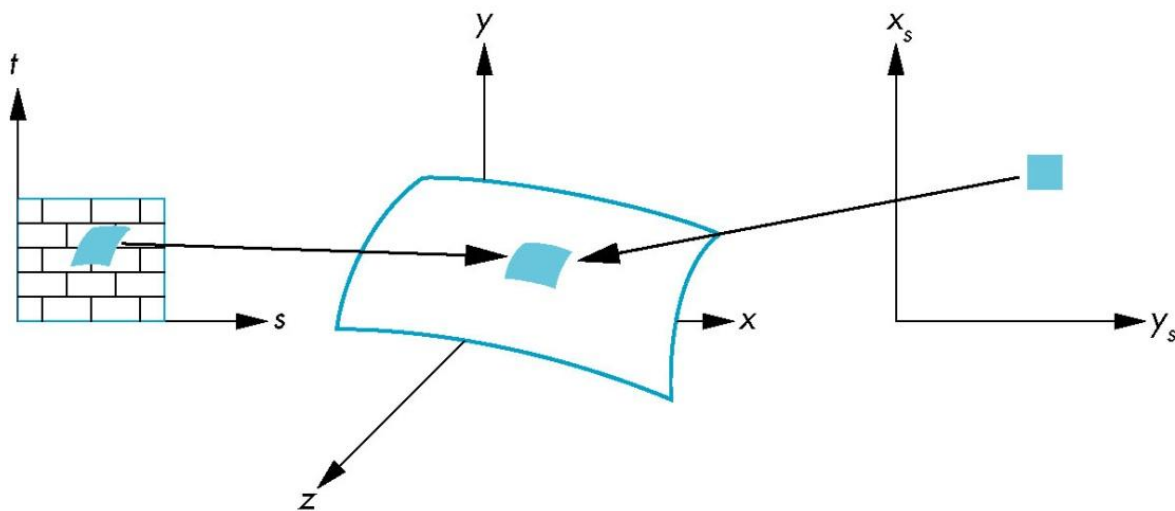
■ 这样的函数一般是很难求出来的？

- ◆ 参数化！



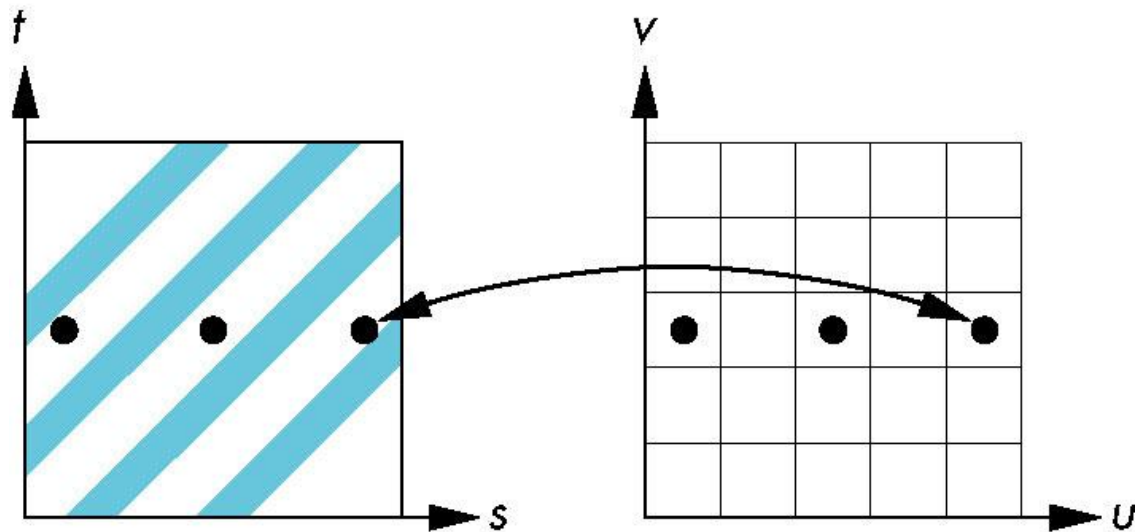
实际困难

- 假设要计算中心在 (x_s, y_s) 的一个矩形像素的颜色，中心点对应于对象上的点 (x, y, z)
- 如果对象是弯曲的，那么矩形像素的原像是一个曲边四边形
- 这个曲边四边形在纹理上的原像才是对当前矩形像素的颜色有贡献的纹理元素



解决方法

- 先不管如何定义逆映射，只考虑如何确定当前像素的颜色（或者说明暗效果）
- 一种方法是用当前像素块的原像的中心对应的纹理元素的颜色
 - ◆ 走样（aliasing）



映射的确定

- 主要分为两种方式

- ◆ 自然参数化
- ◆ 手工指定纹理坐标

- 考虑由参数方程定义的曲面

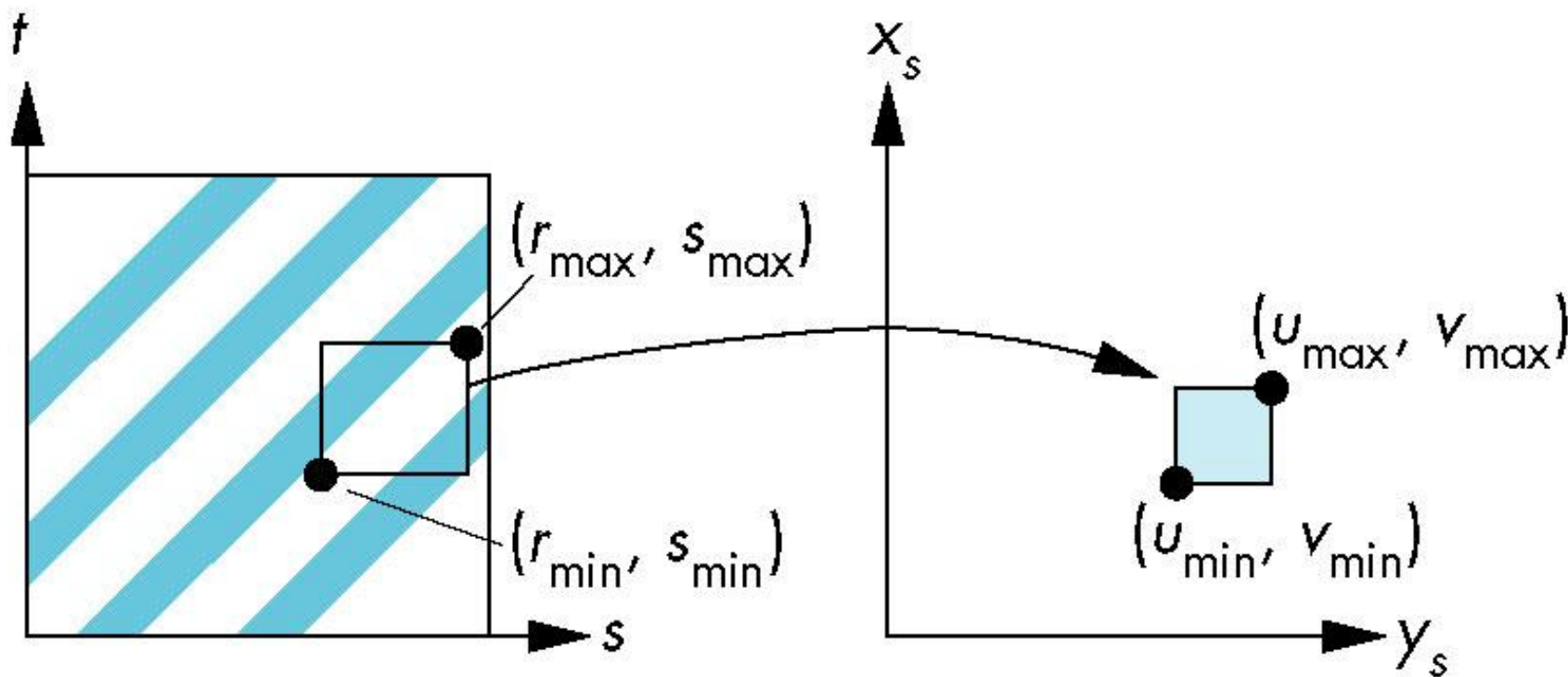
$$p(u, v) = (x(u, v), y(u, v), z(u, v))$$

- 此时通常采用如下形式从纹理元素对应到曲面上的点

$$u = as + bt + c, v = ds + et + f$$

- ◆ 只要 $ae \neq bd$, 上述映射是可逆的

映射的确定

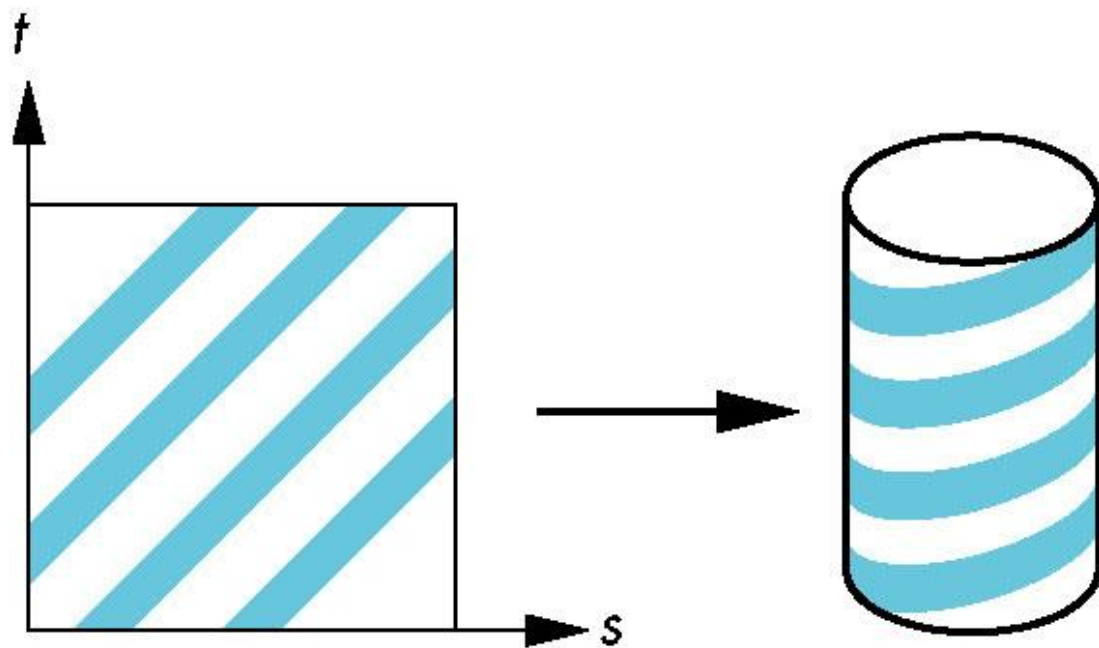


$$u = u_{\min} + \frac{s - s_{\min}}{s_{\max} - s_{\min}} (u_{\max} - u_{\min})$$

$$v = v_{\min} + \frac{t - t_{\min}}{t_{\max} - t_{\min}} (v_{\max} - v_{\min})$$

两步映射

- 解决映射问题的另外一种方法
- 首先把纹理映射到一个简单的中间曲面上
- 例如：映射到圆柱上



圆柱映射

- 假设纹理坐标在单位正方形 $[0,1]^2$ 内变化, 圆柱高 h , 半径 r
- 那么圆柱的参数方程为

$$\begin{aligned}x &= r \cos(2\pi s), \\y &= r \sin(2\pi s), \\z &= ht\end{aligned}$$

- 从纹理坐标到圆柱面上没有变形
- 适合于构造与无底的圆柱面拓扑同构的曲面上的纹理

球映射

- 球的参数方程

$$\begin{aligned}x &= r \cos(2\pi s), \\y &= r \sin(2\pi s) \cos(2\pi t), \\z &= r \sin(2\pi s) \sin(2\pi t)\end{aligned}$$

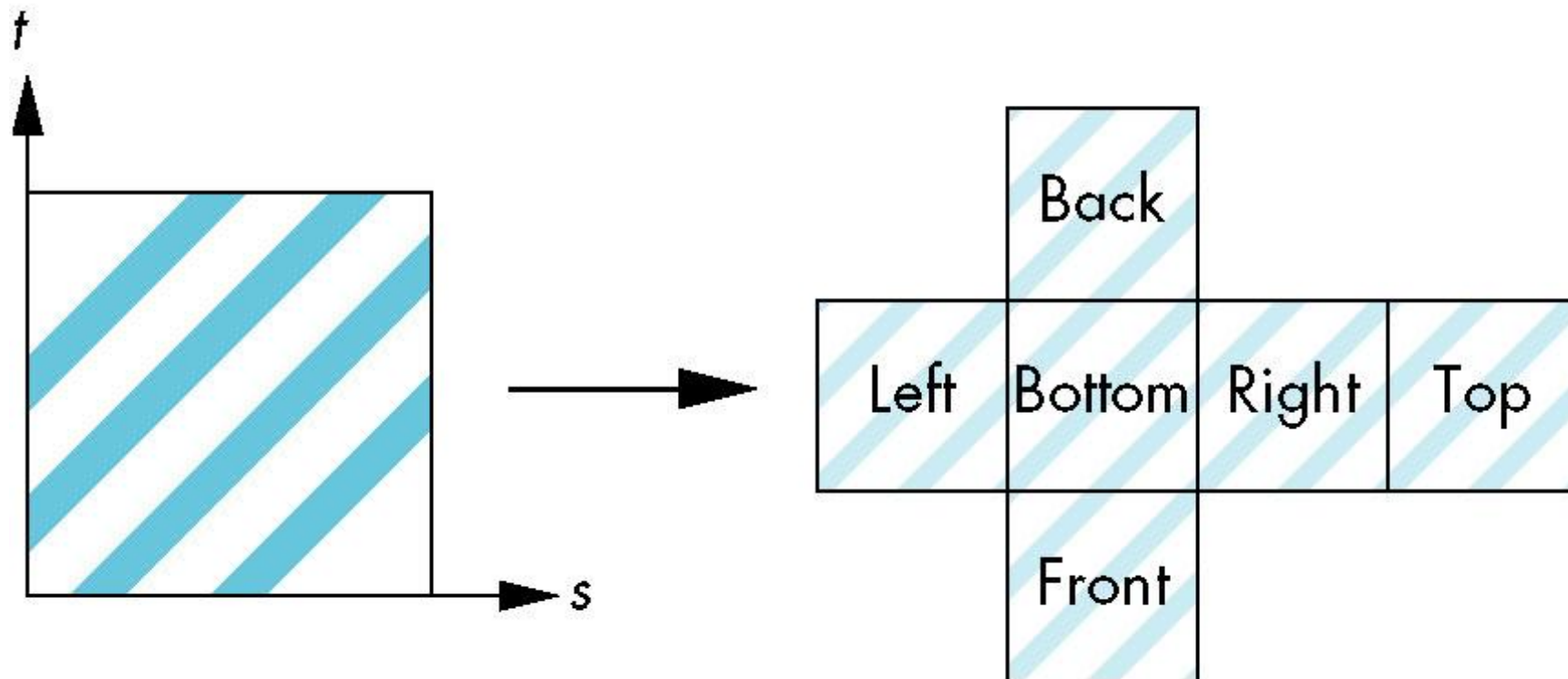
- 类似于地图绘制中的映射

 - ◆ 肯定有变形

- 用在环境映射中

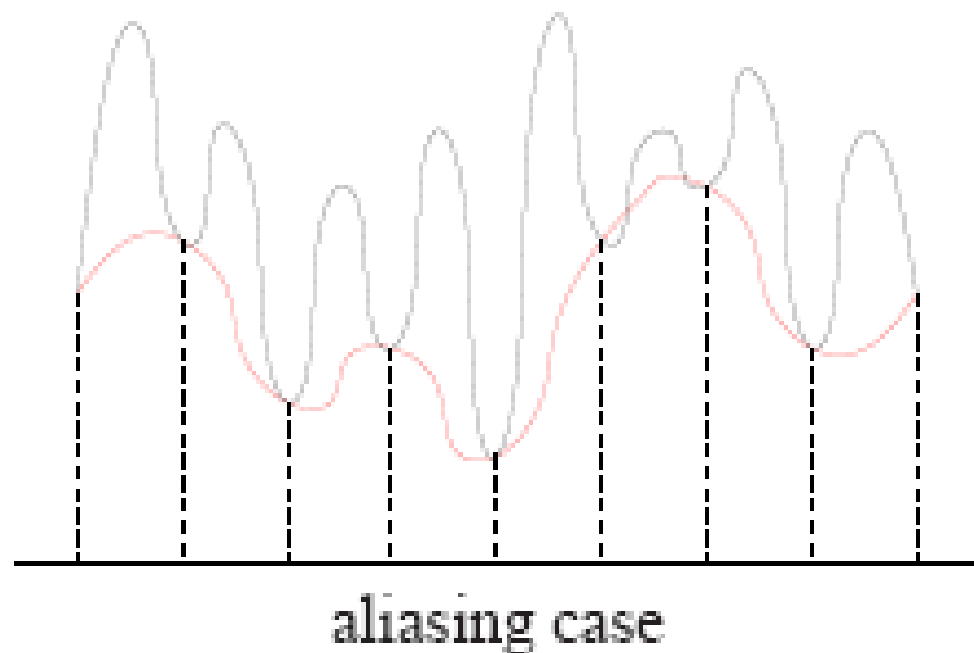
立方体映射

- 适合应用于正交投影
- 也用在环境映射中



纹理滤波

- 走样(Aliasing)和反走样(Anti-aliasing)
- 通常情况下，走样是由于信号**采样频率过低**产生的。因而导致信号的很多高频特征丢失。
- 例如：
 - 右图中黑线表示原始信号，
 - 红线表示走样的信号



纹理滤波

- 要解决这个问题，通常的方法是提高采样率。然而，该方案并不总是可行。
- 例如，在图形学应用中，屏幕分辨率限制了采样频率。
- 一个替代方法是预先对信号进行滤波，得到一个更低频率的信号。

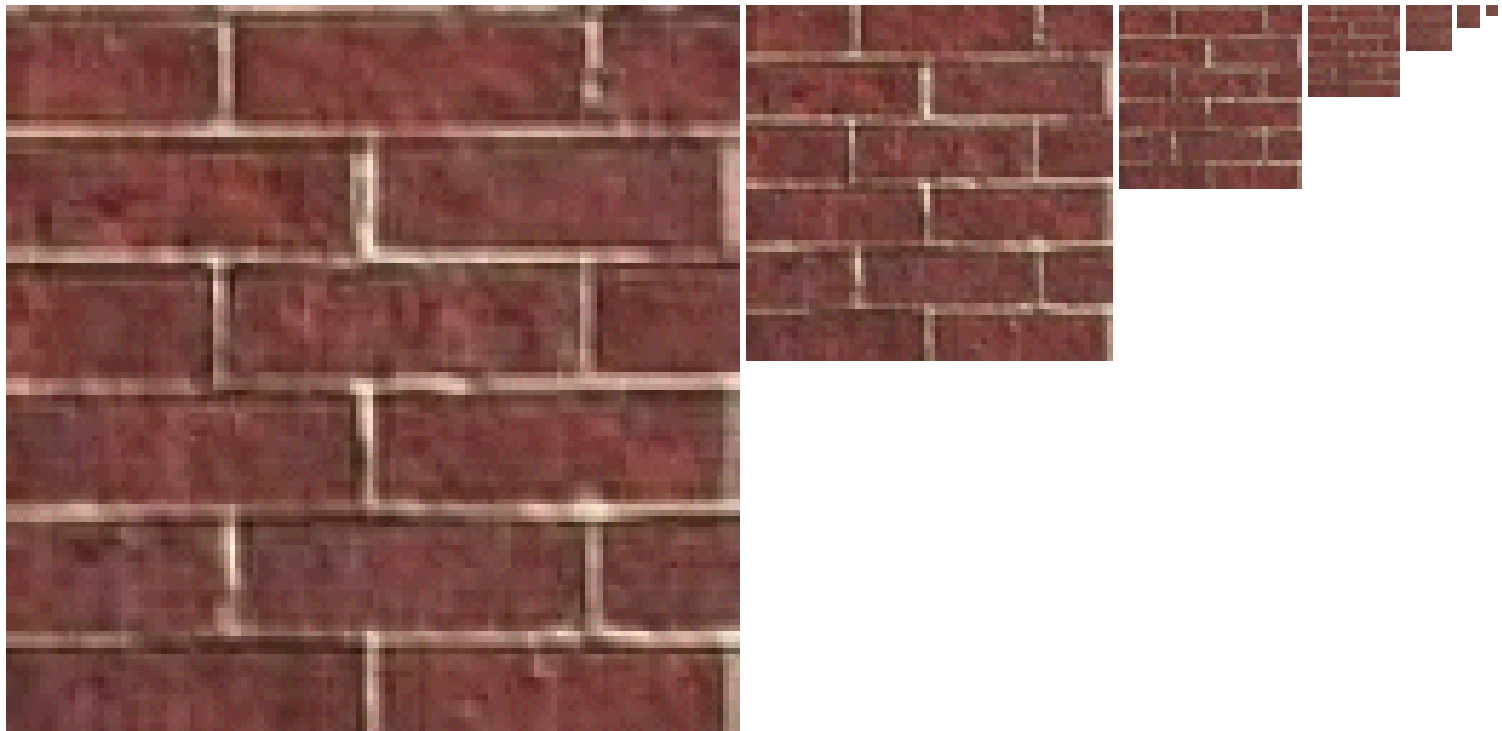
纹理滤波

- **各向同性的 (Isotropic) 滤波**

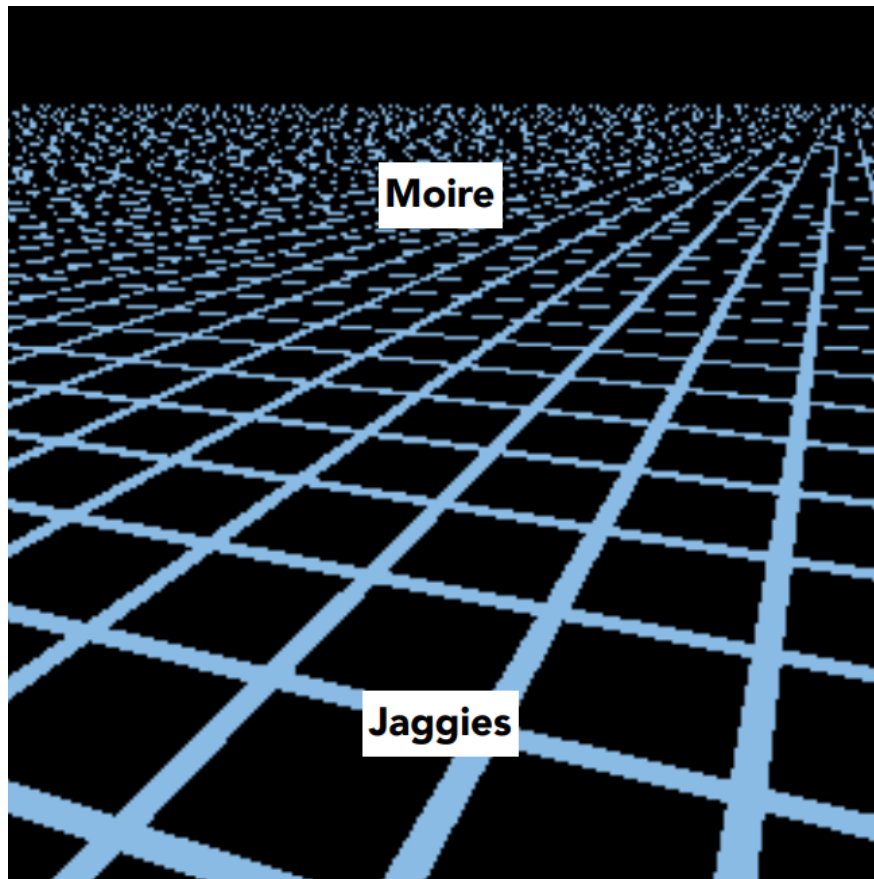
- 材质转换(Mipmapping):构造由一组经过预滤波和重采样图像构成的图像金字塔。这些图像是原图在 $1/2$, $1/4$, $1/8$ 等尺度对应的采样。在光栅化过程中, 我们计算与期望采样率最相近的图像的序号。

纹理滤波

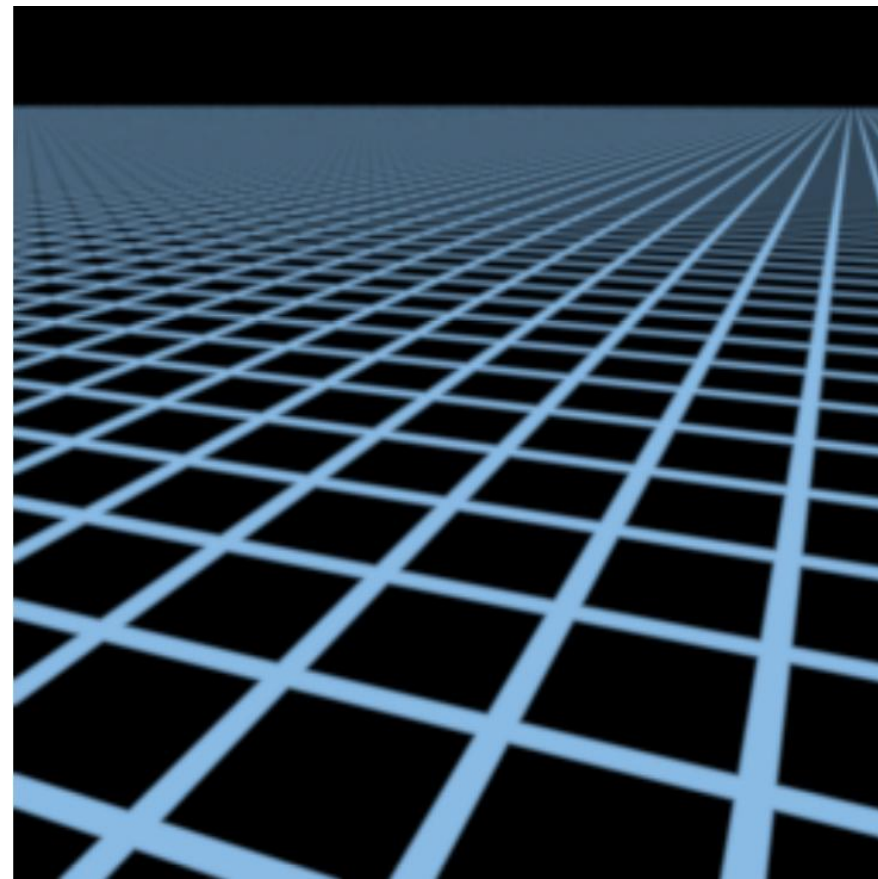
■ 材质转换(Mipmapping)示例



有无MipMap对比



无mipmap

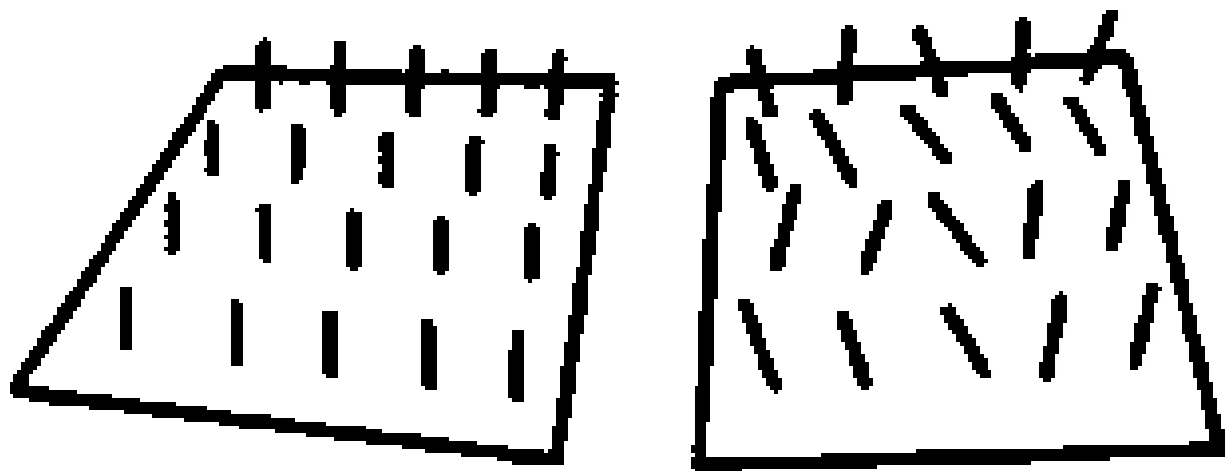


有mipmap

Part 3: 高级主题

法向

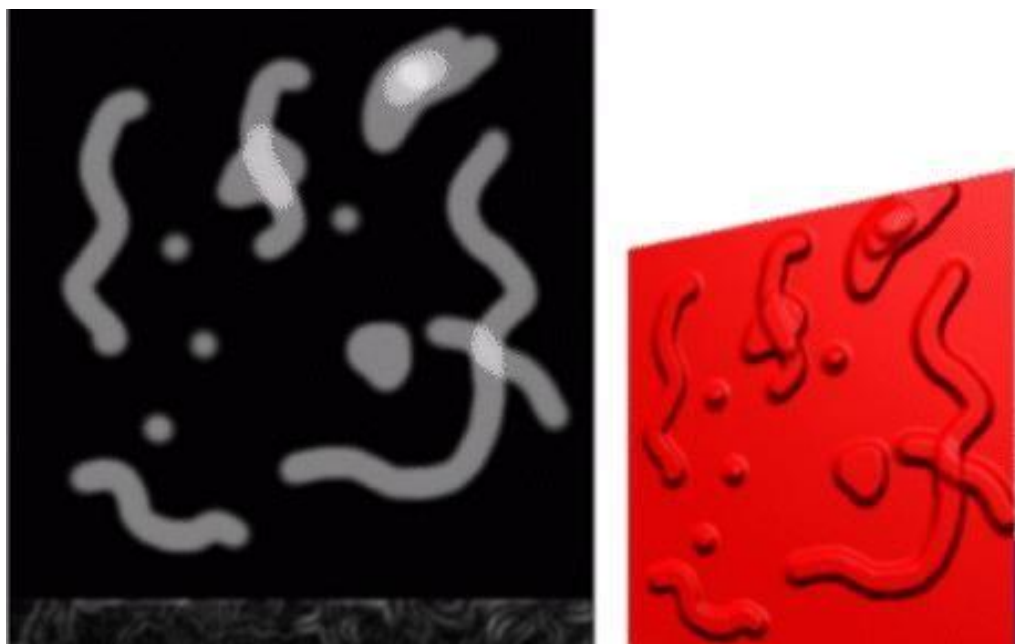
- 曲面上一点的法向量刻画曲面在该点的形状
- 如果用少量扰动曲面法向，那么就会得到具有小变化的曲面
- 如果在生成图像时进行这种扰动，那么就会从光滑的模型得到具有复杂表面模型的图像



凹凸贴图

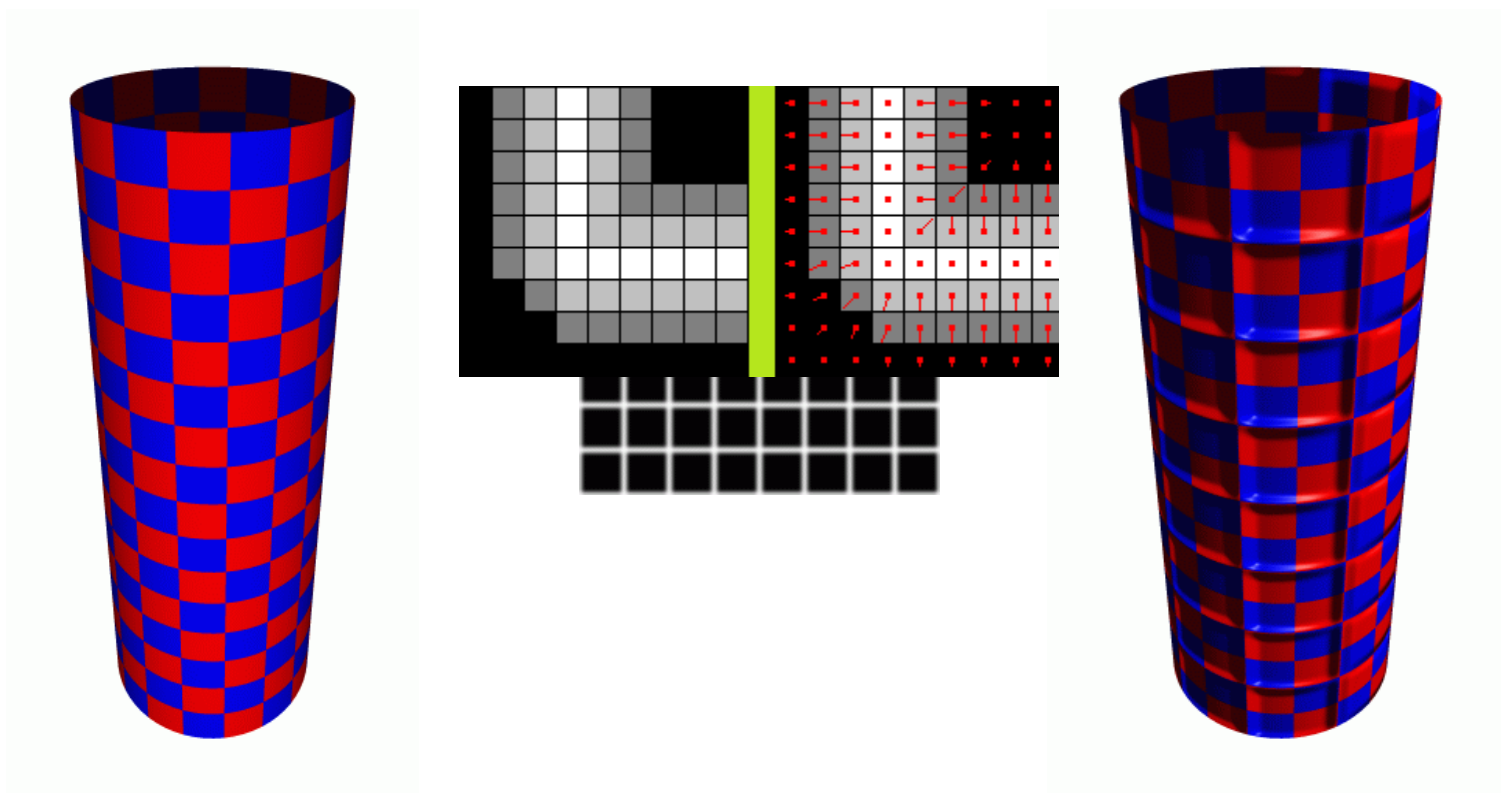
- 把纹理图像当成法向方向上位置的扰动的高度方程

$$\tilde{p}(u,v)=p(u,v)+F(u,v)*N(u,v)$$



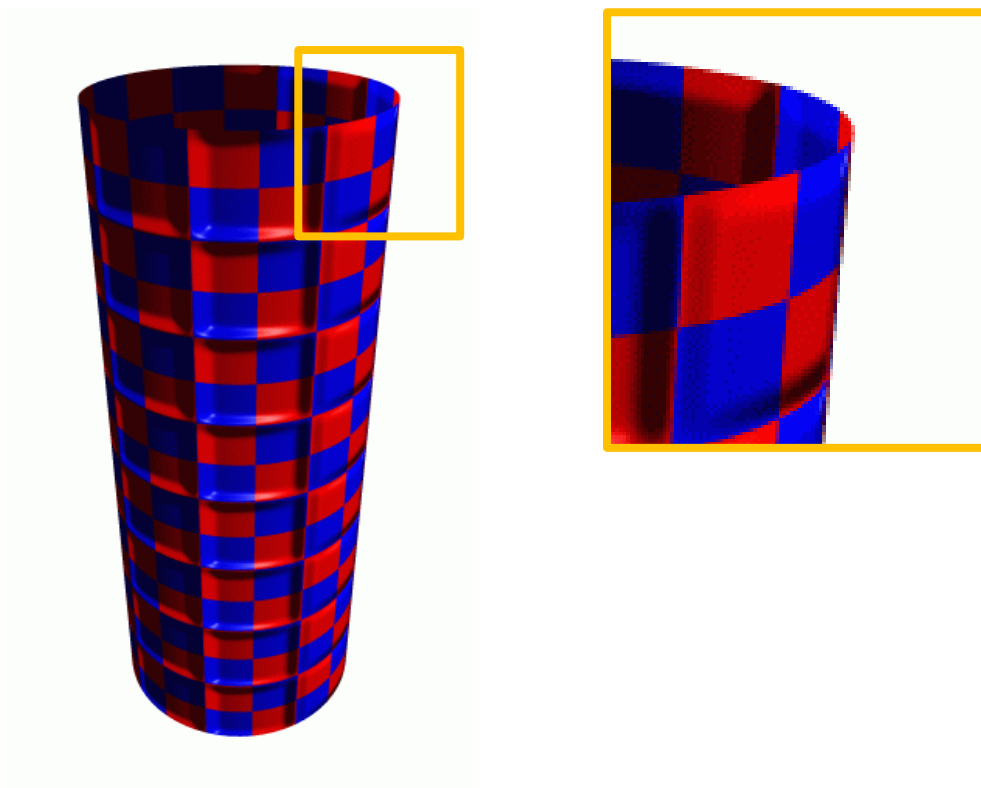
凹凸贴图

■ 另一个凹凸贴图(Bump Mapping)示例



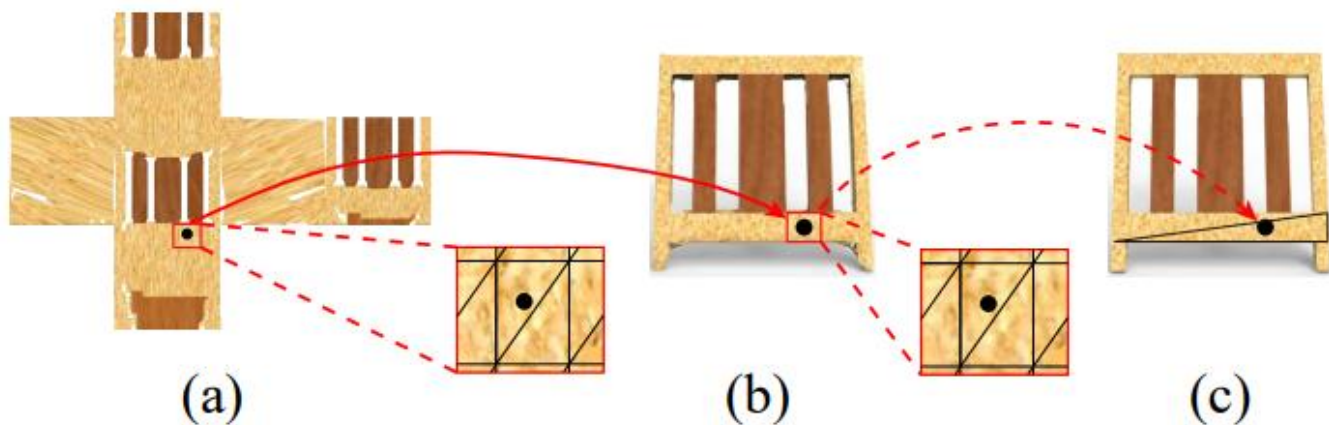
凹凸贴图

- 不能产生侧面轮廓 (silhouette) 效果, 它不支持自遮挡或者自阴影

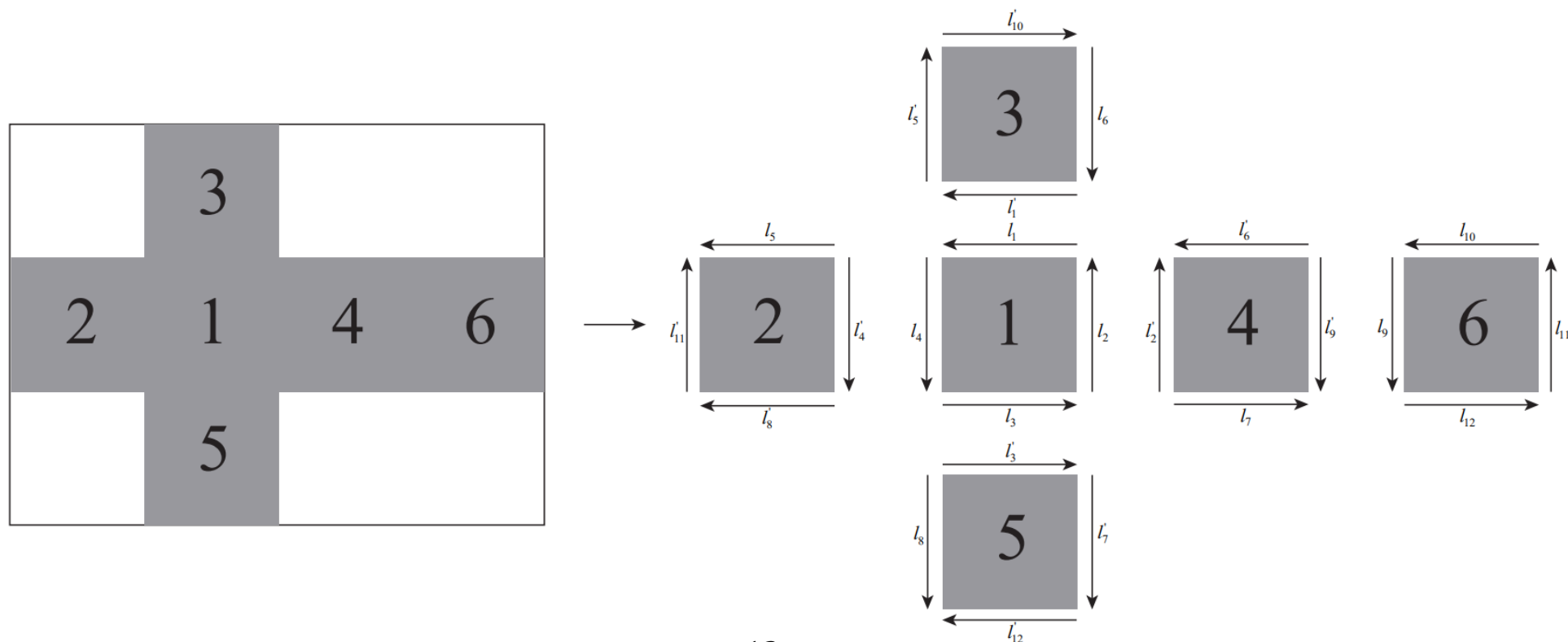


高质量纹理模型生成网络TM-Net

- 包围盒变形表示几何，包围盒展开表示纹理映射。



边缘损失示意



$$L_{seam} = \sum_{i=1}^{12} L1Loss(l_i - l'_i)$$

纹理三维模型生成



谢谢！ 欢迎交流！

高林

gaolin@ict.ac.cn